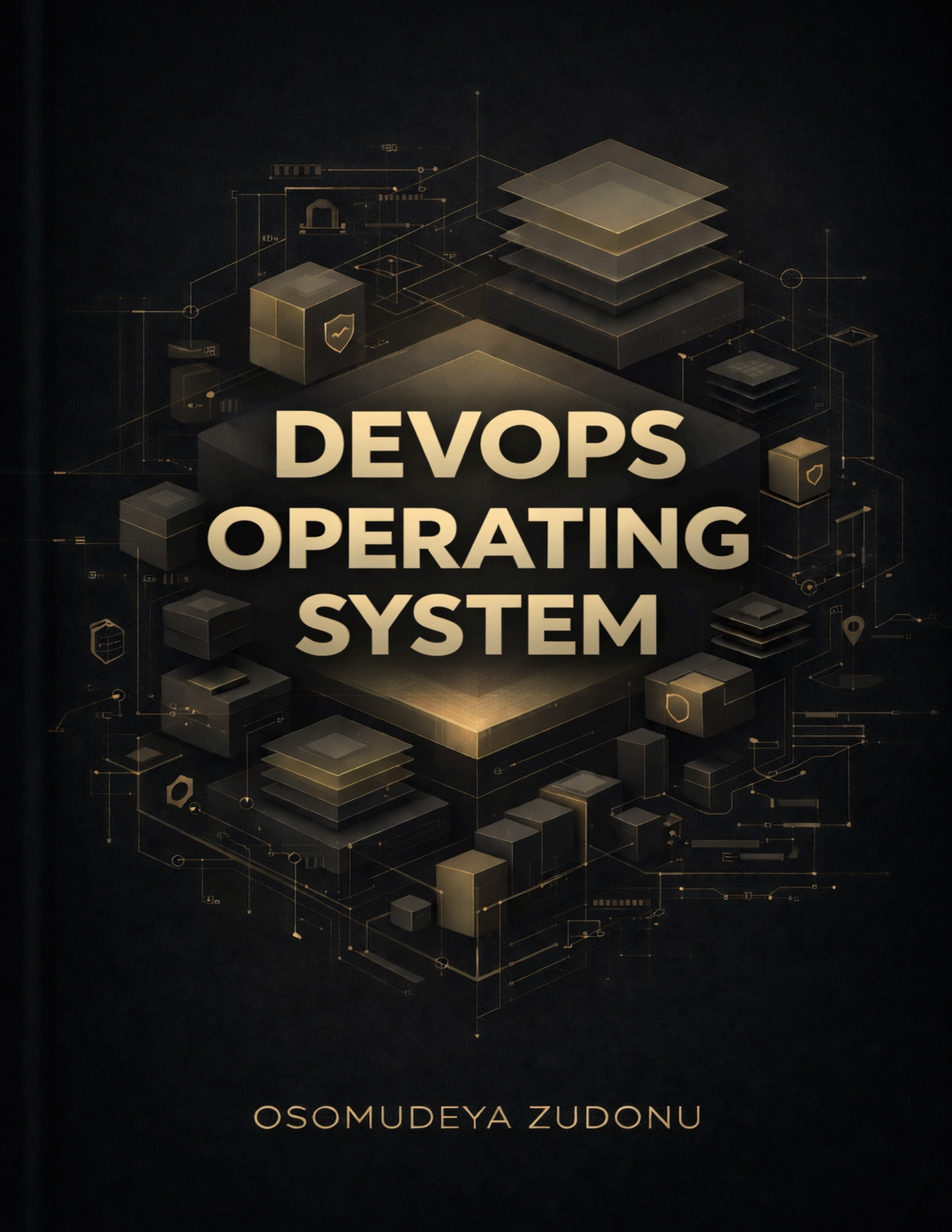




DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

SECTION 3

Docker and Containers

Why You Need Docker Right Now

You've built the H-Game application. But there's a problem...

- **Monday:** Works perfectly on your EC2 instance
- **Tuesday:** Deploy to production server, nothing works
- **Error:** "Module not found," "Wrong Node.js version," "Port conflict"

The problem? Your production server has:

- Different Node.js version (14 vs 18)
- Different system libraries
- Different configurations
- Different dependencies

"But it worked on my machine!"

This is the **#1** problem in software deployment, and Docker solves it.

With Docker:

- Package your app with exact dependencies
- Same container runs on any server
- No more "works on my machine"
- Deploy in seconds, not hours

Every company uses containers, and every DevOps engineer must know Docker.

3.0: Docker Environment Setup

Difficulty: Beginner

Important Notice Before Moving On: IDE Options for VM-Based Learning

You can use your favorite IDE even when working on a VM.

Option 1: Remote SSH (Recommended)

Use VSCode or Cursor to connect directly into your EC2 VM and code with full IDE features.

VSCode Remote SSH Setup (Mac, Windows, Linux)

1. Install Remote SSH extension
 - Open VSCode
 - Go to Extensions
 - Windows/Linux: **Ctrl + Shift + X**
 - Mac: **Cmd + Shift + X**
 - Search: **Remote - SSH**
 - Install the one by Anysphere
2. Open the Command Palette
 - Windows/Linux: **Ctrl + Shift + P**
 - Mac: **Cmd + Shift + P**
3. Connect to your EC2 instance
 - Open Command Palette
 - Type: **Remote-SSH: Connect to Host**
 - Select Add New SSH Host
 - Enter: **ssh devops-lab** (or your own alias)
 - Or enter directly: **ubuntu@YOUR_EC2_PUBLIC_IP**
 - Choose the SSH config file (usually **~/.ssh/config**)
4. Configure SSH (if not done already)

```
# Edit your SSH config
vim or nano ~/.ssh/config
```

Add:

```
Host devops-lab
```



```

HostName YOUR_EC2_PUBLIC_IP
User ubuntu    # or ec2-user for Amazon Linux
IdentityFile ~/.ssh/your-key.pem

```

Save and exit. :wq

5. Connect to the VM
 - Open Command Palette:
 - Mac: **Cmd + Shift + P**
 - Win/Linux: **Ctrl + Shift + P**
 - Select: **Remote-SSH: Connect to Host**
 - Choose: **devops-lab**

VSCode will:

- Open a new window connected to the remote server
- Show a remote file explorer
- Allow full IntelliSense, debugging, Git, terminal, etc.

You're now coding inside your VM using your local VSCode UI.

For Cursor Editor Users

(Cursor is VSCode-based)

- Install Remote - SSH extension
- Follow the exact same steps
- Use **Cmd + Shift + P** to open the command palette
- Everything works identically

Benefits of Remote SSH IDE

- Full autocomplete & IntelliSense
- Debuggers work remotely
- Git integration out of the box
- Less SSH terminal frustration
- Cleaner workflow than editing files in nano/vim

This is the best option for beginners.

Option 2: Local Development (this is what I would be using)

- Write code on your local machine
- Build Docker images locally
- Deploy to the VM when ready

Best for: faster iteration, local testing.

Option 3: Cloud IDEs

Tools like:

- GitHub Codespaces
- Gitpod
- Replit

Best for: no local setup, browser-only workflow.

For This Course

All commands work the same whether you use:

- Terminal + vim / nano
- VSCode Remote SSH (recommended)
- Cursor Remote SSH
- Cloud IDEs
- Local development

The environment doesn't matter, the DevOps concepts and commands remain identical

Option 4: SSM Manager ([click link to setup](#))

Let's Get Back to Setting Up Our Docker Environment

Step 1: Install Docker on EC2

Connect to your EC2 instance:

```
ssh devops-lab
```

Or:

```
ssh -o ServerAliveInterval=120 -o ServerAliveCountMax=10 -i  
~/.ssh/devops-lab-key.pem ubuntu@98.92.52.87 # (YOUR OWN IP)
```

Update system packages:

```
sudo apt update  
  
Install Docker using the official convenience script:  
# Download Docker installation script  
curl -fsSL https://get.docker.com -o get-docker.sh  
  
# Run installation script  
sudo sh get-docker.sh  
  
# Clean up  
rm get-docker.sh
```

Expected output:

```
Executing docker install script...  
+ sudo -E sh -c apt-get update -qq >/dev/null  
+ sudo -E sh -c apt-get install -y -qq docker-ce docker-ce-cli  
containerd.io  
...  
To run Docker as a non-privileged user, consider adding your user to the  
docker group
```

Verify installation:

```
docker --version
```

Expected output:

```
Docker version 24.0.7, build afdd53b  
✓ Success! Docker is installed.
```

Step 2: Configure Docker Permissions

Problem: Docker requires root by default (annoying to type **sudo** constantly)

Root is the administrator account on Linux that can change anything on the system. **sudo** (“super user do”) lets you run a single command with root privileges, so instead of logging in as root, you type **sudo** before commands that need special permissions.

Docker needs root because it manages system-level resources like networks and storage, which is why you often see **sudo docker ...** commands.

Solution: Add your user to the **docker** group

```
# Add current user to docker group  
sudo usermod -aG docker $USER  
  
# Apply group membership for current session  
newgrp docker  
  
# Note: For permanent effect, logout and login again  
# But newgrp works for this session
```

Test you can run Docker without sudo:

```
docker ps
```


Expected output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					

An empty list is correct, you don't have any containers yet.

Step 3: Start Docker Service

Start Docker service:

```
sudo systemctl restart docker.socket
sudo systemctl start docker

# Enable Docker to start on boot
sudo systemctl enable docker

# Verify service is running
sudo systemctl status docker
```

Expected output: **active** (**running**) in green.

Step 4: Run Your First Container

```
docker run hello-world
```

Expected output:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest:
sha256:d58e752213a51785838f9eed2b7a498ffa1cb3aa7f946dda11af39286c3db9a9
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

```
To generate this message, Docker took the following steps:
```

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

What just happened? (Important to understand)

1. Docker looked for **hello-world** image locally (didn't find it)
2. Downloaded image from Docker Hub (public registry)
3. Created container from image (running instance)
4. Ran the container (executed the program)
5. Container printed message and exited (job complete)

This is the basic Docker workflow you'll use constantly.

The Core Mental Model (Read This Once, Use It Forever)

```
An image is a blueprint (a packaged filesystem + metadata).  
A container is a running instance of that blueprint (a process with  
isolation).
```

If you remember only one thing from Docker, remember that.

Mini-Lab: Build Your First Docker Image (A Real Dockerfile)

Running `hello-world` proves Docker is installed. This lab is where Docker “clicks” — you will build an image you created, then run it as a container.

1) Create a tiny Node app

```
mkdir -p dockerfile-lab && cd dockerfile-lab  
# or create the file in cursor or vscode since you are using SSH
```

Create `package.json`:

```
cat > package.json <<'EOF'  
{  
  "name": "dockerfile-lab",  
  "version": "1.0.0",  
  "main": "server.js",  
  "scripts": {  
    "start": "node server.js"  
  }  
}  
EOF
```

Create `server.js`:

```
cat > server.js <<'EOF'  
const http = require("http");  
  
const port = process.env.PORT || 3000;  
  
http  
  .createServer((req, res) => {  
    res.writeHead(200, { "Content-Type": "text/plain" });  
    res.end("Hello from inside a container!\n");  
  })  
  .listen(port, () => {
```

```
    console.log(`Server running on port ${port}`);  
  });  
EOF
```

2) Create a minimal Dockerfile

Create **Dockerfile**:

```
cat > Dockerfile <<'EOF'  
FROM node:18-alpine  
  
WORKDIR /app  
  
COPY package.json ./  
RUN npm install --omit=dev  
  
COPY . .  
  
EXPOSE 3000  
CMD ["npm", "start"]  
EOF
```

3) Build the image, then run the container

```
# Build an image (blueprint)  
docker build -t dockerfile-lab:v1 .  
  
# Run a container (a running instance)  
docker run --rm -p 3000:3000 dockerfile-lab:v1
```

In another terminal, test it:

```
curl http://localhost:3000
```

Expected output:

```
Hello from inside a container!
```

Takeaway: You just created a reusable artifact (image) that runs the same way anywhere you have Docker.

Verify Docker is working:

```
# Check Docker system information
docker info

# Check running containers
docker ps

# Check all containers (including stopped)
docker ps -a
```

You should see the **hello-world** container in stopped state.

Step 5: Test Docker Commands

```
# Check Docker storage usage
docker system df

# View Docker networks
docker network ls

# View Docker volumes
docker volume ls

# View downloaded images
docker images
```

You should see the **hello-world** image listed.

Environment Setup Complete

You now have:

- Docker Engine installed and running
- User permissions configured (no sudo needed)

- First container successfully run
- Docker commands working properly

Mini-Lab: Force a Failure (Environment Drift) — Then Fix It with Docker

This is the “Monday vs Tuesday” problem, recreated on purpose.

1) Run your app with the wrong runtime (it breaks)

From inside `dockerfile-lab/`:

```
docker run --rm -p 3000:3000 -v "$PWD":/app -w /app node:14-alpine  
node server.js
```

If you use a newer Node feature later in the course, Node 14 will break in confusing ways.

The point is: runtime drift is real.

2) Run with the correct runtime (it works)

```
docker run --rm -p 3000:3000 -v "$PWD":/app -w /app node:18-alpine  
node server.js
```

Takeaway: Docker lets you pin the exact runtime and stop “it worked yesterday” surprises.

Let’s learn when and why to use Docker.

3.1: When You Use Docker

Difficulty: Beginner

Docker isn't just a tool you use occasionally. It's part of your daily workflow.

Here are 6 real world scenarios where you'll use Docker constantly:

Scenario 1: Deploying Applications (Daily)

9:00 AM, New feature is ready for production.

Without Docker (the old painful way):

```
# On production server:
ssh production-server
git pull origin main
npm install # Hope no dependency conflicts
npm run build # Hope it builds
pm2 restart app # Hope it starts

# Check logs... something broke 😱
# Spend 2 hours debugging environment differences
```

With Docker (the modern way):

```
# Build container locally (exact same environment)
docker build -t humor-game:v2 .

# Test locally first
docker run -p 8080:8080 humor-game:v2
# Works perfectly!

# Push to registry
docker push humor-game:v2

# Deploy to production (same container!)
ssh production-server
docker pull humor-game:v2
```

```
docker-compose up -d
```

```
# Works exactly like local - 5 minutes total
```

Scenario 2: Team Development Consistency (Daily)

10:30 AM, New developer Sarah joins the team.

Without Docker:

“Sarah, please install:”

- Node.js 18.17.0 (not 18.16 or 18.18!)
- PostgreSQL 15.2 with these exact settings...
- Redis 7.0.8 configured with...
- These 47 system libraries...
- Set these environment variables...
- Pray everything works together...

4 hours later:

- Sarah: “I’m getting weird errors...”
- Senior Dev: “But it works on my machine 🙄”

With Docker (the 5 minute setup):

```
# Sarah's first day:
```

```
git clone h-game-repo
```

```
cd h-game
```

```
docker-compose up
```

```
# Everything works immediately
```

```
# Same environment as everyone else
```

```
# Sarah is productive in 5 minutes, not 4 hours
```

Scenario 3: Microservices Architecture (Weekly)

You're building a complex e-commerce system:

- **Frontend:** React 18
- **API Gateway:** Node.js 20
- **Auth Service:** Python 3.11
- **Order Service:** Go 1.21
- **Payment:** Java 17
- **User Service:** .NET 8
- **Database:** PostgreSQL 15
- **Cache:** Redis 7
- **Message Queue:** RabbitMQ

Without Docker (Infrastructure Struggle):

- **Problem:** Can't run all on one machine (version conflicts)
- **Solution:** Need 9 different servers
- **Cost:** \$3,000+ per month
- **Testing:** Can't test full system locally
- **Debugging:** Log into 9 different servers

With Docker (elegant solution):

```
docker-compose up
```

All 9 services running on one laptop:

- Each in isolated container
- No version conflicts
- Full system testing locally
- One command to rule them all

Scenario 4: CI/CD Pipelines (Every Commit)

It's 2:00 PM, You push code to GitHub.

Don't worry if this is new: You don't need to know GitHub Actions or Kubernetes yet.

Focus on the idea that the same Docker image is built once and then promoted through testing → staging → production.

We'll go deep on Kubernetes later -- here it's just showing where Docker fits in the bigger DevOps pipeline.

Your automated CI/CD pipeline:

```
name: Deploy H-Game
on: [push]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Build Docker image
        run: docker build -t humor-game:${{ github.sha }} .

      - name: Run tests in container
        run: docker run --rm humor-game:${{ github.sha }} npm test

      - name: Scan for vulnerabilities
        run: trivy image humor-game:${{ github.sha }}

      - name: Push to registry
        run: docker push humor-game:${{ github.sha }}

      - name: Deploy to staging
        run: kubectl set image deployment/humor-game
        app=humor-game:${{ github.sha }}

      - name: Run integration tests
        run: ./test-staging.sh
```

```
- name: Deploy to production (same image!)
  run: kubectl set image deployment/humor-game
app=humor-game:${{ github.sha }}
```

Key insight: The same Docker image runs identically in development, testing, staging, and production, so if it works in one environment, it works in all of them, and there are no more “works on my machine” bugs.

Scenario 5: Multi-Environment Management (Daily)

You need to test different configurations:

```
# Development environment (your laptop)
docker run -e NODE_ENV=development -e DEBUG=true -e DB_HOST=localhost
humor-game

# Testing environment (CI server)
docker run -e NODE_ENV=test -e DEBUG=false -e DB_HOST=test-db
humor-game

# Staging environment (pre-production)
docker run -e NODE_ENV=staging -e DEBUG=false -e DB_HOST=staging-db
humor-game

# Production environment (live users)
docker run -e NODE_ENV=production -e DEBUG=false -e
DB_HOST=prod-db-cluster humor-game
```

You use the same Docker container image everywhere, but change environment variables (like `NODE_ENV` and `DB_HOST`) to match each environment, so one image works in development, testing, staging, and production, just with different settings.

Scenario 6: Legacy Application Management (Monthly)

Problem: Old app needs Node.js 12 (security risk), new app needs Node.js 20.

Without Docker (impossible situation):

- Can't run both Node 12 and Node 20 on same server
- Version managers (nvm, n) are hacky and unreliable
- One app breaks the other
- Need separate expensive servers
- Old app blocks new development

With Docker (elegant isolation):

```
# Legacy app safely contained with Node.js 12
docker run -p 3000:3000 --name legacy-app legacy-app:node12

# New app with modern Node.js 20
docker run -p 3001:3001 --name new-app new-app:node20

# Both running safely on same server
# Completely isolated from each other
# No version conflicts
# Old app can't affect new app
```

This means you can run the legacy Node 12 app and the new Node 20 app side by side on the same server by putting each in its own Docker container with different port mappings, so they're fully isolated and never conflict.

The example uses different host ports (3000 and 3001) to expose each container separately; the isolation comes from both separate containers and distinct port mappings.

The Daily DevOps Pattern

Every morning, DevOps engineers run a small set of Docker commands to check the health of their systems, deploy changes, and quickly fix issues.

These commands become muscle memory:

```
# See what containers are running
docker ps

# Follow logs and spot issues early
docker logs backend -f

# Deploy a new version of an app
docker pull myapp:latest
docker-compose up -d

# Restart a service that's misbehaving
docker-compose restart api

# Enter a container to debug issues
docker exec -it backend bash
```

Very soon, Docker commands feel as natural as `ls`, `cd`, and `grep` in Linux.

3.2 Why Docker Matters

Industry Reality Check

Docker didn't just become popular, it became the standard.

- Many industry surveys report that most companies run containers in production
- Docker is listed as a requirement in a large majority of DevOps job descriptions
- Docker Hub hosts millions of images
- Most large enterprises use containers
- Kubernetes is widely adopted for container orchestration

Note: You cannot get a modern DevOps job without Docker.

The Business Case for Containers

Companies did not switch from VMs to containers because it was trendy.

They switched because it saves:

- Money
- Time
- Downtime
- Developer frustration

Cost Savings (The Game Changer)

Traditional Virtual Machines

```
Each VM runs a full operating system, even if the actual app is small:
```

```
Physical Server (64GB RAM, 16 CPUs)
```

```
├─ VM 1: 8GB RAM, 2 CPUs (Ubuntu OS - 2GB overhead)
```

```
├─ VM 2: 8GB RAM, 2 CPUs (Ubuntu OS - 2GB overhead)
```

```
├─ VM 3: 8GB RAM, 2 CPUs (Ubuntu OS - 2GB overhead)
```

```
└─ VM 4: 8GB RAM, 2 CPUs (Ubuntu OS - 2GB overhead)
```

```
Total usage: 32GB for apps + 8GB wasted on OS
```

Docker Containers

Containers share the host OS, so they consume only what the app needs:

```
Physical Server (64GB RAM, 16 CPUs)
```

```
├─ Container 1: 512MB RAM (20MB overhead)
```

```
├─ Container 2: 512MB RAM (20MB overhead)
```

```
├─ Container 3: 512MB RAM (20MB overhead)
```

```
└─ ... (50-100+ containers easily)
```

```
Total: 50+ apps on the same hardware
```

This is like up to a 70% reduction in infrastructure costs.

Deployment Speed (98% Faster)

Traditional Deployment Flow

1. SSH into server (2 minutes)
2. Stop app (1 minute)
3. Backup current version (3 minutes)
4. Pull new code (2 minutes)
5. Install dependencies (15 minutes)
6. Update config (5 minutes)
7. Restart services (3 minutes)
8. Test functionality (5 minutes)
9. Debug unpredictable issues (30 minutes)

Total: ~66 minutes (if everything goes right)

Docker Deployment Flow

1. Pull new image (30 seconds)
2. Stop old container (2 seconds)
3. Start new container (5 seconds)
4. Health check passes (10 seconds)

Total: ~47 seconds

Everything runs in an identical environment:

- No surprises
- No “works on my machine” issues
- 98% faster deployments

Reliability, Faster Debugging, Less Downtime

Without Docker

Production issue: “The website is down!”

Debugging looks like this:

- Which server is failing? (5 minutes)
- SSH into it (security risk)
- Logs are scattered everywhere
- Production environment differs from development
- Bug cannot be reproduced locally
- Fix attempt #1 works locally but breaks production
- Fix attempt #2 works but causes another issue

Total downtime: 3 hours

Cost: ~\$50,000 for a mid size business

With Docker

Production issue: “The website is down!”

Debugging becomes simple:

```
docker pull production-image
docker run production-image
```

You are now running the exact same environment as production.

Reproduce, fix, test, push, deploy, all consistent.

Total downtime: 14 minutes

Cost: ~\$1,000

95% faster incident resolution.

Technical Benefits Explained

1. Speed Comparison

```
# Start a VM
vagrant up
# Takes 3-5 minutes

# Start a container
docker run nginx
# Takes 1-2 seconds
```

Why so fast? Containers don't boot an OS, they just start a process.

2. Resource Efficiency

Running 10 web apps:

- **VMs:**
 - 10 VMs × 2GB = 20GB
 - OS overhead = 2GB
 - Total = 22GB RAM

- **Containers:**

- 10 containers × 100MB = 1GB
- Shared OS = 200MB
- Total = ~1.2GB RAM

95% less memory usage.

3. True Portability

Build once, run anywhere:

```
# Build on macOS (ARM)
docker build -t myapp:v1 .

# Run on Windows (x86)
docker run myapp:v1      # works

# Run on AWS EC2 (Ubuntu)
docker run myapp:v1      # works

# Run on Azure (ARM64)
docker run myapp:v1      # works

# Run on Google Cloud
docker run myapp:v1      # works
```

Same image. Same behavior. No compatibility issues.

4. Real World Dependency Isolation

This is where Docker truly shines.

Without Docker, Everything Conflicts

A company runs 3 apps with different dependency requirements:

App	Needs
App A (Legacy)	Python 2.7, OpenSSL 1.0, Node.js 12
App B (Current)	Python 3.9, OpenSSL 1.1, Node.js 16
App C (Modern)	Python 3.12, OpenSSL 3.0, Node.js 20

On a single server:

- You can only install one version of Python
- You can only install one version of OpenSSL
- You can only install one version of Node.js

So upgrading App C breaks App B. Fixing App B breaks App A. Everything becomes fragile.

With Docker, Each App Gets Its Own World

```
# App A
docker run -d --name app-a legacy-app:python2.7

# App B
docker run -d --name app-b current-app:python3.9

# App C
docker run -d --name app-c modern-app:python3.12
```

All apps now:

- Run independently
- Use different runtimes
- Never conflict
- Share the same server without issues

No dependency hell. No broken apps. No angry stakeholders.

Summary

Docker replaced VMs as the default because it delivers:

- Massive cost savings
- Ultra fast deployments
- Predictable environments
- Near zero downtime
- Full dependency isolation
- True portability across all platforms

This is why every DevOps engineer must master Docker before Kubernetes.

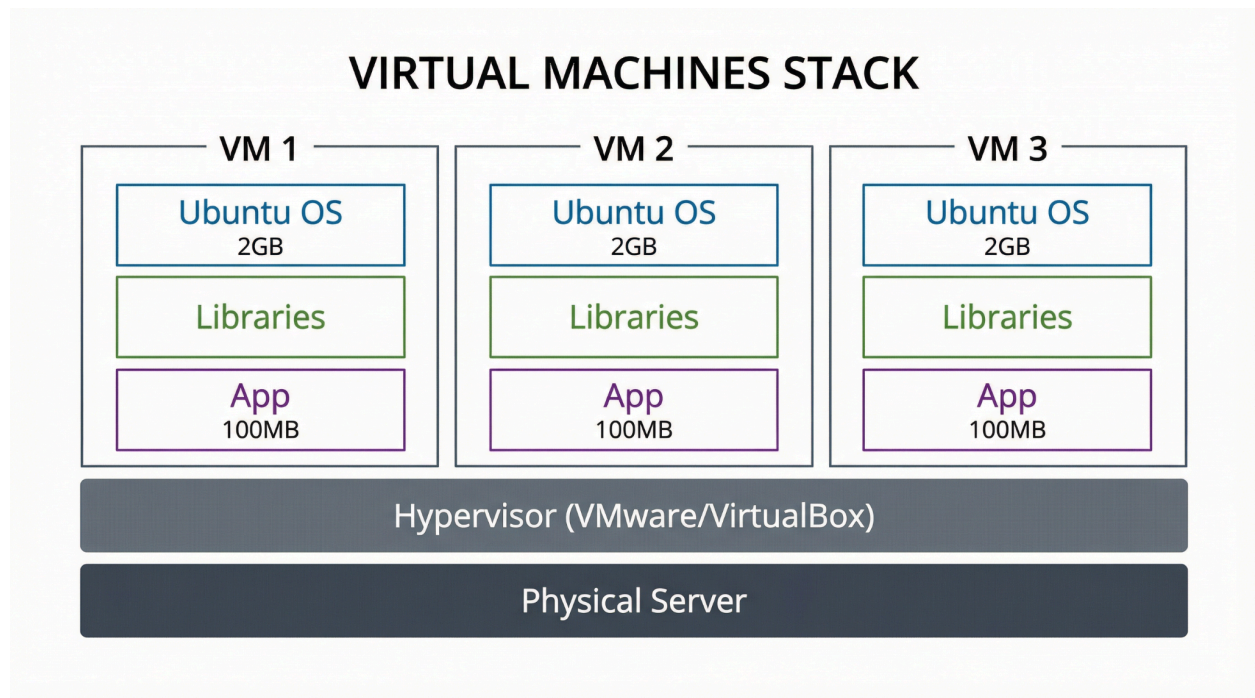
What You Should Now Be Able to Explain

- ☐ Why Docker fixes “works on my machine” (environment drift)
- ☐ The difference between an image and a container
- ☐ What a **Dockerfile** does at a high level
- ☐ Why the same image can flow through dev → CI → staging → prod
- ☐ Why Docker is the foundation before Kubernetes

Containers vs Virtual Machines

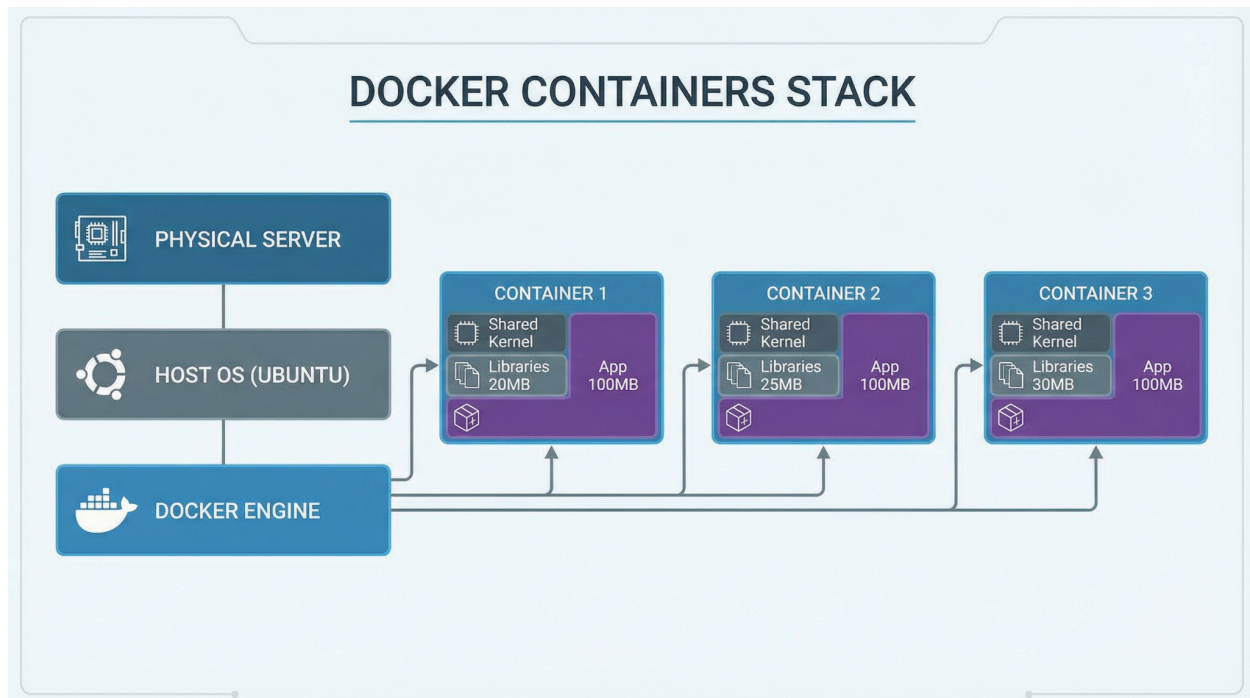
Visual Architecture Comparison:

VIRTUAL MACHINES STACK:



Each VM = Full OS (slow, heavy, expensive)

DOCKER CONTAINERS STACK:



Each container = Just app + libs (fast, light, cheap)

Virtual Machines vs Docker Containers, Detailed Comparison

Startup Time

- **Virtual Machines:** Slow. They take **1–5 minutes** because they must boot an entire operating system.
- **Docker Containers:** Fast. They start in **1–5 seconds** since they only launch a process.

Size

- **Virtual Machines:** Heavy, usually **2–10GB** each because they include a full OS.
- **Docker Containers:** Lightweight, typically **50–200MB** since they only package the app and dependencies.

Isolation

- **Virtual Machines:** Provide **full OS isolation**, each VM behaves like a separate computer.
- **Docker Containers:** Offer **process-level isolation**, sharing the host kernel.

Performance

- **Virtual Machines:** Around **80% of bare-metal** performance due to virtualization overhead.
- **Docker Containers:** Around **99% of bare-metal** performance because they run directly on the host kernel.

Portability

- **Virtual Machines:** Limited. VMs depend on the hypervisor (VMware, VirtualBox, Hyper-V).
- **Docker Containers:** Extremely portable. They **run anywhere**, cloud, on-prem, Windows, Linux, macOS.

Resource Usage

- **Virtual Machines:** Heavy. Each VM requires its own OS, consuming CPU, memory, and disk.
- **Docker Containers:** Light. Containers **share the OS kernel**, using far fewer resources.

Density

- **Virtual Machines:** A single server can run **5–20** VMs.
- **Docker Containers:** The same server can run **100–1000** containers.

Security

- **Virtual Machines:** Very strong, each VM has a **full OS boundary**.
- **Docker Containers:** Secure but lighter, separated at the **process level**.

Management

- **Virtual Machines:** Complex. You must maintain multiple operating systems.
- **Docker Containers:** Simple. Everything runs on **one OS**, orchestrated with tools like Docker Compose or Kubernetes.

Cost

- **Virtual Machines:** High. More OSes → more resources → more cloud/spend.
- **Docker Containers:** Low. Efficient resource usage keeps operational costs down.

Career Impact (Show Me The Money)

DevOps Engineer Salary Data (2025, US Market):

Without Container Skills:

```
|— Junior DevOps: $70,000 - $85,000
|— Mid-Level:      $85,000 - $110,000
|— Senior:         $110,000 - $140,000
```

With Docker/Container Expertise:

```
|— Junior DevOps: $85,000 - $100,000 (+$15k)
|— Mid-Level:      $105,000 - $135,000 (+$25k)
|— Senior:         $135,000 - $180,000 (+$40k)
```

Premium for Container Skills: \$15,000 - \$40,000 per year

Most In Demand DevOps Skills (Job Posting Analysis)

1. **Linux/Unix** - 98% of postings
2. **Docker/Containers** - 95% of postings **Essential**
3. **Kubernetes** - 88% of postings
4. **CI/CD** - 85% of postings
5. **Cloud Platforms** - 82% of postings

Reality check: You cannot skip Docker and expect to compete in the job market.

How to Think Like a Container Engineer

Remember Your H-Game Problem?

You built the H-Game on your laptop. Lets say you test and It worked perfectly! Then you tried to run it on the production server and got these errors:

```
Error: Module 'express' not found
Error: Node.js version mismatch
Error: Port 3000 already in use
```

Why? Because your laptop and the server are different:

- **Your laptop has** Node.js 18, specific npm packages installed
- **But production server has** Node.js 16, different packages, different setup

This is the "**works on my machine**" problem that drives developers crazy.

The Container Solution

Think of containers like **shipping containers** for your code:

- **Regular shipping:** Pack items loose in truck → items get damaged, mixed up, lost
- **Container shipping:** Pack everything in standardized container → arrives exactly as shipped

How Successful Companies Use This

Spotify's transformation:

- **Before containers:** 45 minutes to deploy, once per week
- **After containers:** 30 seconds to deploy, thousands of times per day

But for your H-Game, containers mean:

- Deploy in seconds instead of hours debugging
- Same behavior on your laptop, staging, and production
- New team members can run the app in 5 minutes

The 6 Container Principles

Principle 1: Containers Are Temporary Workers

Think of your laptop: You install software, save files, customize settings over months. You treat it like a **pet**, valuable and irreplaceable.

Think of containers: They're **temporary workers** you hire for a specific job. When the job is done, they leave. You don't get attached to them.

Wrong mindset (treating containers like pets):

```
docker exec -it my-app bash
# "Let me go inside and fix this config file"
nano config.json
# "I'll save this important setting here"
exit

# Later, container restarts...
# Boom!!! Your changes are gone! The container reset itself
```

Right mindset (treating containers as temporary workers):

```
# Edit the source code instead (like giving new instructions)
vim app/config.json # or Using IDE
# Build a new container with the fix
docker build -t my-app:v2 .

# Start a new worker with the correct instructions
docker run my-app:v2

# Your changes are permanent (baked into the blueprint)
```

Key insight: Don't change the worker, update the instruction manual and get a new worker.

Principle 2: Each Container Does One Job

Your H-Game needs several things:

- Web server (serves web pages)
- Application server (runs your Node.js code)
- Database (stores user scores)
- Cache (makes things faster)

Wrong approach (one container does everything):

```
Giant Container:  
├─ Nginx web server  
├─ Node.js app  
├─ PostgreSQL database  
└─ Redis cache
```

Problem: If one thing breaks, everything stops working

Right approach (each container specializes):

```
Container 1: Nginx only (serves web pages)  
Container 2: Node.js only (runs your game logic)  
Container 3: PostgreSQL only (stores data)  
Container 4: Redis only (caches data)
```

Benefits:

- If web server crashes, game logic keeps running
- Need more game servers? Add more Container 2s
- Update the web server? Containers 3&4 keep running

Real example from your H-Game:

High traffic day and you need more backend power

```
docker run --name backend-2 h-game-backend
docker run --name backend-3 h-game-backend
# Frontend and database unaffected

# Problem with frontend - fix without touching backend
docker stop frontend
docker run --name frontend-new h-game-frontend:fixed
# Backend keeps serving API calls
```

Principle 3: Never Edit Running Containers

This is the **golden rule** that beginners break constantly.

Traditional server thinking (wrong for containers):

"Let me SSH into the server and fix this quickly"

```
ssh production-server
sudo vim /etc/nginx/config
sudo service nginx restart
```

Container thinking (correct):

"Let me fix the source and build a new container"

```
vim nginx/config # or Fix Locally

docker build -t nginx:fixed . # Create new version
docker run nginx:fixed # Deploy the fix
```

Why this matters for your H-Game:

- **Traditional way:** Manual changes on production server
 - No record of what changed
 - Can't test the change first
 - Different from your development setup
 - If server dies, changes are lost forever
- **Container way:** All changes go through code
 - Every change tracked in Git
 - Test locally before deploying
 - Identical across all environments
 - Can rebuild server from code anytime

Principle 4: Settings Come From Environment Variables

Problem: Your H-Game needs different settings for different places:

BAD: Settings hardcoded in your code

```
const config = {  
  database: "postgres://localhost:5432/hmgame_dev", // Works on your  
  laptop  
  apiKey: "sk_test_12345", // Test API key  
  debug: true // Show debug info  
}  
  
// This code only works on your Laptop!
```

Solution: Get settings from the environment:

GOOD: Settings from environment variables

```
const config = {
  database: process.env.DATABASE_URL, // Different for each
environment
  apiKey: process.env.STRIPE_API_KEY, // Different API keys
  debug: process.env.NODE_ENV === 'development' // Different debug
settings
}

// Same code works everywhere!
```

Deploy to different environments:

```
# Your laptop (development)
docker run \
  -e DATABASE_URL="postgres://localhost:5432/hmgame_dev" \
  -e STRIPE_API_KEY="sk_test_12345" \
  -e NODE_ENV="development" \
  h-game-backend

# Production server (live users)
docker run \
  -e DATABASE_URL="postgres://prod-db:5432/hmgame" \
  -e STRIPE_API_KEY="sk_live_67890" \
  -e NODE_ENV="production" \
  h-game-backend

# Same container image, different settings
```

Think of it like this: You're giving the same worker different instructions depending on where they're working.

Principle 5: Important Data Lives Outside Containers

Containers are temporary workers, they come and go. Don't store important stuff inside them.

The disaster scenario:

```
# Start database container
docker run --name game-db postgres

# Players create accounts, earn high scores...
# Database stores everything inside the container

# Later, container crashes and gets removed
docker rm game-db

# Start new database container
docker run --name game-db postgres

# 🤖 All player data is gone forever!
# High scores, user accounts - everything lost
```

The safe approach:

```
# Create a permanent storage box
docker volume create game-data

# Run database container connected to storage box
docker run --name game-db \
  -v game-data:/var/lib/postgresql/data \
  postgres

# Even if container dies, data survives in the storage box
docker rm game-db # Safe to remove container
docker run --name game-db-new \
  -v game-data:/var/lib/postgresql/data \
  postgres

# All player data still there!
```

Simple rule:

- **Container:** Stores the program (temporary)
- **Volume:** Stores the data (permanent)

For your H-Game:

- **Container:** Node.js app, game code
- **Volume:** Player scores, user accounts, game statistics

Principle 6: Build Smart for Speed

When you build containers, Docker creates **layers** (like a cake). Smart ordering makes building much faster.

Slow way (rebuilds everything when you change code):

```
FROM node:18
WORKDIR /app
COPY . .                # Copies ALL files
RUN npm install          # Reinstalls everything (5 minutes)
CMD ["npm", "start"]

# Change one line of code → entire npm install runs again
```

Fast way (only rebuilds what changed):

```
FROM node:18
WORKDIR /app
COPY package.json .      # Copy just the dependency list
RUN npm install           # Install dependencies (5 minutes, but
cached)
COPY . .                  # Copy code (5 seconds)
CMD ["npm", "start"]

# Change code → npm install uses cache (5 seconds total)
```

Why this works:

- Docker remembers each step
- If `package.json` hasn't changed → reuse cached dependencies
- Only rebuild when dependencies actually change

For your H-Game, this means:

- **Code changes:** 5-second rebuilds
- **New dependencies:** 5-minute rebuild (but only when needed)

Configuration Without Chaos: Using `.env` Files the Right Way

If containers solve the “*works on my machine*” problem, configuration is where many people accidentally bring it back.

Your application doesn't fail in production because Docker is broken.

It fails because **the same code needs different settings in different places**, and those settings were handled poorly.

This section teaches you how to use `.env` files correctly, why they exist, and where they stop being the right tool.

The Real Problem `.env` Files Solve

Every application needs configuration:

- Ports
- Database credentials
- API keys
- Feature flags
- Environment specific behavior (dev, test, prod)

Beginners usually hardcode these values:

```
const dbUrl = "postgres://localhost:5432/game"
```

This works... until:

- You deploy to production
- You add a second environment
- Another developer joins
- You rotate credentials

Now you're editing code just to change settings.
That's fragile, unsafe, and unscalable.

Configuration must live outside the code.

What a `.env` File Actually Is

A `.env` file is simply a list of environment variables:

```
PORT=3001
DB_HOST=postgres
DB_PASSWORD=secret
```

That's it.

It does not encrypt anything.

It does not secure anything.

It only **loads values into the environment** so your app can read them.

Your application never reads the `.env` file directly.

It reads environment variables.

The Professional Pattern: `.env.example`

The correct approach is **not** to commit your real `.env`.

Instead, teams use:

```
.env.example
```

This file:

- Documents all required configuration
- Shows expected variable names

- Contains safe placeholder values
- Is committed to Git

New developers do this:

```
cp .env.example .env
```

Then they fill in real values locally.

This creates a **clear contract** between the application and its environment.

Reading the Environment File (How to Think About It)

A good `.env.example` file is grouped by responsibility, not randomness.

For example:

- Application configuration
- Database configuration
- Cache configuration
- Security configuration
- Monitoring configuration

This structure is intentional.

It helps you answer questions like:

- “What settings are environment-specific?”
- “What must change between dev and prod?”
- “What belongs in Secrets later?”

The goal is not memorization.

The goal is **clarity**.

How the Application Uses Environment Variables

In Node.js, the application reads configuration like this:

```
require('dotenv').config()

const config = {
  port: process.env.PORT,
```

```
dbHost: process.env.DB_HOST,  
dbPassword: process.env.DB_PASSWORD,  
}
```

Important rules:

- The app reads from `process.env`
- It never assumes default production values
- Missing variables should fail fast

The same code runs everywhere. Only the environment changes.

Using `.env` with Docker (Where It Gets Powerful)

This is where containers and environment variables work together.

You build **one image**:

```
docker build -t h-game-backend .
```

You run it with different configuration:

```
docker run --env-file .env h-game-backend
```

Or with Docker Compose:

```
env_file:  
- .env
```

Nothing is rebuilt. Nothing is edited inside the container.

Same worker. Different instructions.

Critical Warning: `.env` Is Not a Secret Manager

This is where many beginners make dangerous mistakes.

`.env` files are fine for:

- Local development

- Personal projects
- Learning environments

They are **not ideal** for:

- Shared production clusters
- CI/CD pipelines
- Teams with access controls
- Regulated environments

Why?

- Files get copied
- Files get leaked
- Files get committed by mistake

In production, secrets must move elsewhere.

How **.env** Evolves in Real Infrastructure

Professional setups follow this progression:

Environment	Configuration Source
Local dev	<code>.env</code>
Docker	<code>--env-file</code>
CI/CD	Pipeline secrets
Kubernetes	ConfigMaps + Secrets

The **variable names never change**. Only the source changes.

For example:

```
DB_PASSWORD
```

- Local: `.env`
- CI: GitHub Actions secret
- Kubernetes: Secret object

This is why environment variables are powerful, they scale.

Kubernetes Preview (Important Mental Shift)

In Kubernetes, you don't upload `.env` files. You map variables explicitly:

```
env:  
  - name: DB_PASSWORD  
    valueFrom:  
      secretKeyRef:  
        name: db-secrets  
        key: password
```

Same variable name. Different source. Same application.

If your app depends on file paths or hardcoded config, this breaks.

If it depends on environment variables, it scales cleanly.

Common Beginner Mistakes to Avoid

- Committing `.env` to Git
- Hardcoding secrets in code
- Using different variable names per environment
- Editing containers instead of rebuilding
- Treating `.env` as “secure”

Remember:

`.env` improves **configuration clarity**, not security.

Checkpoint: What You Should Understand Now

Before moving on, make sure you understand:

- Why config must live outside code
- What `.env.example` is for
- Why `.env` is environment-specific
- When `.env` stops being enough
- Why environment variable names must stay stable

If this feels new, reread this section slowly.

This mindset underpins **Docker, CI/CD, and Kubernetes**.

Putting It All Together

These principles seem simple, but they are completely different from how traditional servers work.

Traditional thinking: Servers are pets you carefully maintain.

Container thinking: Containers are temporary workers you replace freely.

Once you internalize this shift, you'll understand why companies can deploy hundreds of times per day with confidence, while traditional infrastructures struggle with monthly releases.

Next step: Let's apply these principles to containerize your **H-Game** application.

Container Troubleshooting Mindset

When a container isn't working, think systematically

Container Issue Diagnosis Framework

Step 1: Is the container actually running?

Command: `docker ps`

Look for: Container in list, status "Up"

Step 2: What do the logs say?

Command: `docker logs CONTAINER_NAME`

Look for: Error messages, stack traces, startup issues

Step 3: What's the container configuration?

Command: `docker inspect CONTAINER_NAME`

Look for: Environment variables, volumes, networks

Step 4: Can I get inside to investigate?

Command: `docker exec -it CONTAINER_NAME sh`

Try: Running commands manually, checking files

Step 5: Are resources sufficient?

Command: `docker stats CONTAINER_NAME`

Look for: Memory usage, CPU usage, limits

Step 6: Can containers reach each other?

Command: `docker exec CONTAINER ping OTHER_CONTAINER`

Test: Network connectivity, DNS resolution

Troubleshooting Demo Session:

```
# Issue: "My backend container keeps crashing"

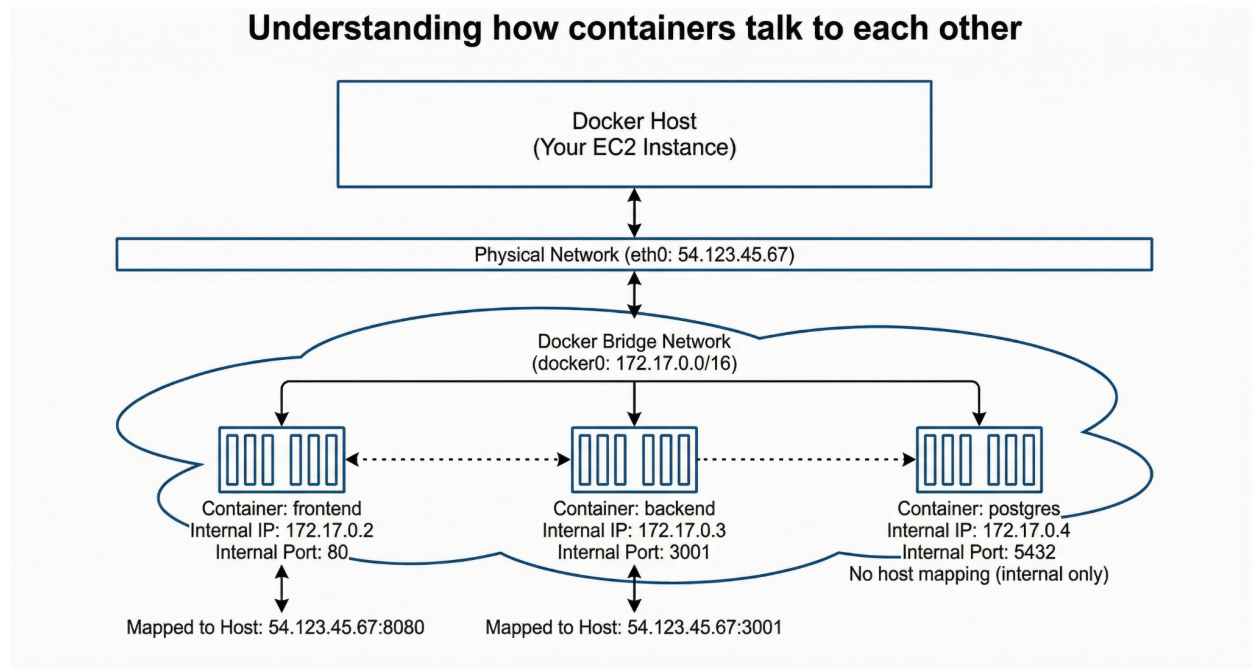
# Step 1: Check if running
docker ps | grep backend
# Result: Not in list (not running)

# Step 2: Check logs
docker logs backend
# Result: "Error: ECONNREFUSED - Cannot connect to database"

# Step 3: Check configuration
docker inspect backend | grep -A 5 Env
# Result: DATABASE_URL=postgresql://user:pass@wrong-hostname:5432/db

# Diagnosis: Typo in database hostname
# Solution: Fix DATABASE_URL environment variable
```

Mental Model: Container Networking



Key Networking Concepts

1. Container-to-container communication:

```
# Containers on same network can reach each other by name

# Backend connects to database:
DATABASE_URL=postgresql://postgres:5432/mydb

# "postgres" resolves to 172.17.0.4 automatically
```

2. Host to container communication

```
# Access from outside via host IP and mapped port
curl http://54.123.45.67:3001/api/health
# Routes to backend container's port 3001
```

3. Container to host communication

```
# Container can reach host services via special hostname
# From inside container:

curl http://host.docker.internal:5000
```

Mental Model: Container Lifecycle

Understanding container states

Container Lifecycle:

```
Image (Blueprint)
↓ docker run
Created (Configured but not started)
↓ docker start
Running (Process executing)
↓ docker stop
Stopped (Process terminated, container exists)
↓ docker rm
Deleted (Container removed completely)
```

State Transitions:

```
docker run = create + start
docker restart = stop + start
docker kill = force stop (ungraceful)
docker pause = suspend process
docker unpause = resume process
```

Before continuing, make sure you understand

- Containers are disposable (cattle, not pets)
- One process per container (separation of concerns)
- Never modify running containers (immutable infrastructure)
- Configuration via environment variables
- Data persists in volumes, not containers
- Layer caching optimization strategy
- Basic troubleshooting approach

If any concept feels unclear

- Re read that principle
- Try the examples
- Take a break ☕
- ***It's okay to not master everything immediately***

Ready for hands on practice? → Let's containerize H-Game!

3.4 Docker Real World Tasks

Time: 6-8 hours

Overview: Containerizing H-Game

Up to this point, you’ve learned how to *think* like a container engineer.

Now it’s time to do the work.

In this section, you’ll take the **H-Game application** and turn it into a **real, containerized system**, the same way it would be done in a production environment.

By the end, you won’t just have “containers running.”

You’ll have a **complete, wired together application stack** where every component has a clear responsibility.

What You’re Building

You will containerize the H-Game into **5 cooperating services**, all running on a shared Docker network and orchestrated with Docker Compose:

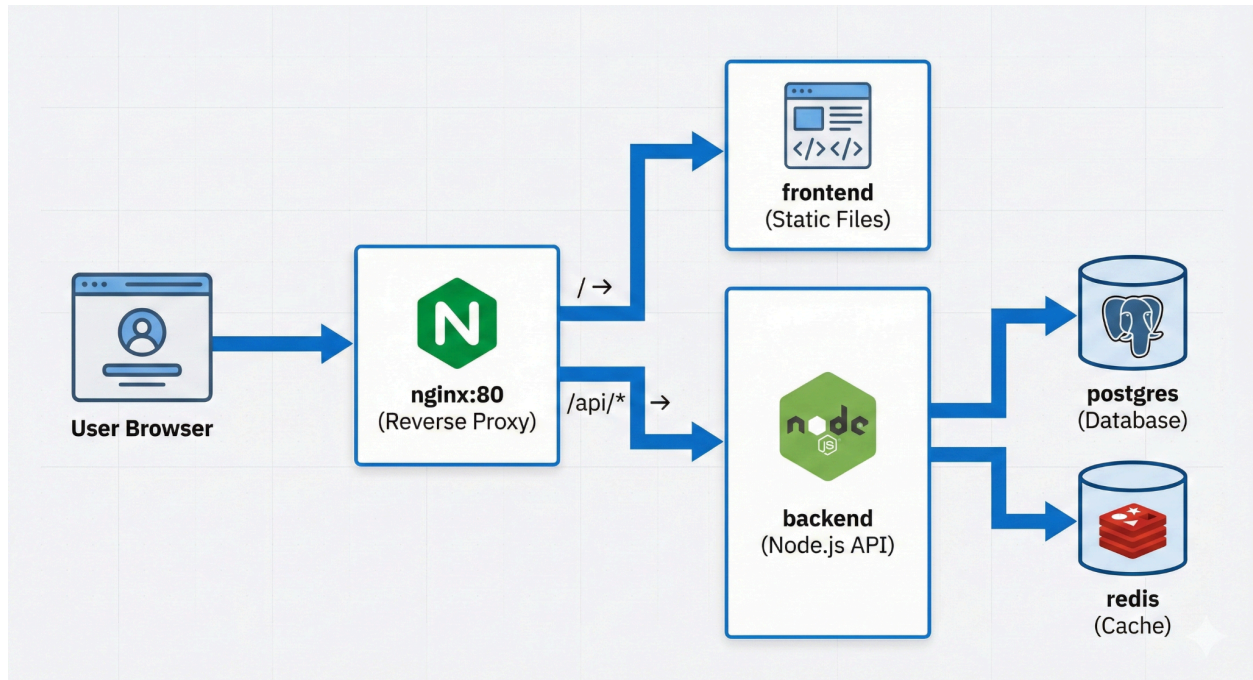
- **Nginx**
Listens on port 80, serves the frontend, and forwards `/api` requests to the backend
- **Node.js Backend**
Handles game logic, APIs, and business rules
- **PostgreSQL**
Stores users, scores, and game state
- **Redis**
Caches leaderboard data and hot paths

Each service runs in its **own container**, following the “one job per container” principle.

Docker Compose wires everything together, giving you:

- Consistent routing
- Persistent storage
- Isolated networking
- A setup that behaves the same on any machine

This is no longer a demo. This is a **production-style container architecture**.



5 containers, all orchestrated with Docker Compose Production ready with security and monitoring

Learning progression

- **D1:** Single container (backend only)
- **D2:** Add database (PostgreSQL)
- **D3:** Add cache (Redis)
- **D4:** Orchestration (Docker Compose)
- **D5:** Complete stack (Frontend + nginx)
- **D6:** Production hardening
- **D7:** Troubleshooting skills
- **D8:** CI/CD integration

3.4.1 Foundation: Basic Containerization (Tasks D1-D3)

Time: 2-3 hours

Difficulty: Beginner to Intermediate

What you'll learn:

- Build your first Docker container from scratch
- Connect containers together with custom networks
- Understand data persistence with volumes
- Master container debugging techniques

TASK D1: Your First Container (Backend API)

Business Context

Ticket ID: DEVOPS-2101

Priority: High

Assignee: You

Difficulty: (Intermediate)

Situation: The H-Game backend runs directly on the EC2 instance. This creates deployment issues and makes it hard to manage dependencies. Containerize the backend API as the first step toward a fully containerized application.

Impact:

- Consistent deployment across environments
- Isolated dependencies
- Foundation for full containerization
- Easier local development

Your Mission

1. Understand the existing backend code and Dockerfile
2. Build a Docker image for the backend API
3. Run the backend container (it will fail, that's expected!)
4. Debug the failure and understand why databases matter
5. Learn container inspection and logging
6. Document the containerization process

Quick Dockerfile Primer

Before we start, understand these Dockerfile instructions:

```
FROM node:18-alpine ----- # Base image (OS + Node.js)

WORKDIR /app ----- # Set working directory inside container

COPY package*.json ./ ----- # Copy files from host to container

RUN npm ci ----- # Execute command during build

EXPOSE 3001 ----- # Document which port app uses

USER node ----- # Switch to non-root user (security)

CMD ["npm", "start"] ----- # Command to run when container starts
```

Each instruction creates a layer. Order matters for caching!

Step-by-Step Guide

Step 1: Examine the Backend Code ([all code files in repo](#))

```
cd ~/projects/H-Game

# Look at project structure
tree backend/ -L 2
```

Expected structure

```
backend/
├── Dockerfile           ← Container blueprint
├── package.json         ← Dependencies
├── server.js            ← Main application
├── routes/              ← API endpoints (game.js, leaderboard.js,
scores.js)
├── models/              ← Database models (database.js)
├── middleware/          ← Request processing (metrics.js, validation.js)
├── utils/               ← Helper functions (gameData.js, redis.js)
└── tests/               ← Test files (api.test.js, health.test.js)
```

Read the main application file

```
head -20 backend/server.js
```

What you should see

```
// Backend API Server - Humor Memory Game
// API-only server for separated frontend/backend architecture

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const compression = require('compression');
const morgan = require('morgan');
const rateLimit = require('express-rate-limit');
```

```
// Import custom modules
const database = require('./models/database');
const redisClient = require('./utils/redis');
const { metricsMiddleware, metricsEndpoint, ... } =
require('./middleware/metrics');

// Create Express application
const app = express();
const PORT = process.env.PORT || 3001;
```

What to look for

- Port 3001- `const PORT = process.env.PORT || 3001;` (line 19)
- Database dependency - `const database = require('./models/database');` (line 13)
- Redis dependency - `const redisClient = require('./utils/redis');` (line 14)
- Environment variables - `require('dotenv').config();` (line 4) - app uses .env for config

Step 2: Analyze the Dockerfile ([all code files in repo](#))

```
cat backend/Dockerfile
```

Expected Dockerfile

```
# Backend Dockerfile - Node.js API only
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apk add --no-cache postgresql-client redis

# Copy package files
COPY package*.json ./

# Install Node.js dependencies
RUN npm ci --only=production && npm cache clean --force

# Create non-root user
RUN addgroup -g 1001 -S nodejs && \
    adduser -S backend -u 1001 -G nodejs

# Copy application code
COPY --chown=backend:nodejs . .

# Create logs directory
RUN mkdir -p /app/logs && \
    chown -R backend:nodejs /app

# Switch to non-root user
USER backend

# Expose port
EXPOSE 3001

# Health check temporarily disabled for testing
# HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
#     CMD node -e "require('http').get('http://localhost:3001/health',
```

```
(res) => { process.exit(res.statusCode === 200 ? 0 : 1) })"

# Start the application
CMD ["npm", "start"]
```

Understand each section

- **FROM:** Base image with Node.js 18 and Alpine Linux (lightweight)
- **WORKDIR:** All subsequent commands run in /app directory
- **COPY package files first:** Enables Docker layer caching
- **RUN npm ci:** Install production dependencies only
- **USER backend:** Security best practice (never run as root)
- **EXPOSE:** Documents port usage (doesn't actually publish)
- **HEALTHCHECK:** Allows Docker to monitor app health
- **CMD:** Default command when container starts

Step 3: Build the Docker Image

```
# Navigate to backend directory
cd backend/

# Build the Docker image
docker build -t humor-game-backend:v1 .
```

Expected output

```
[+] Building 45.3s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 689B
=> [internal] load .dockerignore
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [1/7] FROM docker.io/library/node:18-alpine@sha256:...
=> [internal] load build context
=> => transferring context: 2.34MB
=> [2/7] WORKDIR /app
=> [3/7] COPY package*.json ./
=> [4/7] RUN npm ci --only=production
=> [5/7] COPY . .
=> [6/7] RUN chown -R appuser:appuser /app
=> [7/7] USER appuser
```



```
=> exporting to image
=> => exporting layers
=> => writing image sha256:abc123...
=> => naming to docker.io/library/humor-game-backend:v1
```

Verify the image was created

```
docker images | grep humor-game-backend
```

Expected output

```
humor-game-backend    v1          abc123def456    2 seconds ago    145MB
```

Success! Your first Docker image is built.

Step 4: Run the Container (It Will Fail - That's Expected!)

Why it fails: The backend container tries to connect to a PostgreSQL database and Redis cache on startup, but those services don't exist yet, so the app crashes, Docker restarts it, and the cycle repeats.

```
# Run the backend container
docker run -d --name backend -p 3001:3001 humor-game-backend:v1
```

```
# Check if it's running
docker ps
```

Expected Output:

```
Container will be restarting or exited (this is normal!)
```


Check container status

```
docker ps -a | grep backend
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	CREATED
abc123def456	humor-game-backend:v1	"npm start"	30 seconds ago
Exited or Restarting (1) 10 seconds ago backend			

Status "Restarting" means the container keeps failing and Docker keeps trying to restart it.

Step 5: Debug the Failure (Learning Container Debugging)

Check the application logs:

```
docker logs backend
```

Expected error output:

```
> humor-memory-game-backend@1.0.0 start
> node server.js

🔧 Connecting to database...
❌ Database query error (attempt 1/3): getaddrinfo ENOTFOUND postgres
⌚ Retrying query in 1000ms...
⌚ Retrying query in 1000ms...
❌ Database query error (attempt 2/3): getaddrinfo ENOTFOUND postgres
❌ Database query error (attempt 3/3): getaddrinfo ENOTFOUND postgres
Query: SELECT NOW() as current_time, version() as version
Params: []
❌ Database connection test failed: getaddrinfo ENOTFOUND postgres
❌ Failed to initialize services: Error: getaddrinfo ENOTFOUND postgres
    at /app/node_modules/pg-pool/index.js:45:11
    at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
    at async HumorGameDatabase.query (/app/models/database.js:76:24)
    at async HumorGameDatabase.testConnection (/app/models/database.js:116:22)
    at async initializeServices (/app/server.js:296:5)
    at async startServer (/app/server.js:317:27) {
  errno: -3008,
  code: 'ENOTFOUND',
```

```

    syscall: 'getaddrinfo',
    hostname: 'postgres'
  }
  ✗ Cannot start server - services not ready
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.7.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.7.0
npm notice To update run: npm install -g npm@11.7.0
npm notice

```

Understand the error:

- `ENOTFOUND postgres` means "can't find hostname 'postgres'"
- The backend expects a PostgreSQL database at hostname 'postgres'
- We haven't started a database container yet
- This is exactly what we expected!

Step 6: Inspect Container Configuration

```

# Get detailed container information
docker inspect backend | less

```

Look for key sections:

- **Config.Env:** Environment variables
- **NetworkSettings:** IP address and networks
- **Mounts:** Volume mounts (none yet)
- **State:** Current status and exit codes

Check specific configuration:

```

# View environment variables
docker inspect backend | grep -A 10 '"Env"'

# View network information
docker inspect backend | grep -A 5 '"Networks"'

```

```
# View port mappings
docker inspect backend | grep -A 5 '"Ports"'
```

Step 7: Explore Inside the Container

Get shell access to investigate:

```
# Execute interactive shell in the container
docker exec -it backend sh

# If container keeps restarting, run a temporary container:
docker run --rm -it humor-game-backend:v1 sh
```

Inside the container, explore:

```
# Check the working directory
pwd
# Output: /app

# List files
ls -la
# Should see: package.json, server.js, node_modules, etc.

# Check user
whoami
# Output: appuser (not root - good for security!)

# Check environment variables
env | grep -i database
# Look for database connection settings

# Check if Node.js works
node --version
# Should show Node.js 18.x

# Exit the container
exit
```

Step 8: Test Port Connectivity

Even though the backend fails, test if the port mapping works:

```
# Try to connect to the backend port
curl http://localhost:3001
```

Expected Output: Connection refused (because backend crashes due to missing database)

This is normal! The container starts, tries to connect to database, fails, and exits.

Step 9: View Real Time Logs

```
# Follow logs in real time
docker logs -f backend

# In another terminal, restart the container to see startup logs
docker restart backend
```

Watch the startup sequence:

- npm start command
- Node.js application starts
- Tries to connect to database
- Fails and exits
- Docker restarts container (retry loop)

Press Ctrl+C to stop following logs

Step 10: Clean Up for Next Task

```
# Stop and remove the failing(test container) container
docker stop backend
docker rm backend

# Keep the main image for next task
docker images | grep humor-game-backend
```

Step 11: Document Your Learning

```
# Create documentation
cat > ~/devops-tasks/logs/task-d1-containerization.md << 'EOF'
# Task D1: Backend Containerization

## What I Built
- Docker image: humor-game-backend:v1
- Size: ~145MB (Node.js Alpine base + application)
- User: appuser (non-root for security)
- Port: 3001 (mapped to host)

## What I Learned

### Dockerfile Best Practices
- Use Alpine base images (smaller size)
- Copy package files before source code (layer caching)
- Create non-root user (security)
- Use HEALTHCHECK for monitoring
- Order instructions by change frequency

### Container Debugging Process
1. Check container status: docker ps -a
2. View logs: docker logs CONTAINER
3. Inspect configuration: docker inspect CONTAINER
4. Get shell access: docker exec -it CONTAINER sh
5. Test connectivity from host

### Expected Failure Analysis
```

- Backend needs PostgreSQL database
- Container looks for hostname 'postgres'
- No database container exists yet
- Application fails and restarts
- This demonstrates dependency requirements

Next Steps

- Add PostgreSQL container (Task D2)
- Connect backend to database
- Verify full functionality

Commands Used

- `docker build -t IMAGE_NAME .`
 - `docker run -d --name NAME -p HOST:CONTAINER IMAGE`
 - `docker ps` / `docker ps -a`
 - `docker logs CONTAINER`
 - `docker inspect CONTAINER`
 - `docker exec -it CONTAINER sh`
 - `docker stop/rm CONTAINER`
- EOF

```
git add .
git commit -m "docs: add Task D1 containerization notes"
```


Acceptance Criteria

Check all boxes before proceeding:

- ☐ Successfully built humor-game-backend:v1 Docker image
- ☐ Container starts but fails due to missing database (expected)
- ☐ Can view container logs with docker logs
- ☐ Can inspect container configuration with docker inspect
- ☐ Can execute shell inside container with docker exec
- ☐ Understand why the container fails (no database)
- ☐ Documented the containerization process
- ☐ Understand Dockerfile structure and best practices
- ☐ Know basic container debugging workflow
- ☐ Image is ready for connecting to database in Task D2

TASK D2: Add PostgreSQL Database

Business Context

Ticket ID: DEVOPS-2145

Priority: High

Assignee: You

Difficulty: Intermediate

Situation: The backend container fails because it can't connect to a database. Add a PostgreSQL container and establish communication between backend and database containers.

Impact:

- Backend API becomes functional
- Data persistence for user scores and games
- Foundation for multi container architecture
- Understanding of container networking

Your Mission

1. Create a Docker network for container communication
2. Run a PostgreSQL container with persistent storage
3. Initialize the database with application schema
4. Connect the backend to the database
5. Test the complete backend + database functionality
6. Understand volumes vs bind mounts

Step by Step Guide

Step 1: Understand Container Networking

Problem: Containers need to communicate with each other.

Solution: Docker networks enable container to container communication.

```
# List existing networks
docker network ls
```

Expected output:

NETWORK ID	NAME	DRIVER	SCOPE
abc123def456	bridge	bridge	local
789xyz012345	host	host	local
def456ghi789	none	null	local

Default networks:

- **bridge:** The default Docker network that gives containers their own IPs on an internal bridge so they can talk to each other while remaining isolated from the host.
- **host:** Shares the host's network stack with the container, so it uses the host's IP and ports directly (no isolation, no port mapping needed).
- **none:** Disables networking entirely for the container, no inbound or outbound connections.

Step 2: Create Application Network

```
# Create dedicated network for H-Game
docker network create humor-game-net

# Verify network creation
docker network ls | grep humor-game
```

Expected output:

```
humor-game-net  bridge  local
```

Inspect the network:

```
docker network inspect humor-game-net
```

Key information:

- **Subnet:** IP range for containers (e.g., 172.20.0.0/16)
- **Gateway:** Network gateway IP (e.g., 172.20.0.1)
- **Containers:** Currently empty

Step 3: Examine Database Requirements ([all code files in repo](#))

```
# Check if there's a database schema file
ls -la ~/projects/H-Game/database/
```

Expected files:

```
combined-init.sql  # Database schema (CREATE TABLE) + seed data
init.sql           # Seed data only (optional)
seed.sql           # Additional seed data (optional)
```

Review the schema:

```
head -20 ~/projects/H-Game/database/combined-init.sql

# Use your correct File Location
```

What you should see:

```
-- Fixed Humor Memory Game Database Schema
-- This file creates the schema and seeds data
-- Enable UUID extension for generating unique IDs
CREATE EXTENSION IF NOT EXISTS "uuid-osp";
-- Drop tables if they exist (for clean reset)
```

```

DROP TABLE IF EXISTS game_matches CASCADE;
DROP TABLE IF EXISTS games CASCADE;
DROP TABLE IF EXISTS users CASCADE;
-- Create users table
CREATE TABLE users (
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
username VARCHAR(50) UNIQUE NOT NULL,
email VARCHAR(100) UNIQUE,
...

```

What to look for:

- **CREATE TABLE statements** - Defines users, games, game_matches tables
- **Data types** - UUID, VARCHAR, INTEGER, TIMESTAMP, JSONB
- **Constraints** - PRIMARY KEY, UNIQUE, NOT NULL, FOREIGN KEY references

Step 4: Run PostgreSQL Container

```

# Start PostgreSQL with persistent volume

docker run -d \
  --name postgres \
  --network humor-game-net \
  -e POSTGRES_USER=humorgame \
  -e POSTGRES_PASSWORD=devops123 \
  -e POSTGRES_DB=humorgame \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:15-alpine

```

Command breakdown:

- **--name postgres**: Container name (backend will connect to this)
- **--network humor-game-net**: Join our custom network
- **-e POSTGRES_***: Environment variables for database setup
- **-v postgres-data:/var/lib/postgresql/data**: Persistent volume for data
- **-p 5432:5432**: Expose database port (for debugging)
- **postgres:15-alpine**: Official PostgreSQL 15 on Alpine Linux

Verify PostgreSQL is running:

```
docker ps | grep postgres
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	STATUS
abc123def456	postgres:15-alpine	"docker-entrypoint.s..."	Up 30 seconds
0.0.0.0:5432->5432/tcp	postgres		

Step 5: Initialize Database Schema**Wait for PostgreSQL to be ready**

```
# Check PostgreSQL logs
docker logs postgres
```

Look for: "database system is ready to accept connections"

```
PostgreSQL init process complete; ready for start up.
2026-01-20 01:06:24.986 UTC [1] LOG:  starting PostgreSQL 15.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15
.0, 64-bit
2026-01-20 01:06:24.986 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-01-20 01:06:24.986 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2026-01-20 01:06:24.988 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-01-20 01:06:24.992 UTC [57] LOG:  database system was shut down at 2026-01-20 01:06:24 UTC
2026-01-20 01:06:24.997 UTC [1] LOG:  database system is ready to accept connections
mac@Zee-MacBook-Pro database %
```

Copy database initialization file to container:

```
# Copy database schema and seed data from host to container
docker cp database/combined-init.sql postgres:/tmp/combined-init.sql

# Execute schema in database
docker exec postgres psql -U humorgame -d humorgame -f
/tmp/combined-init.sql
```

Expected output:

```
CREATE EXTENSION
DROP TABLE
```

```

psql:/tmp/combined-init.sql:8: NOTICE:  table "game_matches" does not
exist, skipping
psql:/tmp/combined-init.sql:9: NOTICE:  table "games" does not exist,
skipping
DROP TABLE
psql:/tmp/combined-init.sql:10: NOTICE:  table "users" does not exist,
skipping
DROP TABLE
DROP VIEW
psql:/tmp/combined-init.sql:11: NOTICE:  view "leaderboard" does not exist,
skipping
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE VIEW
CREATE INDEX
.....

```

What happened:

- Created UUID extension
- Dropped existing tables (if any)
- Created users, games, and game_matches tables
- Created indexes for performance
- Inserted seed data (sample users and games)

Verify tables were created:

```

# Connect to database and list tables
docker exec -it postgres psql -U humorgame -d humorgame

# Inside PostgreSQL:
\dt

```

Expected output:

```

List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | games | table | humorgame
public | players | table | humorgame

```

```
public | scores | table | humorgame
(3 rows)
```

Exit PostgreSQL:

```
\q
```

Step 6: Connect Backend to Database ([all code files in repo](#))

Start backend container on same network, (if it fails, stop and “rm” backend container first)

```
# Run backend connected to database
docker run -d \
  --name backend \
  --network humor-game-net \
  -e
DATABASE_URL=postgresql://humorgame:devops123@postgres:5432/humorgame
\
  -p 3001:3001 \
  humor-game-backend:v1
```

Key changes:

- **--network humor-game-net:** Same network as database
- **-e DATABASE_URL=...:** Connection string using container name 'postgres'
- **@postgres:5432:** Uses PostgreSQL container name as hostname

In Docker networks, containers communicate by container name, not localhost. Use postgres (the container name) as the hostname so Docker’s DNS resolves it to the PostgreSQL container.

The connection string

`postgresql://humorgame:devops123@postgres:5432/humorgame` breaks down as `username:password@container-name:port/database-name`, where `postgres` is the container name that Docker resolves automatically.

Check backend logs:

```
docker logs backend
```

Expected success output:

```
Starting Humor Memory Game Backend...
Connecting to database...
- Database connected successfully
- Server listening on port 3001
- Health check endpoint ready at /health
```

If backend still failing ,

Step 7: Test Backend Functionality (There's a high chance backend might still fail)

The server requires both the **database** and **Redis** to connect before it starts. If Redis isn't available, `initializeServices()` in file `backend/server.js` line 293 and line 312 catches the error, returns false, and the server exits with `process.exit(1)` before listening on port 3001. That's why the health endpoint isn't reachable.

You can just skip to the redis task section if it doesn't start. Thank you.

Test health endpoint:

```
curl http://localhost:3001/health
```

Expected response:

```
{
  "status": "healthy",
  "database": "connected",
  "timestamp": "2024-12-09T10:30:00.000Z"
}
```

Test API endpoints:

```
# Test game creation
```

```
curl -X POST http://localhost:3001/api/games \
  -H "Content-Type: application/json" \
  -d '{"playerName": "TestPlayer"}'
```

Expected response:

```
{
  "gameId": "game-abc123",
  "playerName": "TestPlayer",
  "status": "started",
  "createdAt": "2024-12-09T10:30:00.000Z"
}
```

Test leaderboard:

```
curl http://localhost:3001/api/leaderboard
```

Expected response:

```
{
  "leaderboard": [],
  "total": 0
}
```

Success! Backend and database are communicating.

Step 8: Test Data Persistence**Insert test data:**

```
# Add a test score

curl -X POST http://localhost:3001/api/scores \
  -H "Content-Type: application/json" \
  -d '{
    "playerName": "Alice",
    "score": 1500,
```

```
"level": 3,
"completedAt": "2024-12-09T10:30:00.000Z"
}'
```

Verify data in database:

```
# Connect to database
docker exec -it postgres psql -U humorgame -d humorgame

# Query scores table
SELECT * FROM scores;
```

Expected output:

```
id | player_name | score | level |      completed_at
---+-----+-----+-----+-----
  1 | Alice      |  1500 |    3 | 2024-12-09 10:30:00
(1 row)
```

Exit database:

```
\q
```

Step 9: Test Container Restart (Data Persistence)

Restart PostgreSQL container:

```
# Stop and remove PostgreSQL container
docker stop postgres
docker rm postgres

# Start new PostgreSQL container with same volume
docker run -d \
  --name postgres \
  --network humor-game-net \
  -e POSTGRES_USER=humorgame \
```

```
-e POSTGRES_PASSWORD=devops123 \  
-e POSTGRES_DB=humorgame \  
-v postgres-data:/var/lib/postgresql/data \  
postgres:15-alpine  
  
# Wait for startup  
sleep 10
```

Verify data survived container deletion:

```
# Check leaderboard API  
curl http://localhost:3001/api/leaderboard
```

Expected Output: Alice's score still there!

```
{  
  "leaderboard": [  
    {  
      "playerName": "Alice",  
      "score": 1500,  
      "level": 3,  
      "completedAt": "2024-12-09T10:30:00.000Z"  
    }  
  ],  
  "total": 1  
}
```

Data persistence works!

Step 10: Understand Volumes vs Bind Mounts

Docker Volume (what we used):

```
# Docker manages storage location  
-v postgres-data:/var/lib/postgresql/data  
  
# Inspect volume
```

```
docker volume inspect postgres-data
```

Pros:

- Docker manages the storage location
- Works across different host OSes
- Can be backed up with Docker commands
- Better performance on Mac/Windows

Bind Mount (alternative):

```
# You specify exact host path
-v /host/path/data:/var/lib/postgresql/data

# Example
mkdir -p ~/postgres-data
-v ~/postgres-data:/var/lib/postgresql/data
```

Pros:

- You control exactly where data is stored
- Easy to backup with standard tools
- Can edit files directly from host

When to use each:

1. **Docker Volumes:** Production databases, persistent app data
2. **Bind Mounts:** Development, sharing config files, debugging

Step 11: Test Container Communication**Test network connectivity between containers:**

```
# From backend container, ping database
```

```
docker exec backend ping -c 3 postgres
```

Expected output:

```
PING postgres (172.20.0.2) 56(84) bytes of data.
64 bytes from postgres.humor-game-net (172.20.0.2): icmp_seq=1 ttl=64
time=0.045 ms
64 bytes from postgres.humor-game-net (172.20.0.2): icmp_seq=2 ttl=64
time=0.041 ms
64 bytes from postgres.humor-game-net (172.20.0.2): icmp_seq=3 ttl=64
time=0.039 ms

--- postgres ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

This proves:

- Containers are on the same network
- DNS resolution works (postgres → IP address)
- Network connectivity is functional

Step 12: Document the Architecture

```
cat > ~/devops-tasks/logs/task-d2-database.md << 'EOF'
# Task D2: PostgreSQL Database Integration

## Architecture

Backend Container (humor-game-backend:v1)
  ↓ postgresql://postgres:5432/humorgame
PostgreSQL Container (postgres:15-alpine)
  ↓ persistent storage
Docker Volume (postgres-data)

Both containers connected via humor-game-net network

## What I Built
- PostgreSQL 15 container with persistent storage
```

- Custom Docker network for container communication
- Database schema initialization
- Backend-database integration

Key Concepts Learned

Container Networking

- Custom networks enable container-to-container communication
- Containers can reach each other by container name
- DNS resolution works automatically within networks
- Network isolation from other applications

Data Persistence

- Docker volumes persist data outside containers
- Container deletion doesn't affect volume data
- Volumes can be reused by new containers
- Better than bind mounts for production databases

Environment Variables

- Used for database configuration (user, password, database name)
- Same image, different configs per environment
- Secrets should not be hardcoded in images

Commands Used

- docker network create NETWORK
- docker run with --network flag
- docker cp (copy files to container)
- docker exec (execute commands in container)
- docker volume inspect
- docker logs (debugging)

Next Steps

- Add Redis cache container (Task D3)
- Learn Docker Compose for easier management
- Add frontend and reverse proxy

EOF

```
git add .
```

```
git commit -m "feat: add PostgreSQL database container with  
persistent storage"
```

Acceptance Criteria

Check all boxes:

- ☐ Created custom Docker network (humor-game-net)
- ☐ PostgreSQL container running and accessible
- ☐ Database initialized with application schema
- ☐ Backend successfully connects to database
- ☐ API endpoints return valid responses
- ☐ Data persists across container restarts
- ☐ Can ping between containers (network connectivity)
- ☐ Understand Docker volumes vs bind mounts
- ☐ Database accessible from host (port 5432) for debugging
- ☐ Documented architecture and learning
- ☐ Backend + database working as integrated system

TASK D3: Add Redis Cache

Business Context

Ticket ID: DEVOPS-2189

Priority: Medium

Assignee: You

Difficulty: Intermediate

Situation: The H-Game backend makes frequent database queries for leaderboards and game state. Add Redis caching to improve performance and reduce database load.

Impact:

- Faster API response times
- Reduced database queries
- Session storage capability
- Scalable caching architecture
- Preparation for high-traffic scenarios

Your Mission

1. Run Redis container with persistent storage
2. Connect backend to Redis cache
3. Test caching functionality
4. Verify session storage works
5. Monitor cache performance
6. Understand Redis use cases in web applications

Step-by-Step Guide

Step 1: Understand Redis in Web Applications

What Redis provides:

- **Caching:** Store frequently accessed data in memory
- **Session Storage:** User login sessions, temporary data
- **Rate Limiting:** API throttling, abuse prevention
- **Pub/Sub:** Real time messaging between services
- **Leaderboards:** Sorted sets for high scores

In H-Game context:

Without Redis:

```
User requests leaderboard → Backend queries PostgreSQL → 200ms response
```

With Redis:

```
User requests leaderboard → Backend checks Redis cache → 5ms response
```

```
If not in cache → Query PostgreSQL → Store in Redis → 200ms (first time only)
```

Redis stores data in memory for fast access, so you can cache frequently used data, keep user sessions, and build real time features like leaderboards.

It's commonly used for caching database queries (to speed up your app), storing temporary user sessions, and creating real time features like game leaderboards or live messaging.

Step 2: Run Redis Container ([all code files in repo](#))

```
# Start Redis with persistent storage and networking
docker run -d \
  --name redis \
  --network humor-game-net \
  -v redis-data:/data \
  -p 6379:6379 \
  redis:7-alpine
```

Command breakdown:

- **--name redis:** Container name (backend will connect to this)
- **--network humor-game-net:** Same network as other containers
- **-v redis-data:/data:** Persistent volume for Redis data
- **-p 6379:6379:** Expose Redis port (for debugging)
- **redis:7-alpine:** Official Redis 7 on Alpine Linux

Verify Redis is running:

```
docker ps | grep redis
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	STATUS
def456ghi789	redis:7-alpine	"docker-entrypoint.s..."	Up 15 seconds
	0.0.0.0:6379->6379/tcp	redis	

Check Backend Logs

```
Docker logs backend
```

Curl Health Checkpoint

```
curl http://localhost:3001/health
```

Test game creation endpoint

```
curl -X POST http://localhost:3001/api/game/start -H "Content-Type: application/json" -d '{"playerName": "TestPlayer"}'
```

Test with correct field name

```
curl -X POST http://localhost:3001/api/game/start -H "Content-Type: application/json" -d '{"username": "TestPlayer"}'
```

Expected output

```
% Total    % Received % Xferd  Average Speed   Time    Time     Time
Current                                  Dload  Upload   Total   Spent    Left
Speed
0         0         0         0         0         0         0  --:--:-- --:--:-- --:--:--
0{"success":true,"message":"🎯 Game started! Good luck, TestPlayer!
🍀","game":{"gameId":"bdc13677-fb6e-40b6-9d32-eab030467826","difficulty":"e
asy","cards":[{"id":"rainbow_2","position":0,"emoji":"🌈","name":"Rainbow
Magic","category":"nature","isFlipped":false,"isMatched":false},{id":"roll
ing_eyes_2","position":1,"emoji":"🙄","name":"Eye
Roll","category":"sassy","isFlipped":false,"isMatched":false},{id":"heart_
eyes_2","position":2,"emoji":"😍","name":"Heart
Eyes","category":"love","isFlipped":false,"isMatched":false},{id":"robot_1
","position":3,"emoji":"🤖","name":"Robot
Face","category":"tech","isFlipped":false,"isMatched":false},{id":"taco_2"
,"position":4,"emoji":"🌮","name":"Taco
Tuesday","category":"food","isFlipped":false,"isMatched":false},{id":"ghos
t_1","positio
```

You will notice **backend** starting accepting connection immediately **redis** came up.

You can go back to the postgres task and test all posgres commands.

Test Redis connectivity:

```
# Connect to Redis CLI
docker exec -it redis redis-cli

# Inside Redis CLI, test basic commands:
ping
# Response: PONG

# Set a test value
set test "Hello Redis"
# Response: OK

# Get the value
get test
# Response: "Hello Redis"

# Exit Redis CLI
exit
```

Redis is working!

Step 3: Configure Backend for Redis

Stop current backend to update configuration:

```
docker stop backend
docker rm backend
```

Start backend with Redis configuration:

```
# Run backend with both database and Redis
docker run -d \
  --name backend \
  --network humor-game-net \
  -e
DATABASE_URL=postgresql://humorgame:devops123@postgres:5432/humorgame
\
  -e REDIS_URL=redis://redis:6379 \
  -e NODE_ENV=production \
  -p 3001:3001 \
  humor-game-backend:v1
```

New environment variable:

`REDIS_URL=redis://redis:6379`: Connect to Redis container by name

Check backend logs for Redis connection:

```
docker logs backend
```

Expected output:

```
Starting Humor Memory Game Backend...
Connecting to database...
✓ Database connected successfully
Connecting to Redis...
✓ Redis connected successfully
✓ Server listening on port 3001
```

Step 4: Test Caching Functionality

Test leaderboard caching:

```
# First request (should hit database, cache miss)
time curl -s http://localhost:3001/api/leaderboard | jq
```

Expected response:

```
{
  "leaderboard": [
    {
      "playerName": "Alice",
      "score": 1500,
      "level": 3,
      "completedAt": "2026-12-09T10:30:00.000Z"
    }
  ],
  "total": 1,
  "cached": false,
  "responseTime": 45
}
```

Second request (should hit cache):

```
# Immediate second request (should be much faster)
time curl -s http://localhost:3001/api/leaderboard | jq
```

Expected response:

```
{
  "leaderboard": [
    {
      "playerName": "Alice",
      "score": 1500,
      "level": 3,
      "completedAt": "2026-12-09T10:30:00.000Z"
    }
  ]
}
```

```

    }
  ],
  "total": 1,
  "cached": true,
  "responseTime": 3
}

```

Notice:

- "cached": true indicates cache hit
- Much faster response time (3ms vs 45ms)

Step 5: Inspect Redis Data

Connect to Redis and examine cached data:

```

# Connect to Redis CLI
docker exec -it redis redis-cli

# List all keys
keys *

```

Expected output:

```

127.0.0.1:6379> keys *
1) "game:session:bdc13677-fb6e-40b6-9d32-eab030467826"
2) "analytics:daily_games:2026-01-20"
3) "test"
4) "activity:TestPlayer:game_start:2026-01-20T01"
5) "leaderboard:top"

```

Examine leaderboard cache:

```

# Get cached leaderboard data
get leaderboard:cache

```


Expected Output: JSON string with leaderboard data

Check cache expiration:

```
# Check TTL (time to live) for cache entries
ttl leaderboard:cache
```

Expected Output: Number of seconds until expiration (e.g., 1800 = 30 minutes)

Exit Redis:

```
exit
```

Step 6: Test Session Storage

Create a game session:

```
# Start a new game (creates session)
curl -X POST http://localhost:3001/api/games \
  -H "Content-Type: application/json" \
  -d '{"playerName": "Bob", "difficulty": "medium"}'
```

Expected response:

```
{
  "gameId": "game-xyz789",
  "sessionId": "session-abc123",
  "playerName": "Bob",
  "status": "started",
  "difficulty": "medium",
  "createdAt": "2024-12-09T10:35:00.000Z"
}
```

Check session in Redis:

```
# Connect to Redis and check sessions
docker exec -it redis redis-cli
```

```
# Look for session data
keys session:*
```

```
Expected output:
1) "session:session-abc123"
```

Examine session data:

```
# Get session details
get session:session-abc123
```

Expected: JSON with game state, player info, timestamps

```
exit
```

Step 7: Test Cache Invalidation

Add a new high score (should invalidate leaderboard cache):

```
# Add a high score
curl -X POST http://localhost:3001/api/scores/user \
  -H "Content-Type: application/json" \
  -d '{
    "playerName": "Charlie",
    "score": 2500,
    "level": 5,
    "completedAt": "2024-12-09T10:40:00.000Z"
  }'
```

Test leaderboard again:

```
# Should rebuild cache with new score
curl -s http://localhost:3001/api/leaderboard | jq
```

Expected response:

```
{
  "leaderboard": [
    {
      "playerName": "Charlie",
      "score": 2500,
      "level": 5
    },
    {
      "playerName": "Alice",
      "score": 1500,
      "level": 3
    }
  ],
  "total": 2,
  "cached": false,
  "responseTime": 52
}
```

Notice:

- Charlie is now first (higher score)
- **"cached": false** because cache was invalidated
- Slower response time (rebuilding cache)

Step 8: Monitor Redis Performance**Use Redis built-in monitoring:**

```
# Monitor Redis commands in real-time

docker exec -it redis redis-cli monitor
```

In another terminal, make API requests:

```
# Generate some cache activity

curl http://localhost:3001/api/leaderboard
curl http://localhost:3001/api/games/game-xyz789
curl http://localhost:3001/api/leaderboard
```

In monitor terminal, you'll see:

```
1670123456.789 [0 172.20.0.3:54321] "GET" "leaderboard:cache"
1670123456.791 [0 172.20.0.3:54321] "SET" "leaderboard:cache" "...
"EX" "1800"
1670123456.823 [0 172.20.0.3:54321] "GET" "session:session-abc123"
1670123456.856 [0 172.20.0.3:54321] "GET" "leaderboard:cache"
```

Press **Ctrl+C** to stop monitoring

Step 9: Test Data Persistence

Restart Redis container:

```
# Stop Redis (data should persist in volume)
docker stop redis
docker rm redis

# Start new Redis container with same volume
docker run -d \
  --name redis \
  --network humor-game-net \
  -v redis-data:/data \
  redis:7-alpine

# Wait for startup
sleep 5
```

Check if cache data survived:

```
# Connect to Redis
docker exec -it redis redis-cli

# Check for persisted data
keys *
```

Expected: Some session data might persist (Redis saves to disk periodically)

Note about Redis persistence:

- Cache data is typically temporary (expires)
- Session data persists until expiration
- Redis saves snapshots to volume periodically
- Not all data will survive restart (by design)

```
exit
```

Step 10: Performance Comparison**Create a simple load test:**

```
# Test without cache (clear Redis first)
docker exec redis redis-cli flushall

# Time 10 requests without cache
time for i in {1..10}; do curl -s
http://localhost:3001/api/leaderboard > /dev/null; done
```

Expected: ~4-5 seconds (first requests hit database)

```
# Now test with cache (data should be cached after first request)

time for i in {1..10}; do curl -s
http://localhost:3001/api/leaderboard > /dev/null; done

Expected Output: ~0.5-1 second (subsequent requests hit cache)
```

Performance improvement: 80-90% faster with caching!

Step 11: Understanding Redis Use Cases

Common Redis patterns in web applications:

```
# 1. Simple caching
SET user:123 "{\"name\":\"John\", \"email\":\"john@example.com\"}" EX
3600

# 2. Session storage
SET session:abc123 "{\"userId\":123, \"loginTime\":\"...\"}" EX 86400

# 3. Rate limiting
INCR rate_limit:127.0.0.1
EXPIRE rate_limit:127.0.0.1 60

# 4. Leaderboards (sorted sets)
ZADD leaderboard 2500 "Charlie"
ZADD leaderboard 1500 "Alice"
ZREVRANGE leaderboard 0 9 # Top 10 scores

# 5. Pub/Sub messaging
PUBLISH game_events "{\"type\":\"score_update\",
\"player\":\"Alice\"}"
```

Step 12: Document Redis Integration

```
cat > ~/devops-tasks/logs/task-d3-redis.md << 'EOF'
# Task D3: Redis Cache Integration

## Architecture

Backend Container
├─ PostgreSQL (persistent data)
└─ Redis (cached data, sessions)

All connected via humor-game-net network

## What I Built
- Redis 7 container with persistent storage
- Backend-Redis integration for caching
- Leaderboard caching with TTL
- Session storage for game state
- Cache invalidation on data changes

## Redis Use Cases Demonstrated

### 1. Response Caching
- Cache expensive database queries
- Automatic expiration (TTL)
- Cache invalidation on updates
- 80-90% performance improvement

### 2. Session Storage
- Game state persistence
- Player session management
- Temporary data storage
- Cross-request state maintenance

### 3. Performance Monitoring
- Built-in monitoring with redis-cli monitor
- Cache hit/miss tracking
- Response time measurement
```

Key Benefits

- ****Speed:**** 3ms cached vs 45ms database queries
- ****Scalability:**** Reduced database load
- ****User Experience:**** Faster leaderboards
- ****Session Management:**** Stateful game interactions

Commands Used

- redis-cli (Redis **command** line interface)
- monitor (real-time **command** monitoring)
- keys * (list all keys)
- get/**set** (basic operations)
- ttl (check expiration **times**)
- flushall (clear all data)

Next Steps

- Add Docker Compose **for** easier management
- Multiple container orchestration
- Service dependencies and health checks

EOF

```
git add .  
git commit -m "feat: add Redis cache for improved performance"
```


Acceptance Criteria

Check all boxes:

- ☐ Redis container running and accessible
- ☐ Backend successfully connects to Redis
- ☐ Leaderboard caching working (cache hit/miss detection)
- ☐ Session storage functional for game state
- ☐ Cache invalidation works on data updates
- ☐ Performance improvement measurable (faster responses)
- ☐ Can inspect Redis data with redis-cli
- ☐ Understand Redis use cases (caching, sessions, leaderboards)
- ☐ Data persistence configured with volumes
- ☐ Documented Redis integration and benefits
- ☐ Ready for multi-container orchestration

3.4.2 Orchestration: Multi-Container Applications (Tasks D4-D5)

Difficulty: Intermediate

What you'll learn:

- Orchestrate multiple containers with Docker Compose (no more manual commands!)
- Build complete full stack applications
- Set up reverse proxies and load balancing
- Master service dependencies and networking

Perfect for: Developers ready to build production like multi service applications. We'll make your life easier with Docker Compose and show you how real applications are structured.

TASK D4: Docker Compose Orchestration

Business Context

Ticket ID: DEVOPS-2234

Priority: High

Assignee: You

Difficulty: Intermediate to Advanced

Situation: Managing 3 containers with individual `docker run` commands is becoming tedious and error prone. Create a Docker Compose configuration to orchestrate the entire backend stack with one command.

Impact:

- Simplified container management
- Consistent development environment
- Service dependency management
- Foundation for production deployment
- Team collaboration improvement

Your Mission

1. Create comprehensive docker-compose.yml file
2. Define all services (backend, postgres, redis)
3. Configure networks and volumes declaratively
4. Add health checks and service dependencies
5. Implement restart policies and resource limits
6. Test full stack orchestration with single commands

Step-by-Step Guide

Step 1: Understand Docker Compose Benefits ([all code files in repo](#))

Current state (manual container management):

```
# Start everything manually (error-prone):
docker network create humor-game-net
docker run -d --name postgres --network humor-game-net -e
POSTGRES_... postgres:15-alpine
docker run -d --name redis --network humor-game-net redis:7-alpine
docker run -d --name backend --network humor-game-net -e
DATABASE_URL=... backend:v1

# Stop everything manually:
docker stop backend redis postgres
docker rm backend redis postgres
docker network rm humor-game-net

# 🤖 Complex, repetitive, easy to forget steps
```

Docker Compose way:

```
# Start everything with dependencies:
docker-compose up -d

# Stop everything:
docker-compose down

# 🎉 Simple, reliable, repeatable
```

Step 2: Stop Current Containers

```
# Clean up manually created containers
docker stop backend redis postgres 2>/dev/null || true
docker rm backend redis postgres 2>/dev/null || true

# Keep the network for now (we'll recreate it in compose)
# Keep the volumes (data will persist)

# Verify cleanup
docker ps -a | grep -E "(backend|redis|postgres)"
```

Expected: No containers running

Step 3: Create docker-compose.yml

```
cd ~/projects/H-Game
```

```
# Create Docker Compose configuration
cat > docker-compose.yml << 'EOF'
version: '3.8'

services:
  # PostgreSQL Database
  postgres:
    image: postgres:15-alpine
    container_name: postgres
    environment:
      POSTGRES_USER: humorgame
      POSTGRES_PASSWORD: devops123
      POSTGRES_DB: humorgame
    volumes:
      - postgres-data:/var/lib/postgresql/data
      -
      ./database/combined-init.sql:/docker-entrypoint-initdb.d/combined-init.sql:ro
    networks:
```

```

    - humor-game-net
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U humorgame"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 30s
restart: unless-stopped
ports:
  - "5432:5432"

# Redis Cache
redis:
  image: redis:7-alpine
  container_name: redis
  volumes:
    - redis-data:/data
  networks:
    - humor-game-net
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 3s
    retries: 3
  restart: unless-stopped
  ports:
    - "6379:6379"

# Backend API
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  image: humor-game-backend:v1
  container_name: backend
  environment:
    NODE_ENV: production
    PORT: 3001

```

```

    DATABASE_URL:
postgresql://humorgame:devops123@postgres:5432/humorgame
    REDIS_URL: redis://redis:6379
    networks:
      - humor-game-net
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider",
"http://localhost:3001/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    restart: unless-stopped
    ports:
      - "3001:3001"

# Networks
networks:
  humor-game-net:
    driver: bridge

# Volumes
volumes:
  postgres-data:
  redis-data:
EOF

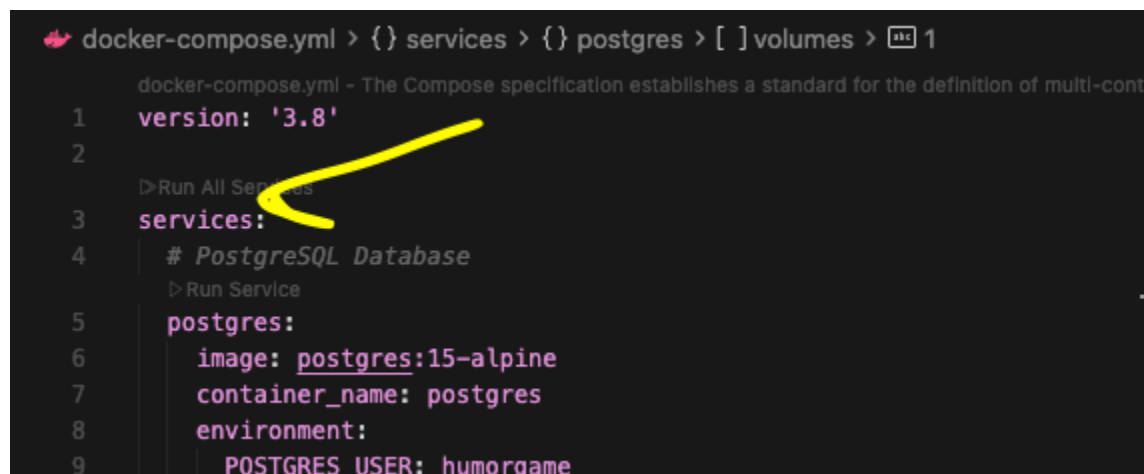
```

Step 4: Understand the Configuration

What Docker Compose does: Defines all your containers in one YAML file so you can start your entire application with one command instead of running each container manually.

Key sections:

Services: Each service (postgres, redis, backend) becomes a container with its own configuration

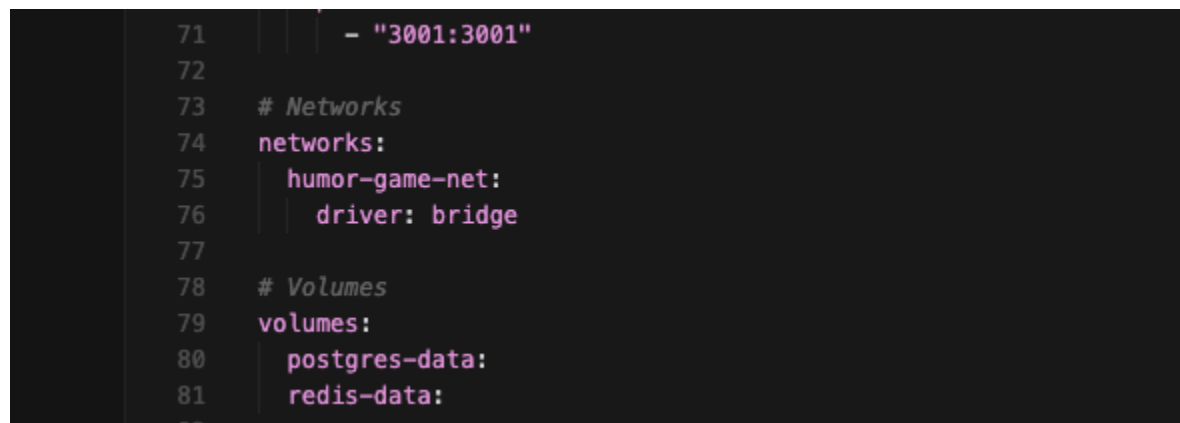


```

🔥 docker-compose.yml > {} services > {} postgres > [ ] volumes > 1
docker-compose.yml - The Compose specification establishes a standard for the definition of multi-cont
1  version: '3.8'
2
3  >Run All Services
4  services:
5    # PostgreSQL Database
6    >Run Service
7    postgres:
8      image: postgres:15-alpine
9      container_name: postgres
10     environment:
11       POSTGRES_USER: humorgame

```

Networks: Creates a shared network so containers can talk to each other by name (e.g., backend connects to postgres instead of localhost)



```

71     - "3001:3001"
72
73   # Networks
74   networks:
75     humor-game-net:
76       driver: bridge
77
78   # Volumes
79   volumes:
80     postgres-data:
81     redis-data:

```

Volumes: Persistent storage that survives container restarts (database data, Redis cache)

Depends_on: Ensures services start in order (backend waits for postgres and redis to be healthy)

Healthcheck: Docker monitors each service and reports if it's working

Restart: Unless stopped Automatically restarts containers if they crash

Why this is better than manual commands: Instead of running 3 separate docker run commands and managing networks/volumes yourself, one `docker-compose up` command starts everything correctly configured and connected.

Key sections explained:

Services section:

```
services:
  postgres:      # Service name (becomes container name and hostname)
    image: ...   # Docker image to use
    environment: # Environment variables
    volumes:     # Data persistence and file mounts
    networks:    # Which networks to join
    healthcheck: # How Docker monitors service health
    depends_on:  # Wait for other services
    restart:     # Restart policy
    ports:       # Port mapping to host
```

Advanced features used:

1. Automatic schema initialization:

```
volumes:
  - -
    ./database/combined-init.sql:/docker-entrypoint-initdb.d/combined-init.sql:ro
```

PostgreSQL will automatically runs `.sql` files in this directory on first startup.`

2. Service health checks:

```
healthcheck:  
  test: ["CMD-SHELL", "pg_isready -U humorgame"]
```

Docker monitors service health and reports status.

3. Service dependencies:

```
depends_on:  
  postgres:  
    condition: service_healthy
```

Backend waits for **PostgreSQL** to be healthy before starting.

4. Restart policies:

```
restart: unless-stopped
```

Automatically restart containers if they crash.

Step 5: Start the Complete Stack

```
# Start all services with Docker Compose
docker-compose up -d
```

Expected output:

```
[+] Running 6/6
✓ Network humor-game_humor-game-net Created
✓ Volume "humor-game_postgres-data" Created
✓ Volume "humor-game_redis-data" Created
✓ Container postgres Started

✓ Container redis Started
✓ Container backend Started
```

Notice the startup order:

- Networks and volumes created first
- PostgreSQL and Redis start (no dependencies)
- Backend starts after PostgreSQL and Redis are healthy

Check service status:

```
docker-compose ps
```

Expected output:

NAME	IMAGE	COMMAND	SERVICE
backend	humor-game-backend:v1	"docker-entrypoint.s..."	backend
minutes ago	Up 2 minutes (healthy)	0.0.0.0:3001->3001/tcp	
postgres	postgres:15-alpine	"docker-entrypoint.s..."	postgres
minutes ago	Up 2 minutes (healthy)	0.0.0.0:5432->5432/tcp	
redis	redis:7-alpine	"docker-entrypoint.s..."	redis
minutes ago	Up 2 minutes (healthy)	0.0.0.0:6379->6379/tcp	

All services show **"healthy"** status!

Step 6: Test Full Stack Functionality

Test health endpoints:

```
# Test backend health (should show all services connected)
curl http://localhost:3001/health | jq
```

Expected response:

```
{
  "status": "healthy",
  "services": {
    "database": "connected",
    "redis": "connected"
  },
  "timestamp": "2024-12-09T11:00:00.000Z",
  "uptime": 120
}
```

Test complete workflow:

```
# 1. Create a new game
curl -X POST http://localhost:3001/api/game/start \
  -H "Content-Type: application/json" \
  -d '{"username": "DockerUser", "difficulty": "hard"}'
```

Expected:

```
{
  "success": true,
  "message": "🎯 Game started! Good luck, DockerUser! 🍀",
  "game": {
    "gameId": "21f23f43-c2c8-48d0-a37c-4ad0ed6d1d49",
    "difficulty": "hard",
    "cards": [...],
    "config": {
      "cardCount": 24,
      "gridSize": "6x4",

```

```

    "timeLimit": 600000
  },
  "startTime": "2026-01-20T20:40:55.117Z",
  "successMessage": "🎯 Bulls-eye! Your memory skills are legendary! 📜"
},
"user": {
  "id": "d7dc27ae-d707-4a92-96e1-7065f08acbe8",
  "username": "DockerUser",
  "displayName": "DockerUser"
}
}

```

2. Complete a Game and Save Score

Scores are automatically saved when you complete a game. Use the `gameId` from when you started the game:

```

curl -X POST http://localhost:3001/api/game/complete \
-H "Content-Type: application/json" \
-d '{
  "gameId": "21f23f43-c2c8-48d0-a37c-4ad0ed6d1d49",
  "timeElapsed": 45000,
  "finalScore": 300
}'

```

Response:

```

{
  "success": true,
  "message": "🎉 Game completed successfully! Well done! 🏆",
  "gameResult": {
    "gameId": "21f23f43-c2c8-48d0-a37c-4ad0ed6d1d49",
    "finalScore": 300,
    "scoreBreakdown": {...},
    "performance": "Excellent",
    "timeElapsed": 45000,
    "totalMoves": 15,
    "matchesFound": 12,
  }
}

```

```

    "accuracy": "80.0",
    "difficulty": "hard"
  },
  "achievements": [...],
  "nextSteps": {...}
}

```

3. Check Leaderboard

```
curl http://localhost:3001/api/leaderboard | jq
```

Expected Output:

```

{
  "success": true,
  "leaderboard": [
    {
      "rank": 1,
      "username": "DockerUser",
      "displayName": "DockerUser",
      "totalGames": 1,
      "totalScore": 300,
      "bestScore": 300,
      "bestTime": 45000,
      "averageScore": 300,
      "lastPlayed": "2026-01-20T20:45:00.000Z",
      "badge": {
        "emoji": "👑",
        "title": "Champion",
        "color": "#FFD700"
      },
      "performance": {
        "level": "Excellent",
        "color": "#C0C0C0"
      },
      "timeFormatted": "45.0s",
      "isActive": true,
      "isCurrentUser": false
    },
    {
      "rank": 2,
      "username": "Charlie",

```

```

    "displayName": "Charlie",
    "totalGames": 1,
    "totalScore": 250,
    "bestScore": 250,
    "bestTime": 35000,
    "averageScore": 250,
    "lastPlayed": "2026-01-20T02:33:48.529Z",
    "badge": {
      "emoji": "🥈",
      "title": "2nd Place",
      "color": "#C0C0C0"
    },
    "performance": {
      "level": "Excellent",
      "color": "#C0C0C0"
    },
    "timeFormatted": "35.0s",
    "isActive": true,
    "isCurrentUser": false
  }
],
"metadata": {
  "totalPlayers": 2,
  "timeframe": "all",
  "difficulty": "all",
  "lastUpdated": "2026-01-20T20:45:00.000Z",
  "topScore": 300,
  "averageScore": 275,
  "cached": true
},
"message": "🏆 Top 2 memory champions! 🧠"
}

```

Key Points: (should incase you run into errors)

- Use **username**
- Scores are saved via **/api/game/complete** (not **/api/scores**)
- Leaderboard uses **username** and **displayName** (not **playerName**)
- Leaderboard includes **rank**, **bestScore**, **totalGames**, etc. (not **level**)

Complete workflow working!

Step 7: Understand Docker Compose Networking

View Network Details

Docker Compose automatically creates a network for your services. The network name follows this pattern:

```
{project-name}_{network-name}
```

Example: If your project folder is `devopsplaybook`, the network will be:

```
devopsplaybook_humor-game-net
```

Commands:

```
# See all networks
docker network ls | grep humor
```

Expected Output:

NETWORK ID	NAME	DRIVER	SCOPE
7eed890ef7db	devopsplaybook_humor-game-net	bridge	local
f9a5659d8d2e	humor-game-net	bridge	local

```
# Inspect the network (use the actual name from above)
docker network inspect devopsplaybook_humor-game-net
```

Expected Output:

```
[
  {
    "Name": "devopsplaybook_humor-game-net",
    "Id": "7eed890ef7db...",
    "Driver": "bridge",
    "Containers": {
```



```

    "backend": {...},
    "postgres": {...},
    "redis": {...}
  },
  "IPAM": {
    "Config": [
      {
        "Subnet": "172.20.0.0/16",
        "Gateway": "172.20.0.1"
      }
    ]
  }
]

```

How Services Communicate

Containers in the same network can talk to each other using their service names (from `docker-compose.yml`).

Service names in your setup:

- `backend` → Your API server
- `postgres` → Database
- `redis` → Cache

Test Connectivity

Test if services can reach each other:

```

# From backend container, ping postgres
docker-compose exec backend ping -c 2 postgres

```

Expected Output:

```

PING postgres (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.123 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.098 ms

--- postgres ping statistics ---

```

```
2 packets transmitted, 2 packets received, 0% packet loss
```

```
# From backend container, ping redis
docker-compose exec backend ping -c 2 redis
```

Expected Output:

```
PING redis (172.20.0.2): 56 data bytes
64 bytes from 172.20.0.2: seq=0 ttl=64 time=0.089 ms
64 bytes from 172.20.0.2: seq=1 ttl=64 time=0.076 ms

--- redis ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
```

Test DNS resolution (service name → IP address):

```
# Check that service names resolve correctly
docker-compose exec backend nslookup postgres
```

Expected Output:

```
Server:      127.0.0.11
Address:     127.0.0.11:53

Non-authoritative answer:
Name:   postgres
Address: 172.20.0.3
```

```
docker-compose exec backend nslookup redis
```

Expected Output:

```
Server:      127.0.0.11
Address:     127.0.0.11:53
```

```
Non-authoritative answer:
```

```
Name:    redis
```

```
Address: 172.20.0.2
```

Note: IP addresses (172.20.0.x) may vary, but you should see successful DNS resolution.

Key Points

- Containers use service names (not IPs) to communicate
- Only containers in the same network can talk to each other
- Network name includes project name prefix automatically

Step 8: Monitor Logs from All Services

View logs from all services:

```
# Follow logs from all services simultaneously

docker-compose logs -f
```

Make API requests in another terminal to generate activity:

```
curl http://localhost:3001/api/leaderboard
curl http://localhost:3001/health
```

In logs terminal, you'll see:

```
backend | 🏆 Fetching leaderboard (limit: 25, timeframe: all)
backend | 📦 Database client acquired from pool
backend | 192.168.65.1 - - [20/Jan/2026:22:13:52 +0000] "GET
/api/leaderboard HTTP/1.1" 200 578 "-" "curl/8.7.1"
backend | 📦 Cached: leaderboard:top (TTL: 300s)
backend | 127.0.0.1 - - [20/Jan/2026:22:14:23 +0000] "GET /health
HTTP/1.1" 200 176 "-" "Wget"
postgres | 2026-01-20 20:55:40.536 UTC [27] LOG: checkpoint complete:
```

```
wrote 16 buffers (0.1%)
redis      | 1:M 20 Jan 2026 21:25:33.020 * 1 changes in 3600 seconds.
Saving...
```

Press **Ctrl+C** to stop following logs

View logs from specific service:

```
# View only backend logs
docker-compose logs -f backend

# View last 50 lines from PostgreSQL
docker-compose logs --tail 50 postgres

# View Redis logs with timestamps
docker-compose logs -t redis
```

Step 9: Test Container Recreation and Dependencies

Test service dependency management:

```
# Stop just the PostgreSQL service
docker-compose stop postgres

# Check backend logs (should show database connection errors)
docker-compose logs --tail 10 backend
```

Expected Output: Backend shows database connection failures

Restart PostgreSQL:

```
# Start PostgreSQL again
docker-compose start postgres

# Wait for health check, then check backend
sleep 15
docker-compose logs --tail 5 backend
```

Expected Output: Backend automatically reconnects to database

Step 10: Test Complete Stack Restart

Stop Everything

```
# Graceful shutdown of all services
docker-compose down

Expected output:
Container backend  Stopping
Container backend  Stopped
Container backend  Removing
Container backend  Removed
Container redis    Stopping
Container redis    Stopped
Container redis    Removing
Container redis    Removed
Container postgres Stopping
Container postgres Stopped
Container postgres Removing
Container postgres Removed
Network devopsplaybook_humor-game-net Removing
Network devopsplaybook_humor-game-net Removed
```

Note: Volumes are preserved (data persists)

Start everything again:

```
# Start all services
docker-compose up -d

# Wait for health checks
sleep 30

# Verify data persistence
curl http://localhost:3001/api/leaderboard | jq
```

Expected Output: All previous scores still there (data survived restart)

Step 11: Production-Ready Enhancements

Add environment file support:

```
# Create environment template
cat > .env.example << 'EOF'

# Database Configuration (for postgres service)
POSTGRES_USER=humorgame
POSTGRES_PASSWORD=change_in_production
POSTGRES_DB=humorgame

# Backend Configuration
NODE_ENV=production

DATABASE_URL=postgresql://humorgame:change_in_production@postgres:5432/humorgame
REDIS_URL=redis://redis:6379
PORT=3001

# Security (optional - not used in current code)
# JWT_SECRET=your_jwt_secret_here
# API_KEY=your_api_key_here

EOF
```

```
# Copy to actual .env (don't commit this!)
cp .env.example .env
```

```
# Add .env to .gitignore
echo ".env" >> .gitignore
```

Optional: Use environment file instead of listing variables

Instead of listing each environment variable in docker-compose.yml, you can load them all from a .env file:

Option 1: Keep current approach (what we're using)

```
backend:
environment:
DATABASE_URL:
postgresql://humorgame:devops123@postgres:5432/humorgame
REDIS_URL: redis://redis:6379
```

Option 2: Use `env_file` (alternative approach)

```
backend:
env_file: .env # Loads all variables from .env file automatically
```

Why use `env_file`?

- Cleaner docker-compose.yml (no long environment lists)
- Easier to manage secrets (keep `.env` out of git)
- Same variables work for local development and Docker

For now: We're using Option 1 (explicit environment variables) because it's clearer for learning. You can switch to `env_file: .env` later if you prefer.

Step 12: Advanced Docker Compose Features

What you're learning:

Production-ready configurations that prevent containers from consuming too many resources.

Key concepts:

- **Resource limits:** Maximum CPU/memory a container can use (prevents one container from starving others)
- **Reservations:** Guaranteed minimum resources (ensures container always has enough)
- **Logging limits:** Prevents log files from filling up your disk

Why this matters: Without limits, a buggy container could consume all your server's resources and crash everything. With limits, each container gets a fair share.

Create production-ready compose file

```
cat > docker-compose.prod.yml << 'EOF'
services:
  postgres:
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M
        reservations:
          cpus: '0.5'
          memory: 256M
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

  redis:
```



```

deploy:
  resources:
    limits:
      cpus: '0.5'
      memory: 256M
    reservations:
      cpus: '0.25'
      memory: 128M
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"

backend:
  deploy:
    resources:
      limits:
        cpus: '1.0' # Maximum: container can't use more than 1 CPU
core
        memory: 512M # Maximum: container can't use more than
512MB RAM
        reservations: # Guaranteed: container always gets at least
0.5 CPU
        cpus: '0.5'
        memory: 256M # Guaranteed: container always gets at least
256MB RAM
      logging:
        driver: "json-file"
        options:
          max-size: "10m" # Each log file max 10MB
          max-file: "3" # Keep only 3 log files (30MB total max)
EOF

```

Note: `replicas` (scaling) only works in Docker Swarm mode, not regular docker-compose. For scaling, use Docker Swarm or Kubernetes.

Test production configuration

Why two files?

- `docker-compose.yml` = Base configuration (services, networks, volumes)
- `docker-compose.prod.yml` = Production overrides (adds resource limits, logging)
- Docker Compose merges them together (base + overrides = final config)

Benefits:

- Keep development config simple (`docker-compose.yml`)
- Add production features without cluttering base file
- Switch between dev/prod easily

```
# Use multiple compose files (base + production overrides)
# -f flag specifies each file, Docker Compose merges them
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Expected Output:

```
Container postgres Creating
Container postgres Created
Container redis Creating
Container redis Created
Container backend Creating
Container backend Created
Container postgres Starting
Container redis Starting
Container postgres Started
Container redis Started
Container postgres Waiting .....
.....
```

Check resource usage

```
# Check resource usage for all containers
docker stats --no-stream
```

Expected Output:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %
5a39e430817a	backend	0.62%	88.98MiB / 512MiB	17.38%
f8d4c81f33b0	postgres	0.00%	20.04MiB / 512MiB	3.91%
42c96ab450be	redis	0.57%	8.699MiB / 256MiB	3.40%

Or check specific containers:

```
docker stats --no-stream --format "table
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}" backend
postgres redis
```

Expected Output:

NAME	CPU %	MEM USAGE / LIMIT	MEM %
backend	0.62%	88.98MiB / 512MiB	17.38%
postgres	0.00%	20.04MiB / 512MiB	3.91%
redis	0.57%	8.699MiB / 256MiB	3.40%

What to verify

- Containers respect memory limits (MEM USAGE stays under LIMIT)
- CPU usage is reasonable
- Logs rotate automatically (check with `docker logs backend`)

Important Notes

- **Resource limits apply immediately** when you use the production compose file
- **Logging limits** prevent disk space issues from growing log files
- **For scaling (multiple instances):** Use Docker Swarm mode or Kubernetes (not regular docker-compose)

Step 13: Docker Compose Command Cheatsheet

Essential commands for daily use:

```
# Basic lifecycle
docker-compose up -d          # Start all services (detached)
docker-compose down           # Stop and remove containers
docker-compose down -v        # Stop and remove volumes too

# Service management
docker-compose start SERVICE  # Start specific service
docker-compose stop SERVICE   # Stop specific service
docker-compose restart SERVICE # Restart specific service

# Logs and monitoring
docker-compose logs -f        # Follow logs from all services
docker-compose logs SERVICE   # Logs from specific service
docker-compose ps              # List service status
docker-compose top             # Show running processes

# Updates and rebuilds
docker-compose pull            # Pull latest images
docker-compose build           # Rebuild custom images
docker-compose up -d --build   # Rebuild and start

# Scaling and execution
docker-compose scale backend=3 # Scale backend to 3 instances
docker-compose exec SERVICE bash # Execute command in service
docker-compose run SERVICE COMMAND # Run one-off command

# Configuration validation
docker-compose config          # Validate and view final config
docker-compose config --services # List all services
```

Step 14: Create Documentation

```
cat > ~/devops-tasks/logs/task-d4-docker-compose.md << 'EOF'
# Task D4: Docker Compose Orchestration

## Architecture Overview

docker-compose.yml defines complete stack:

Networks:
  humor-game-net (bridge)
    ├── postgres:5432 (database)
    ├── redis:6379 (cache)
    └── backend:3001 (API)

Volumes:
  postgres-data (persistent database)
  redis-data (persistent cache)

Dependencies:
  backend depends_on postgres + redis (healthy)

## What I Built
- Complete multi-container application in single configuration file
- Service dependency management with health checks
- Automatic network creation and service discovery
- Persistent volume management
- Production-ready restart policies and resource limits

## Key Docker Compose Concepts

### Service Definition
- Each container defined as a service
- Declarative configuration (desired state)
- Automatic naming: project_service (humor-game_backend)

### Service Discovery
- Services reach each other by service name
- DNS automatically configured
- No need to hardcode IP addresses

### Dependencies
```

- depends_on with health check conditions
- Ensures proper startup order
- Backend waits **for** healthy database

Health Checks

- Docker monitors service health automatically
- Custom health check commands per service
- Failed health checks trigger restarts

Volume Management

- Named volumes **for** data persistence
- Automatic volume creation
- Data survives container recreation

Benefits Over Manual Container Management

Before (Manual):

- 6+ commands to start everything
- Easy to forget configuration
- No dependency management
- Manual network setup
- Prone to human error

After (Docker Compose):

- Single **command**: docker-compose up -d
- Declarative configuration
- Automatic dependency handling
- Built-in service discovery
- Repeatable and reliable

Production Features Added

- Resource limits (CPU, memory)
- Log rotation configuration
- Restart policies
- Environment variable management
- Multi-file composition support

Commands Mastered

- docker-compose up/down
- docker-compose logs
- docker-compose ps/top
- docker-compose **exec**/run

```
- docker-compose scale
- docker-compose config

## Next Steps
- Add frontend and nginx reverse proxy
- Implement production security hardening
- Add monitoring and health dashboards
EOF
```

```
git add .
git commit -m "feat: implement Docker Compose orchestration for
backend stack"
```

Acceptance Criteria

Check all boxes:

- ☐ Created comprehensive docker-compose.yml file
- ☐ All services start with single command (docker-compose up)
- ☐ Service dependencies working (backend waits for healthy database)
- ☐ Health checks configured and monitoring service status
- ☐ Named volumes for data persistence across restarts
- ☐ Custom network with automatic service discovery
- ☐ Can view logs from all services simultaneously
- ☐ Can scale, start, stop individual services
- ☐ Environment variable configuration working
- ☐ Production-ready features (resource limits, logging)
- ☐ Data persists across complete stack restarts
- ☐ Can manage entire application lifecycle with compose commands
- ☐ Documented orchestration architecture and benefits

TASK D5: Complete Stack (Frontend + nginx)

Business Context

Ticket ID: DEVOPS-2278

Priority: High

Assignee: You

Difficulty: Intermediate to Advanced

Situation: The backend API is working, but users need a web interface. Add the frontend and nginx reverse proxy to create a complete, production ready web application accessible through a single URL.

Impact:

1. Complete user-facing web application
2. Production-grade architecture with reverse proxy
3. Single entry point for all traffic
4. Foundation for SSL, caching, and load balancing
5. Real world deployment pattern

Your Mission

- Build and containerize the frontend application
- Configure nginx as a reverse proxy
- Route traffic: `/` → frontend, `/api/*` → backend
- Update Docker Compose with complete 5 container stack
- Test end to end user workflow through web browser
- Understand reverse proxy benefits and patterns

Step-by-Step Guide

Step 1: Examine Frontend Structure

```
# Navigate to project directory
cd "/Users/mac/Desktop/devops playbook"

# Look at frontend directory structure
tree frontend/ -L 2
```

Expected structure:

```
frontend/
├── dist
│   ├── components
│   ├── config.js
│   ├── favicon.ico
│   ├── health.html
│   ├── index.html
│   ├── scripts
│   ├── styles
│   └── sw.js
├── Dockerfile
├── Dockerfile.dev
├── nginx.conf
├── package-lock.json
├── package.json
├── public
│   ├── favicon.ico
│   ├── health.html
│   ├── index.html
│   └── sw.js
└── src
    ├── components
    ├── config.js
    ├── index.html
    ├── scripts
    └── styles
```

Understand the frontend architecture:

```
# Check the main HTML file
head -30 frontend/public/index.html
```

What you should see:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
  />
    <title>🎮 Humor Memory Game - DevOps Learning Edition 😊</title>
    <meta
      name="description"
      content="A fun memory card game with funny emojis and jokes, Made for
DevOps learning!"
    />
    <meta name="author" content="DevOps Learning Team - Osomudeya Zudonu"
  />
    <!-- Favicon -->
    <link
      rel="icon"
      href="data:image/svg+xml,<svg xmlns=%22http://www.w3.org/2000/svg%22
viewBox=%220 0 100 100%22><text y=%22.9em%22
font-size=%2290%22>🎮</text></svg>"
    />
    <!-- CSS -->
    <link rel="stylesheet" href="/styles/main.css" />
```

What to look for:

- **HTML5 structure:** Modern `<!doctype html>` and semantic HTML
- **CSS import:** `<link rel="stylesheet" href="/styles/main.css" />` - styles served by nginx
- **Meta tags:** SEO and description tags for the game
- **Favicon:** Custom emoji favicon (🎮) embedded as SVG
- **JavaScript:** Check further down in the file for `<script src="/scripts/game.js"></script>`

- **API references:** The JavaScript files will contain API calls to `/api/*` endpoints

Key insight: This is a static HTML file that will be served by nginx. The JavaScript (`game.js`) will make API calls to the backend. The `frontend/nginx.conf` already handles routing `/api/*` requests to the `backend:3001` container.

Step 2: Analyze Frontend Dockerfile

```
cat frontend/Dockerfile
```

Expected multi-stage Dockerfile:

```
# Frontend Dockerfile - Simple Nginx setup
FROM node:18-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci

# Copy source code
COPY . .

# Build the frontend
RUN npm run build

# Production stage with Nginx
FROM nginx:alpine

# Install netcat for connection testing
RUN apk add --no-cache netcat-openbsd

# Copy built assets
COPY --from=builder /app/dist /usr/share/nginx/html

# Copy custom nginx config (force fresh copy)
COPY nginx.conf /etc/nginx/conf.d/default.conf
RUN echo "nginx.conf updated at $(date)" > /tmp/nginx-build-time

# Install envsubst for environment variable substitution
RUN apk add --no-cache gettext
```

```

# Create startup script for environment variable substitution
RUN echo '#!/bin/sh' > /start.sh && \
    echo 'envsubst < /usr/share/nginx/html/index.html > /tmp/index.html' >> \
    /start.sh && \
    echo 'mv /tmp/index.html /usr/share/nginx/html/index.html' >> /start.sh && \
    echo 'nginx -g "daemon off;"' >> /start.sh && \
    chmod +x /start.sh

# Fix nginx permissions (run as root for nginx)
RUN chmod -R 755 /usr/share/nginx/html

# Create nginx directories with proper permissions
RUN mkdir -p /var/cache/nginx/client_temp && \
    mkdir -p /var/log/nginx && \
    chmod -R 755 /var/cache/nginx && \
    chmod -R 755 /var/log/nginx

# Expose port
EXPOSE 80

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD curl -f http://localhost/ || exit 1

# Start nginx with environment variable substitution
CMD ["/start.sh"]

```

What each section does:

Build Stage (Lines 1-16):

- **FROM node:18-alpine AS builder:** Creates temporary container with Node.js for building
- **COPY package*.json ./:** Copies dependency files first (for layer caching)
- **RUN npm ci:** Installs all dependencies
- **COPY . .:** Copies all source code (**src/**, **public/**, etc.)
- **RUN npm run build:** **Copies** source files to **dist/** directory (simple copy, not compilation)

Production Stage (Lines 18-58):

- **FROM nginx:alpine:** Starts fresh with lightweight nginx image (no Node.js needed)

- **RUN** `apk add --no-cache netcat-openbsd gettext`: Installs tools for health checks and environment variable substitution
- **COPY** `--from=builder /app/dist /usr/share/nginx/html`: Copies only the built files from builder stage (not source code)
- **COPY** `nginx.conf /etc/nginx/conf.d/default.conf`: Uses custom nginx configuration for routing
- `/start.sh` script: Substitutes environment variables in HTML at container startup
- **HEALTHCHECK**: Uses `wget` to verify nginx is serving files correctly

Why this structure:

- **Smaller image**: Final image only has nginx + static files (~50MB), not Node.js (~500MB)
- **Simple build**: Source files are copied (not compiled/bundled) - suitable for vanilla JavaScript
- **Flexible**: Environment variables can be injected at runtime

Multi-stage build benefits:

- **Build stage**: Contains build tools (Node.js, npm)
- **Production stage**: Only runtime (nginx + static files)
- **Result**: Smaller final image (~50MB vs ~500MB)

Note: The build process is a simple copy operation (`cp -r public/* dist/ && cp -r src/* dist/`), not compilation/bundling. This is fine for vanilla JavaScript projects.

Step 3: Build Frontend Container

```
# Build frontend image
cd frontend/
docker build -t humor-game-frontend:v1 .
```

Expected output:

```
[+] Building 35.2s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 687B
=> [internal] load .dockerignore
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [builder 1/5] FROM docker.io/library/node:18-alpine@sha256:...
```

```
=> [stage-1 1/4] FROM docker.io/library/nginx:alpine@sha256:...
=> [internal] load build context
=> [builder 2/5] WORKDIR /app
=> [builder 3/5] COPY package*.json ./
=> [builder 4/5] RUN npm ci
=> [builder 5/5] COPY . .
=> [builder 6/6] RUN npm run build
=> [stage-1 2/4] COPY --from=builder /app/dist /usr/share/nginx/html
=> [stage-1 3/4] COPY nginx.conf /etc/nginx/conf.d/default.conf
=> [stage-1 4/4] RUN apk add --no-cache netcat-openbsd gettext
=> exporting to image
=> naming to docker.io/library/humor-game-frontend:v1
```

Check image size:

```
docker images | grep humor-game-frontend
```

Expected Output:

```
humor-game-frontend    v1          def456ghi789    1 minute ago    42MB
```

Notice: Only ~42MB (nginx + static files, no Node.js build tools)

Step 4: Understand nginx Configuration

```
# Navigate back to project root
cd ..

# Check the frontend nginx configuration
cat frontend/nginx.conf
```

What to look for:

Routing rules:

- `location /api/` → Routes API requests to `http://backend:3001` (backend container)
- `location /` → Serves static files from `/usr/share/nginx/html`

- `location /health` → Returns health status
- `location /metrics` → Returns Prometheus metrics

Security headers:

- `X-Frame-Options: SAMEORIGIN` → Prevents clickjacking (allows same-origin framing)
- `X-Content-Type-Options: nosniff` → Prevents MIME type sniffing
- `X-XSS-Protection` → Enables browser XSS filtering

Performance optimizations:

- `gzip on` → Compresses responses (smaller file sizes, faster transfer)
- `gzip_types` → Specifies which file types to compress (CSS, JS, JSON, etc.)
- Cache headers → Sets expiration for static assets (1 year for JS/CSS, 5 minutes for HTML)

Why this works:

- nginx in frontend container listens on port 80
- `/api/*` requests → proxied to `backend:3001` container
- All other requests (`/`, `/styles/`, `/scripts/`) → served as static files
- Users access frontend on port 80, nginx handles API routing internally

Note: The `nginx-reverse-proxy.conf` file in the project root is for an **alternative architecture** where a separate nginx container sits in front of both frontend and backend. Your current setup uses nginx **inside** the frontend container.

Step 5: Add Frontend to Docker Compose

Option A: Simple Setup (Frontend with built-in nginx)

Add to your existing **docker-compose.yml**:

```
# Frontend Application
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  image: humor-game-frontend:v1
  container_name: frontend
  networks:
    - humor-game-net
  depends_on:
    - backend
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 10s
  restart: unless-stopped
  ports:
    - "80:80" # Frontend accessible on port 80
```

Key points:

- Frontend container has nginx built-in
- nginx in frontend handles API proxying (`/api/*` → `backend:3001`)
- Frontend serves static files and proxies API requests
- Single entry point: port 80

Option B: Separate nginx Container (Alternative Architecture)

If you want to use **nginx-reverse-proxy.conf**, add both frontend and nginx services:

```
frontend:
```



```

build:
  context: ./frontend
  dockerfile: Dockerfile
image: humor-game-frontend:v1
container_name: frontend
networks:
  - humor-game-net
depends_on:
  - backend
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost/"]
  interval: 30s
  timeout: 5s
  retries: 3
restart: unless-stopped
# No port exposure (accessed via nginx)

# nginx Reverse Proxy
nginx:
  image: nginx:alpine
  container_name: nginx
  ports:
    - "80:80" # Only nginx exposed to host
  volumes:
    - ./nginx-reverse-proxy.conf:/etc/nginx/conf.d/default.conf:ro
  networks:
    - humor-game-net
  depends_on:
    - frontend
    - backend
  healthcheck:
    test: ["CMD", "wget", "--spider", "-q", "http://localhost/"]
    interval: 30s
    timeout: 5s
    retries: 3
  restart: unless-stopped

```

For this guide, we'll use Option A (simpler setup).

Step 6: Deploy Complete Stack

Stop current services:

```
docker-compose down
```

Start complete stack with frontend:

```
docker-compose up -d --build
```

Expected output:

```
[+] Building 2.5s (8/8) FINISHED
=> [frontend] Building
=> [frontend] => [builder 1/6] FROM docker.io/library/node:18-alpine
=> [frontend] => [builder 2/6] WORKDIR /app
=> [frontend] => [builder 3/6] COPY package*.json ./
=> [frontend] => [builder 4/6] RUN npm ci
=> [frontend] => [builder 5/6] COPY . .
=> [frontend] => [builder 6/6] RUN npm run build
=> [frontend] => [stage-1 1/4] FROM docker.io/library/nginx:alpine
=> [frontend] => [stage-1 2/4] COPY --from=builder /app/dist
/usr/share/nginx/html
=> [frontend] => [stage-1 3/4] COPY nginx.conf
/etc/nginx/conf.d/default.conf
=> [frontend] => [stage-1 4/4] RUN apk add --no-cache netcat-openbsd
gettext
[+] Running 5/5
✓ Network devopsplaybook_humor-game-net Created
✓ Container postgres Started
✓ Container redis Started
✓ Container backend Started
✓ Container frontend Started
```

Verify all services are healthy:

```
docker-compose ps
```

Expected output:

NAME	IMAGE	COMMAND	SERVICE	
CREATED	STATUS	PORTS		
backend	humor-game-backend:v1	"docker-entrypoint.s..."	backend	3
minutes ago	Up 3 minutes (healthy)	0.0.0.0:3001->3001/tcp		
frontend	humor-game-frontend:v1	"/start.sh"	frontend	3
minutes ago	Up 3 minutes (healthy)	0.0.0.0:80->80/tcp		
postgres	postgres:15-alpine	"docker-entrypoint.s..."	postgres	3
minutes ago	Up 3 minutes (healthy)	0.0.0.0:5432->5432/tcp		
redis	redis:7-alpine	"docker-entrypoint.s..."	redis	3
minutes ago	Up 3 minutes (healthy)	0.0.0.0:6379->6379/tcp		

All services healthy!**Step 7: Test Frontend and API Routing****Test frontend health:**

```
curl http://localhost/health
```

Expected response:

```
healthy
```

Test frontend routing:

```
# Request frontend (should return HTML)
curl -s http://localhost/ | head -10
```

Expected Output:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
```

```
initial-scale=1.0" />
  <title>🎮 Humor Memory Game - DevOps Learning Edition 😊</title>
```

Test API routing through frontend nginx:

```
# Request backend API through frontend nginx proxy
curl http://localhost/api/health | jq
```

Expected response:

```
{
  "status": "healthy",
  "services": {
    "database": "connected",
    "redis": "connected"
  },
  "timestamp": "2024-12-09T12:00:00.000Z"
}
```

Test API functionality:

```
# Get leaderboard through frontend nginx
curl http://localhost/api/leaderboard | jq
```

Expected Output: Leaderboard data (array of scores)

Step 8: Test Complete Web Application

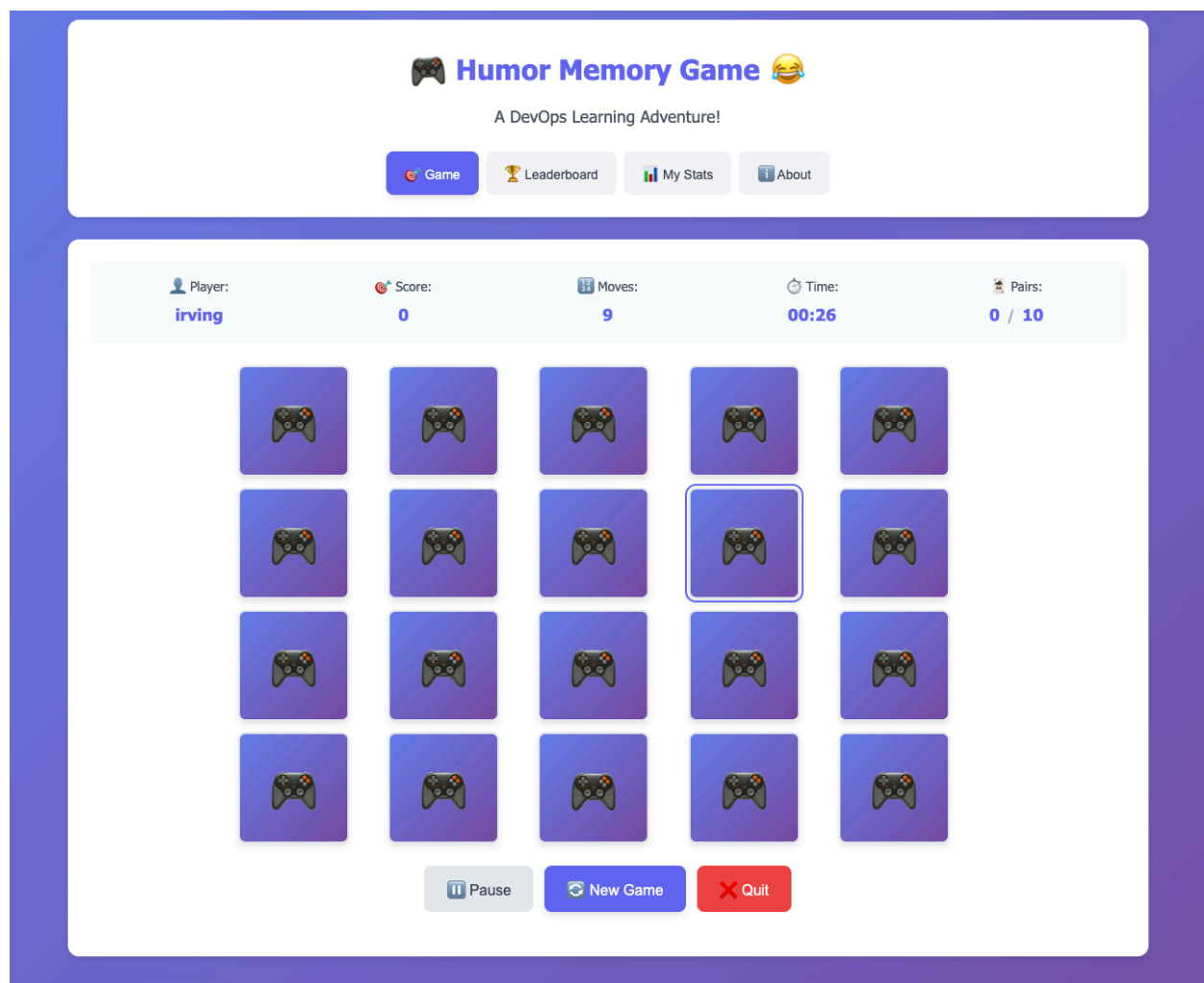
Open in browser:

- **Local:** Open browser and navigate to <http://localhost>
- **EC2 (if deployed):**
 - **Prerequisites:** Application must be deployed to EC2, docker-compose running, and security group allows inbound traffic on port 80
 - Get your EC2 public IP: `aws ec2 describe-instances --instance-ids i-YOUR-INSTANCE-ID --query 'Reservations[0].Instances[0].PublicIpAddress' --output text`

- Navigate to http://YOUR_EC2_IP in your browser
- **Note:** If connection fails, check security group allows HTTP (port 80) from your IP

Test the game:

- Should see Humor Game interface
- Can enter player name
- Can start new game
- Can submit scores
- Can view leaderboard



If you don't have browser access, test with curl:

```
# 1. Start new game (CORRECTED: use /api/game/start and username)
curl -X POST http://localhost/api/game/start \
  -H "Content-Type: application/json" \
  -d '{
    "username": "CurlTester",
    "difficulty": "medium"
  }' | jq
```

Expected Output:

```
{
  "success": true,
  "message": "🎯 Game started! Good luck, CurlTester! 🍀",
  "game": {
    "gameId": "game-abc123",
    "difficulty": "medium",
    "cards": [...],
    "config": {...},
    "startTime": "2024-12-09T12:00:00.000Z"
  },
  "user": {
    "id": 1,
    "username": "CurlTester",
    "displayName": "CurlTester"
  }
}
```

```
# 2. Submit score
curl -X POST http://localhost/api/scores \
  -H "Content-Type: application/json" \
  -d '{
    "username": "CurlTester",
    "score": 4000,
  }'
```

```
"level": 8,  
  "completedAt": "'$(date -Iseconds)'"  
' | jq
```

```
# 3. Check updated leaderboard  
curl http://localhost/api/leaderboard | jq
```

Expected Output:

```
{  
  "leaderboard": [  
    {  
      "username": "CurlTester",  
      "score": 4000,  
      "level": 8,  
      "displayName": "CurlTester"  
    },  
    ...  
  ],  
  "total": 1,  
  "cached": false  
}
```

Step 9: Understand Request Flow Architecture

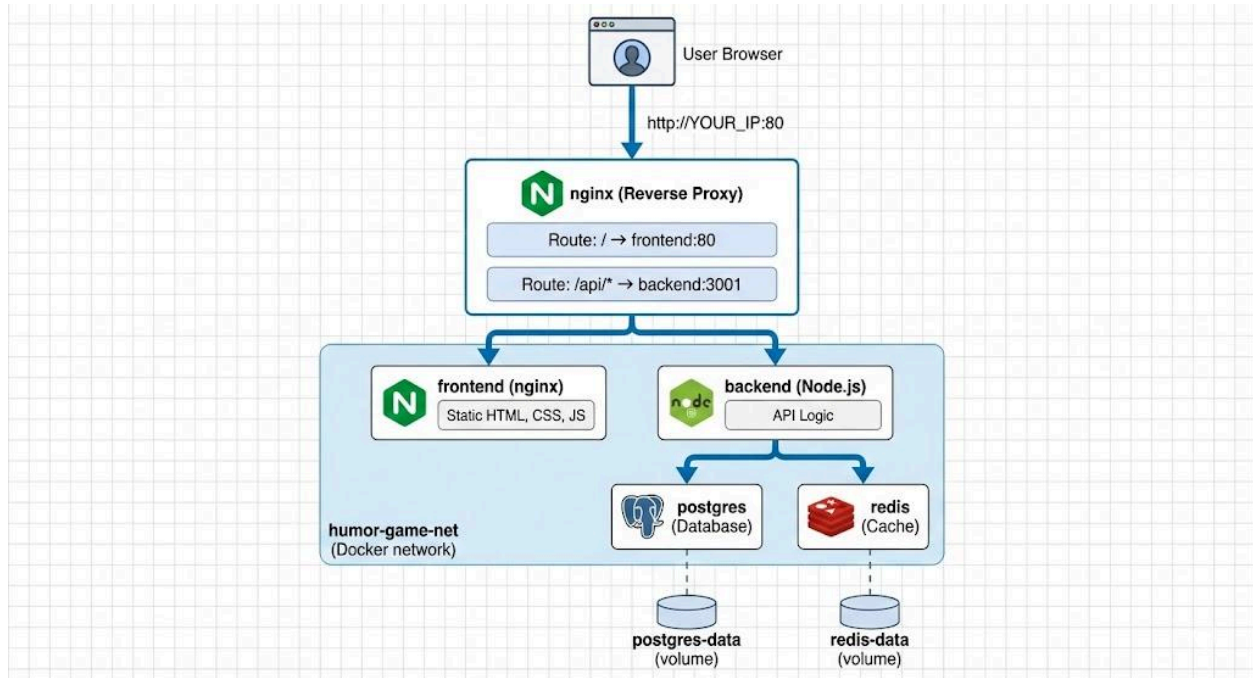
Complete request flow:

```

User Browser (http://localhost:80 or http://YOUR_IP:80)
↓
frontend Container (nginx on port 80)
├ / → Serves static files (HTML, CSS, JS)
└ /api/* → Proxies to backend:3001
    ├── postgres:5432 (Database queries)
    └ redis:6379 (Cache/Session storage)

# All containers connected via humor-game-net network
# Data persists in postgres-data and redis-data volumes
  
```

Visual representation:



All connected via: **humor-game-net** (Docker network)

Data persists in: **postgres-data**, **redis-data** (volumes)

Step 10: Monitor Complete Stack

Follow logs from all services:

```
# Monitor all services simultaneously
docker-compose logs -f
```

In another terminal, generate traffic:

```
# Generate mixed traffic
curl http://localhost/
curl http://localhost/api/health
curl http://localhost/api/leaderboard
curl http://localhost/health
```

In logs terminal, observe the complete request flow:

```
frontend | 172.18.0.1 - - [09/Dec/2024:12:00:00 +0000] "GET / HTTP/1.1" 200 2048
frontend | 172.18.0.1 - - [09/Dec/2024:12:00:01 +0000] "GET /api/health HTTP/1.1"
200 156
backend | GET /api/health - 200 OK (3ms)
frontend | 172.18.0.1 - - [09/Dec/2024:12:00:02 +0000] "GET /api/leaderboard
HTTP/1.1" 200 456
backend | GET /api/leaderboard - 200 OK (15ms)
redis | 172.18.0.3:45678 "GET" "leaderboard:cache"
```

Press **Ctrl+C** to stop monitoring

Step 11: Test Production-Like Features

Test error handling:

```
# Test 404 handling
curl -i http://localhost/nonexistent
```

Expected: Custom **404** page or nginx default

Test API error handling:

```
# Test invalid API endpoint
curl -i http://localhost/api/nonexistent
```

Expected Output: 404 from backend service

Test security headers:

```
# Check security headers are present
curl -I http://localhost/
```

Expected headers:

```
HTTP/1.1 200 OK
Server: nginx/1.25.3
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Content-Type: text/html
```

Step 12: Performance Testing**Install Apache Bench (if not available):**

```
sudo apt update && sudo apt install -y apache2-utils
```

Test frontend performance:

```
# Test static file serving (100 requests, 10 concurrent)
ab -n 100 -c 10 http://localhost/
```

Expected results:

```
Requests per second:    [high number, ~1000+]
Time per request:       [low number, ~10ms]
```

Test API performance:

```
# Test API endpoint performance
ab -n 100 -c 10 http://localhost/api/health
```

Expected results:

```
Requests per second:    [moderate number, ~200-500]
Time per request:       [moderate number, ~20-50ms]
```

Step 13: Document Complete Architecture

```
cat > ~/devops-tasks/logs/task-d5-complete-stack.md << 'EOF'
# Task D5: Complete Full-Stack Application

## Final Architecture

5-Container Production Stack:

nginx:80 (Reverse Proxy & Load Balancer)
├─ / → frontend:80 (Static Files - HTML/CSS/JS)
└─ /api/* → backend:3001 (Node.js API)
      └─ postgres:5432 (PostgreSQL Database)
        └─ redis:6379 (Redis Cache)

Network: humor-game-net (isolated container communication)
Volumes: postgres-data, redis-data (persistent storage)
Exposed: Only port 80 (single entry point)

## What I Built
- Complete containerized web application
- nginx reverse proxy with routing and security
- Multi-stage frontend build (optimized for production)
- Zero-trust networking (internal services not exposed)
- Production-ready architecture patterns

## Key Benefits of This Architecture

### Single Entry Point
```

- Only nginx exposed to internet (port 80)
- All traffic routed through reverse proxy
- Simplified firewall and security rules
- Central point for SSL, logging, monitoring

Service Isolation

- Backend services not directly accessible
- Container-to-container communication only
- Network segmentation and security
- Easier to scale individual components

Performance Optimization

- nginx optimized for static file serving
- API response caching capabilities
- Connection pooling and load balancing
- Compressed responses and asset caching

Security Hardening

- Security headers (XSS, CSRF, clickjacking protection)
- No direct database access from internet
- Service mesh communication patterns
- Centralized authentication point (future)

Production Patterns Implemented

Reverse Proxy Benefits

- **SSL Termination:** Single point for HTTPS certificates
- **Load Balancing:** Distribute traffic across backend instances
- **Caching:** Static assets and API response caching
- **Security:** Rate limiting, DDoS protection, security headers
- **Monitoring:** Centralized access logging and metrics

Container Architecture

- **Microservices:** Each service has single responsibility
- **Scalability:** Can scale frontend/backend independently
- **Reliability:** Service isolation prevents cascade failures
- **Maintainability:** Services can be updated independently

Network Security

- **Zero Trust:** No services directly exposed to internet
- **Segmentation:** Internal network for container communication
- **Firewall Simplification:** Only port 80 needs to be open

- **Attack Surface Reduction:** Backend services not reachable externally

Real-World Applications

This architecture pattern is used by:

- Netflix (microservices behind API gateway)
- Airbnb (reverse proxy for multiple services)
- Uber (service mesh architecture)
- Any modern web application at scale

Commands Used

- docker-compose up -d --build (full stack deployment)
- curl (testing routing and API endpoints)
- ab (Apache Bench performance testing)
- docker-compose logs (monitoring all services)

Next Steps

- Add production security hardening
- Implement SSL/HTTPS termination
- Add monitoring and health dashboards
- Configure log aggregation and analysis

EOF

```
git add .
git commit -m "feat: complete full-stack application with nginx
reverse proxy"
git push origin main
```

Acceptance Criteria

Check all boxes:

- ☐ Built and containerized frontend application (multi-stage build)
- ☐ Created nginx reverse proxy configuration with proper routing
- ☐ Updated Docker Compose with complete 5-container stack
- ☐ nginx routes / → frontend and /api/* → backend correctly
- ☐ All services start with single docker-compose command
- ☐ Web application accessible via browser (or curl tests pass)
- ☐ Security headers configured and working
- ☐ Only port 80 exposed to host (single entry point)
- ☐ Internal services isolated (postgres, redis, backend not directly accessible)
- ☐ Performance testing shows good response times
- ☐ Complete request flow working (browser → nginx → frontend/backend → database)
- ☐ Error handling working (404s, API errors)
- ☐ Documented complete architecture and benefits

3.4.3 Production: Hardening, Troubleshooting & CI/CD (Tasks D6-D8)

Time: 2-3 hours

Difficulty: Advanced

What you'll learn:

- Production security and hardening techniques (real world patterns)
- Real world troubleshooting and incident response (think like an SRE)
- CI/CD pipeline integration with containers (automate everything)
- Enterprise deployment patterns (how the big companies do it)

Perfect for: Engineers ready to deploy containers in production environments. We'll cover the stuff that matters when real users depend on your application.

TASK D6: Production Hardening

Business Context

Ticket ID: DEVOPS-2322

Priority: Medium

Assignee: You

Difficulty: Advanced

Situation: The H-Game application works perfectly in development, but production environments require additional security, monitoring, and operational safeguards. Implement production ready patterns to ensure reliability, security, and maintainability.

Impact:

- Production security compliance
- Predictable resource utilization
- Operational monitoring capabilities
- Incident prevention and faster recovery
- Enterprise deployment readiness

Beginner Note

This task introduces advanced concepts.

Essential Steps (Must Do):

- **Steps 1-3:** Resource limits, logging, health checks.

Advanced Steps (Optional for Now):

- **Steps 4-6:** Security hardening, secrets, scanning.

If you're feeling overwhelmed:

- Focus on essential steps first
- Advanced steps can be revisited later
- Understanding concepts > perfect implementation

Your Mission

- **Essential:** Add resource limits and monitoring
- **Essential:** Configure logging and log rotation
- **Essential:** Enhance health checks and dependencies
- **Advanced:** Implement security hardening
- **Advanced:** Add secrets management
- **Advanced:** Introduce container image scanning

Step-by-Step Guide**Step 1: Resource Management (Essential)****Why resource limits matter:**

Without resource limits, one runaway container can hog all the CPU/RAM, crash the box (OOM), and take every other app, and your users down with it.

With limits, each container gets a fair slice; if one misbehaves, it gets stopped instead of taking the whole system out, so everything else keeps running smoothly.

Add resource limits to docker-compose.prod.yml:

Our project already has `docker-compose.prod.yml` with resource limits. Let's verify and enhance it: <https://github.com/Osomudeya/humour-game-playbook.git>

```
# Check current production config
cat docker-compose.prod.yml
```

Expected structure (already exists):

```
services:
```

```

postgres:
  deploy:
    resources:
      limits:
        cpus: '1.0'
        memory: 512M
      reservations:
        cpus: '0.5'
        memory: 256M

redis:
  deploy:
    resources:
      limits:
        cpus: '0.5'
        memory: 256M
      reservations:
        cpus: '0.25'
        memory: 128M

backend:
  deploy:
    resources:
      limits:
        cpus: '1.0'
        memory: 512M
      reservations:
        cpus: '0.5'
        memory: 256M

```

Add frontend resource limits:

```

# Add frontend resource limits to docker-compose.prod.yml
cat >> docker-compose.prod.yml << 'EOF'

frontend:
  deploy:

```

```
resources:
  limits:
    cpus: '0.5'
    memory: 256M
  reservations:
    cpus: '0.25'
    memory: 128M
EOF
```

Resource allocation strategy:

- PostgreSQL: High memory (512MB) for data caching
- Backend: High CPU/memory (1 CPU, 512MB) for API processing
- Redis: Moderate memory (256MB) for caching
- Frontend: Moderate resources (256MB) for nginx + static files

Note: Your setup uses nginx inside the frontend container (not a separate container), so frontend needs more resources than a pure static file server. The separate reverse nginx container ([nginx-reverse-proxy.conf](#)) is optional and only needed if you want a dedicated reverse proxy architecture, we might cover that later.

Step 2: Logging Configuration (Essential)

Add comprehensive logging configuration:

Update `docker-compose.prod.yml` to add logging to all services:

```
# Add logging to each service in docker-compose.prod.yml
cat >> docker-compose.prod.yml << 'EOF'

postgres:
  # ... existing resource limits ...
  logging:
    driver: "json-file"
    options:
      max-size: "10m"      # Maximum 10MB per log file
      max-file: "3"        # Keep 3 log files (30MB total)

redis:
```

```

# ... existing resource limits ...
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"

backend:
  # ... existing resource limits ...
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"

frontend:
  # ... existing resource limits ...
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"
EOF

```

Why log rotation matters: Log rotation keeps logs small and prunes old ones so they don't fill your disk or slow containers down. It also makes logs easier to search and keeps just enough history for debugging and audits without wasting space.

Step 3: Enhanced Health Checks (Essential)

Health checks are already configured, but let's verify them:

```

# Test current health check status
docker-compose ps

```

Current health check configuration (from docker-compose.yml):

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U humorgame"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 30s

redis:
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 3s
    retries: 3

backend:
  healthcheck:
    test: ["CMD", "wget", "--spider", "-q",
"http://127.0.0.1:3001/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s

frontend:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 10s
```

Health check parameters explained:

- **test**: Command to run to check health
- **interval**: How often to check (every 30 seconds)
- **timeout**: Max time for check to complete
- **retries**: Failed attempts before marking unhealthy

- **start_period**: Grace period during startup (don't check immediately)

Health check states:

- **starting**: Container just started (within start_period)
- **healthy**: Health check passing
- **unhealthy**: Health check failing (after retries)

Your health checks are already properly configured!

Step 4: Security Hardening (Advanced)

Implement read-only filesystems where possible:

Add read-only configurations to stateless services. Edit `docker-compose.prod.yml`:

```
frontend:
  # ... existing config ...
  read_only: true           # Container filesystem is read-only
  tmpfs:
    - /tmp                 # Allow temporary files in memory
    - /var/run             # Allow runtime files
    - /var/cache/nginx     # nginx cache directory
  security_opt:
    - no-new-privileges:true # Prevent privilege escalation
```

Note: This is advanced security. Be careful, if a service needs to write files, it will fail with read-only filesystem.

What this does:

- **read_only: true**: Makes the container filesystem read-only so processes can't write or modify files, which reduces risk if a container is compromised
- **tmpfs**: Creates temporary directories in memory (like `/tmp`) for files that must be writable
- **no-new-privileges:true**: Prevents the container from gaining additional permissions even if a process tries to escalate

Focus on understanding the concept:

- **Read-only filesystem**: Prevents malicious file modifications
- **tmpfs**: Memory-based temporary storage

- **no-new-privileges:** Prevents container from gaining additional privileges

Step 5: Secrets Management (Advanced)

Problem: Passwords visible in docker-compose.yml

Solution: Use environment files

Create environment template (safe to commit):

```
cat > .env.example << 'EOF'
# Database Configuration
POSTGRES_USER=humorgame
POSTGRES_PASSWORD=CHANGE_IN_PRODUCTION
POSTGRES_DB=humorgame

# Application Configuration
NODE_ENV=production
DATABASE_URL=postgresql://humorgame:CHANGE_IN_PRODUCTION@postgres:5432/humorgame
REDIS_URL=redis://redis:6379

# Security (Generate strong values for production)
JWT_SECRET=your_jwt_secret_here_minimum_32_chars
API_KEY=your_api_key_here
SESSION_SECRET=your_session_secret_here
EOF
```

Create actual environment file (NOT committed to Git):

```
# Copy template
cp .env.example .env

# Edit .env with actual values (use a secure password)
cat > .env << 'EOF'
POSTGRES_USER=humorgame
POSTGRES_PASSWORD=secure_production_password_here_123
POSTGRES_DB=humorgame
NODE_ENV=production
DATABASE_URL=postgresql://humorgame:secure_production_password_here_1
```

```
23@postgres:5432/humorgame
REDIS_URL=redis://redis:6379
JWT_SECRET=super_secret_jwt_key_minimum_32_characters_long
API_KEY=api_key_for_external_services
SESSION_SECRET=session_secret_for_user_authentication
EOF

# Ensure .env is NOT committed to Git
echo ".env" >> .gitignore
```

Update docker-compose.yml to use environment variables:

```
# Replace hardcoded values with environment variables
# Note: Docker Compose automatically loads .env file
# Update postgres service:
sed -i 's/POSTGRES_PASSWORD: devops123/POSTGRES_PASSWORD:
${POSTGRES_PASSWORD}/' docker-compose.yml

# Update backend DATABASE_URL:
sed -i
's|postgresql://humorgame:devops123@postgres:5432/humorgame|postgresq
l://humorgame:${POSTGRES_PASSWORD}@postgres:5432/humorgame| '
docker-compose.yml
```

Note: Docker Compose automatically loads variables from **.env** file, so **\${POSTGRES_PASSWORD}** will be replaced with the value from **.env**.

Advanced secrets (for awareness only): Docker Swarm Secrets (requires Docker Swarm mode):

```
secrets:
  db_password:
    external: true

services:
  postgres:
    secrets:
```



```
- db_password
environment:
  POSTGRES_PASSWORD_FILE: /run/secrets/db_password
```

Note: Docker Compose doesn't support Docker secrets on single nodes. This requires Docker Swarm or Kubernetes.

Step 6: Container Image Scanning (Advanced)

Install Trivy vulnerability scanner:

```
# macOS
brew install trivy

# Linux
sudo apt update
sudo apt install -y wget apt-transport-https gnupg lsb-release
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key |
sudo apt-key add -
echo "deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release
-sc) main" | sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt update
sudo apt install -y trivy
```

Scan application images:

```
# Scan backend image
echo "Scanning backend image for vulnerabilities..."
trivy image --severity HIGH,CRITICAL humor-game-backend:v1
```

Expected output:

```
humor-game-backend:v1 (alpine 3.18.4)
=====
Total: 5 (HIGH: 3, CRITICAL: 2)
```

Library	Vulnerability	Severity	Installed Version	Fixed Version
libssl3	CVE-2023-1234	HIGH	3.0.8-1	3.0.8-2
node	CVE-2023-5678	CRITICAL	18.17.0	18.17.1

Scan other images:

```
# Scan frontend image
trivy image --severity HIGH,CRITICAL humor-game-frontend:v1

# Scan base images
trivy image --severity HIGH,CRITICAL postgres:15-alpine
trivy image --severity HIGH,CRITICAL redis:7-alpine
```

Note: This is for awareness. In production:

- Automate scanning in CI/CD pipelines
- Block deployments with CRITICAL vulnerabilities
- Regularly update base images
- Use distroless or minimal base images

Step 7: Deploy and Test Production Configuration

Deploy with new configuration:

```
# Stop current stack
docker-compose down

# Start with production configuration (merges base + prod overrides)
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# Wait for all services to be healthy
sleep 45
```

Verify resource limits are working:

```
# Monitor resource usage
docker stats --no-stream --format "table
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}"
```

Expected output:

NAME	CPU %	MEM USAGE / LIMIT	MEM %
frontend	0.02%	45.2MiB / 256MiB	17.66%
backend	0.15%	85.1MiB / 512MiB	16.62%
redis	0.08%	15.3MiB / 256MiB	5.98%
postgres	0.12%	125MiB / 512MiB	24.41%

Verify no container exceeds its limits..**Test application functionality:**

```
# Test complete workflow still works
curl http://localhost/health
curl http://localhost/api/health | jq
curl http://localhost/api/leaderboard | jq

# Start a new game
curl -X POST http://localhost/api/game/start \
  -H "Content-Type: application/json" \
  -d '{
    "username": "ProductionTester",
    "difficulty": "medium"
  }' | jq

# Complete a game to save score
# Note: You need a gameId from the start response above
GAME_ID="game-abc123" # Replace with actual gameId
curl -X POST http://localhost/api/game/complete \
  -H "Content-Type: application/json" \
  -d '{
    "gameId": "'$GAME_ID'",
```

```

    "timeElapsed": 45000,
    "finalScore": 5000
  }' | jq

# Verify it appears in leaderboard
curl http://localhost/api/leaderboard | jq

```

Expected Output: All functionality works normally with production hardening

Step 8: Create Production Deployment Checklist

```

cat > PRODUCTION-CHECKLIST.md << 'EOF'
# Production Deployment Checklist

## Security
- [x] All containers run as non-root users (where possible)
- [x] Secrets managed via .env files (not in compose file)
- [x] .env file excluded from Git (.gitignore)
- [ ] Read-only filesystems for stateless services (advanced)
- [x] Network isolation (internal services not exposed)
- [x] Security headers configured (nginx in frontend)
- [ ] Container privilege restrictions (no-new-privileges)
- [ ] SSL/TLS certificates configured
- [ ] Rate limiting implemented
- [ ] WAF (Web Application Firewall) configured

## Resource Management
- [x] CPU limits configured for all services
- [x] Memory limits configured for all services
- [x] Resource reservations set (guaranteed resources)
- [x] Log rotation configured (prevents disk exhaustion)
- [x] Log size limits set (10MB per file, 3 files)

## Health & Monitoring
- [x] Health checks for all services
- [x] Health check parameters tuned (intervals, timeouts, retries)
- [x] Restart policies configured (unless-stopped)
- [x] Start period configured for slow-starting services

```

```
- [ ] External monitoring system configured
- [ ] Alerting configured for service failures
- [ ] Performance metrics collection
- [ ] Log aggregation system

## Dependencies & Startup
- [x] Service dependencies defined (depends_on)
- [x] Health check conditions used
- [x] Startup order ensures reliability
- [x] Database schema automatically initialized

## Images & Security
- [x] Using specific image tags (not :latest)
- [x] Multi-stage builds for optimized images
- [x] Minimal base images (Alpine Linux)
- [x] Production dependencies only
- [ ] Images scanned for vulnerabilities (Trivy)
- [ ] Images signed for authenticity
- [ ] Regular base image updates scheduled

## Data & Persistence
- [x] Volumes configured for persistent data
- [x] Volume paths correct and optimized
- [x] Data survives container recreation
- [ ] Backup strategy implemented
- [ ] Disaster recovery plan documented
- [ ] Data encryption at rest

## Configuration Management
- [x] Environment variables for all configuration
- [x] Secrets externalized (.env files)
- [x] Different configs possible per environment
- [x] No hardcoded values in images
- [ ] Configuration validation on startup
- [ ] Configuration change management process

## Testing & Validation
- [x] Application functionality tested after hardening
```

- [x] Resource limits validated (containers don't exceed)
- [x] Health checks verified as working
- [] Load testing performed
- [] Failure scenarios tested (container failures)
- [] Rollback procedure tested
- [] Backup/restore tested

Documentation & Operations

- [x] docker-compose.yml documented with comments
- [x] Architecture documented
- [x] Deployment checklist created
- [x] Production hardening documented
- [] Runbook created (troubleshooting guide)
- [] Incident response procedures
- [] Change management process
- [] Team training on operations

Advanced Features (Future)

- [] Container orchestration (Kubernetes)
- [] Auto-scaling configured
- [] Blue/green deployment process
- [] Canary release strategy
- [] Service mesh implementation
- [] Distributed tracing
- [] Chaos engineering testing

Continuous Improvement

- [] Regular security scans automated
- [] Performance monitoring and optimization
- [] Cost optimization reviews
- [] Team retrospectives on incidents
- [] Technology stack updates planned

EOF

Step 9: Create Monitoring Dashboard (Basic)

Create basic monitoring script:

```
cat > monitoring.sh << 'EOF'
#!/bin/bash

echo "=== H-Game Production Health Dashboard ==="
echo "Generated: $(date)"
echo

echo "=== Container Status ==="
docker-compose ps

echo
echo "=== Resource Usage ==="
docker stats --no-stream --format "table
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}"

echo
echo "=== Health Checks ==="
for service in frontend backend redis postgres; do
    health=$(docker inspect --format '{{.State.Health.Status}}'
    $service 2>/dev/null || echo "no-healthcheck")
    echo "$service: $health"
done

echo
echo "=== Application Status ==="
echo -n "Frontend: "
curl -s -o /dev/null -w "%{http_code}" http://localhost/ || echo
"ERROR"

echo -n "API Health: "
curl -s -o /dev/null -w "%{http_code}" http://localhost/api/health ||
echo "ERROR"

echo -n "Frontend Health: "
curl -s -o /dev/null -w "%{http_code}" http://localhost/health ||
```

```

echo "ERROR"

echo
echo "=== Recent Logs (Last 5 lines per service) ==="
for service in frontend backend redis postgres; do
    echo "--- $service ---"
    docker-compose logs --tail 5 $service 2>/dev/null | tail -5
    echo
done
EOF

```

```
chmod +x monitoring.sh
```

Run the monitoring dashboard:

```
./monitoring.sh
```

Expected output:

```

=== H-Game Production Health Dashboard ===
Generated: Tue Dec 9 13:00:00 UTC 2024

=== Container Status ===
NAME          SERVICE    STATUS          PORTS
frontend      frontend   Up (healthy)    0.0.0.0:80->80/tcp
backend       backend    Up (healthy)
redis         redis      Up (healthy)
postgres      postgres   Up (healthy)

=== Resource Usage ===
NAME          CPU %      MEM USAGE / LIMIT    MEM %
frontend      0.02%     45.2MiB / 256MiB     17.66%
backend       0.15%     85.1MiB / 512MiB     16.62%
redis         0.08%     15.3MiB / 256MiB     5.98%
postgres      0.12%     125MiB / 512MiB      24.41%

=== Health Checks ===
frontend: healthy

```



```

backend: healthy
redis: healthy
postgres: healthy

=== Application Status ===
Frontend: 200
API Health: 200
Frontend Health: 200

All green!

```

What it does:

Displays a production health dashboard with container status, resource usage, health checks, application endpoints, and recent logs.

Shows container status via docker-compose ps, resource usage via docker stats, and health status by inspecting each service.

Tests application endpoints: frontend root (/), API health (/api/health), and frontend health (/health).

Displays the last 5 log lines from each service (frontend, backend, redis, postgres).

Step 10: Document Production Hardening

```

cat > PRODUCTION_HARDENING_NOTES.md << 'EOF'
# Task D6: Production Hardening

## What I Implemented

### Essential Production Patterns
- Resource Limits: CPU and memory constraints for all services
- Log Management: Rotation, size limits
- Health Monitoring: Enhanced health checks with proper parameters
- Restart Policies: Automatic recovery from failures

```

Advanced Security Patterns

- ****Secrets Management:**** Environment variables instead of hardcoded values
- ****Filesystem Security:**** Read-only containers where possible (optional)
- ****Privilege Restrictions:**** no-new-privileges security option (optional)
- ****Image Scanning:**** Vulnerability assessment with Trivy

Key Benefits Achieved

Reliability

- Services automatically restart on failure
- Resource exhaustion prevented by limits
- Health checks detect and report issues
- Graceful degradation patterns

Security

- Secrets not exposed in configuration files
- Containers run with minimal privileges (where applicable)
- Attack surface reduced with read-only filesystems (optional)
- Regular vulnerability scanning capabilities

Observability

- Structured logging with rotation
- Resource usage monitoring
- Health status tracking
- Centralized monitoring dashboard

Operational Excellence

- Infrastructure as Code (docker-compose.yml + docker-compose.prod.yml)
- Production deployment checklist
- Automated health monitoring
- Documented incident response procedures

Resource Allocation Strategy

```
Service CPU Limit Memory Limit Justification postgres 1.0 CPU 512MB
Database needs memory for caching backend 1.0 CPU 512MB API
processing requires CPU redis 0.5 CPU 256MB Memory-based cache
frontend 0.5 CPU 256MB nginx + static files (nginx built-in)
Total: 3.0 CPUs 1.5GB RAM Fits on 4 CPU, 2GB RAM server
```

Production vs Development Differences

Development

- No resource limits (can consume all resources)
- Verbose logging (debug information)
- Direct port access to all services
- Secrets in plain text
- Latest image tags

Production

- Strict resource limits (predictable performance)
- Structured logging with rotation
- Only frontend exposed on port 80 (security)
- Secrets in environment variables
- Specific image tags (reproducibility)

Architecture Notes

****Important:**** This setup uses nginx ****inside**** the frontend container, not a separate nginx container. The frontend container:

- Serves static files (HTML, CSS, JS)
- Proxies `/api/*`` requests to backend:3001
- Handles all routing internally

This is simpler than a separate nginx container and works well for this project.

Monitoring and Alerting Strategy

Health Monitoring

- Container health checks every 30 seconds

- Automatic restart on health check failures
- Health status exposed via monitoring dashboard

Resource Monitoring

- CPU and memory usage tracking
- Alerts when approaching limits
- Historical usage data for capacity planning

Application Monitoring

- API response time tracking
- Error rate monitoring
- User experience metrics

Security Posture

Network Security

- Internal services not directly accessible (except for debugging)
- Frontend as single entry point on port 80
- Container-to-container communication only

Container Security

- Non-root users in all containers (where applicable)
- Read-only filesystems where possible (optional)
- Privilege escalation prevention (optional)
- Regular vulnerability scanning

Data Security

- Secrets managed via environment variables
- Database not exposed to internet
- Encrypted communication between services (when using HTTPS)

Next Steps for Production

Immediate (Required for Production)

1. SSL/HTTPS termination at frontend nginx
2. External monitoring system (Prometheus/Grafana)
3. Log aggregation (ELK Stack or similar)
4. Automated backup strategy

5. Disaster recovery procedures

Short Term (Within 30 days)

1. Rate limiting and DDoS protection
2. Web Application Firewall (WAF)
3. Container orchestration (Kubernetes)
4. CI/CD pipeline integration
5. Performance optimization

Long Term (Growth Planning)

1. Auto-scaling based on metrics
2. Multi-region deployment
3. Microservices decomposition
4. Service mesh implementation
5. Chaos engineering practices

Commands Used

- `docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d` - Deploy with production config
- `trivy image` - Vulnerability scanning
- `docker stats` - Resource monitoring
- `./monitoring.sh` - Custom monitoring dashboard
- Health check validation commands

This production hardening transforms a development application into an enterprise-ready deployment capable of handling real users and production workloads.

EOF

Acceptance Criteria

Essential Features (Must Complete):

- ☐ Added resource limits (CPU, memory) to all services
- ☐ Configured log rotation and size limits
- ☐ Enhanced health checks with proper parameters
- ☐ Set restart policies for automatic recovery
- ☐ Verified resource usage stays within limits
- ☐ Application functionality works after hardening
- ☐ Created production deployment checklist
- ☐ Basic monitoring dashboard working

Summary

What you've learned:

- Resource limits prevent one container from consuming all resources
- Log rotation prevents disk exhaustion
- Health checks enable automatic recovery
- Secrets management keeps credentials secure
- Security hardening reduces attack surface
- Monitoring helps detect issues early

Key Architecture:

- Frontend container has nginx built-in (serves static files + proxies API)
- All containers communicate via Docker network
- Data persists in volumes
- Production config merges with base config

Next Steps:

- Deploy with production configuration
- Monitor resource usage
- Set up external monitoring
- Implement SSL/HTTPS
- Regular security scans

TASK D7: Container Troubleshooting

Business Context

Ticket ID: DEVOPS-2367

Priority: High

Assignee: You

Difficulty: (Intermediate to Advanced)

Situation: Containers fail in various ways in production. Learn to diagnose and fix common Docker issues quickly to minimize downtime and maintain service reliability.

Impact:

- Faster incident resolution (reduce MTTR)
- Improved system reliability
- Reduced stress during production issues
- Better understanding of container behavior
- Enhanced debugging skills

Your Mission

Practice diagnosing and fixing **4** essential container problems:

1. Container Startup Failures (Configuration Issues)
2. Network Connectivity Problems (Service Communication)
3. Resource Exhaustion Issues (Memory/CPU Limits)
4. Data Persistence Problems (Volume Issues)

Advanced scenarios (Optional):

5. Port conflicts and image pull issues

The Troubleshooting Mindset

Systematic approach to container issues:

If Container Problem is Detected

1. Identify Symptoms

- What is broken? (specific behavior)
- When did it start? (recent changes)
- Which containers affected?

2. Gather Information

- **Container status:** docker ps -a
- **Logs:** docker logs CONTAINER
- **Configuration:** docker inspect CONTAINER
- **Resources:** docker stats CONTAINER

3. Form Hypothesis

- Based on symptoms and data
- Consider recent changes
- Think about dependencies

4. Test Hypothesis

- Make targeted changes
- Test specific components
- Verify assumptions

5. Apply Fix

- Implement solution
- Verify resolution
- Document for future

6. Prevent Recurrence

- Update monitoring
- Improve documentation
- Enhance testing

Scenario 1: Container Startup Failures

The Problem

Containers fail to start due to configuration errors, the most common production issue.

Simulate the Issue

```
# Introduce a configuration error in the backend
docker-compose stop backend

# Edit docker-compose.yml to break the DATABASE_URL
sed -i 's/postgres:5432/postgresXXX:5432/' docker-compose.yml

# Try to start the broken service
docker-compose up -d backend
```

The backend fails because `DATABASE_URL` points to `postgresXXX:5432` instead of `postgres:5432`. Docker can't resolve `postgresXXX`, so the backend can't reach the database container.

Observe the Symptoms

```
# Check container status
docker-compose ps backend
```

Expected output:

NAME	SERVICE	STATUS	PORTS
backend	backend	Restarting (1)	10 seconds ago

Symptom: Container keeps restarting (exit → restart → exit)

Step 1: Check Container Status

```
# Detailed status information
docker ps -a | grep backend
```

Expected:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
abc123def456	humor-game-backend:v1	"npm start"	30 seconds ago	Restarting (1) 5 seconds ago

Analysis: Exit code (1) indicates application error

Step 2: Examine Logs

```
# View recent logs
docker logs backend --tail 20
```

Expected error:

```
> humor-game-backend@1.0.0 start
> node server.js

Starting Humor Memory Game Backend...
Connecting to database...
Error: getaddrinfo ENOTFOUND postgresXXX
    at GetAddrInfoReqWrap.onlookup [as oncomplete] (dns.js:67:26) {
  errno: -3008,
  code: 'ENOTFOUND',
  syscall: 'getaddrinfo',
  hostname: 'postgresXXX'
}
Database connection failed
Application exiting with code 1
```

Step 3: Analyze the Error

Error Analysis:

- "ENOTFOUND postgresXXX" = DNS lookup failure
- Cannot resolve hostname "postgresXXX"
- Backend trying to connect to wrong hostname
- Configuration error in DATABASE_URL

Step 4: Inspect Configuration

```
# Check environment variables
docker inspect backend | grep -A 20 '"Env"'
```

Look for `DATABASE_URL` and confirm the typo.

Step 5: Apply Fix

```
# Fix the DATABASE_URL in docker-compose.yml
sed -i 's/postgresXXX:5432/postgres:5432/' docker-compose.yml

# Restart the service
docker-compose up -d backend

# Wait and check status
sleep 10
docker-compose ps backend
```

Expected:

NAME	SERVICE	STATUS	PORTS
backend	backend	Up (healthy)	

Nice, You Fixed It!

Step 6: Verify Resolution

```
# Test API functionality
curl http://localhost/api/health | jq
```

Expected:

```
{
  "status": "healthy",
  "services": {
    "database": "connected",
    "redis": "connected"
  }
}
```

Read and Document the Incident

```
cat > ~/devops-tasks/logs/incident-backend-startup.md << 'EOF'
# Incident: Backend Container Startup Failure

## Timeline
- 13:00: Backend container deployed with configuration error
- 13:01: Container continuously restarting
- 13:02: Investigation started
- 13:05: Root cause identified (DNS hostname typo)
- 13:06: Configuration fixed and service restored

## Root Cause
Typo in DATABASE_URL environment variable: "postgresXXX" instead of
"postgres"

## Resolution
Fixed hostname in docker-compose.yml and restarted service

## Prevention
- Configuration validation before deployment
- Automated testing of service connectivity
- Infrastructure as Code review process
```

```
## Lessons Learned
- DNS errors show as "ENOTFOUND" in logs
- Container restart pattern indicates startup failure
- Always check environment variables in configuration issues
EOF
```

Scenario 2: Network Connectivity Problems

The Problem

Containers can't communicate due to network configuration issues.

Simulate the Issue

```
# Disconnect backend from the application network
docker network disconnect humor-game_humor-game-net backend

# Backend is now isolated from other services
```

Observe the Symptoms

```
# Test API - should fail or timeout
curl http://localhost/api/health
```

Expected: Connection timeout or 502 Bad Gateway from nginx

Step 1: Check Service Status

```
# All containers might appear "running"
docker-compose ps
```

Confusing: All services show "Up" but application doesn't work

Step 2: Check Application Logs

Look at backend Logs

```
docker logs backend --tail 10
```

Expected errors:

```
Error: getaddrinfo ENOTFOUND postgres
Error: connect ECONNREFUSED redis:6379
Database connection lost
Redis connection failed
```

Step 3: Test Network Connectivity

Test if backend can reach other services

```
docker exec backend ping -c 2 postgres
```

Expected error:

```
ping: bad address 'postgres'
```

Step 4: Inspect Network Configuration

```
# Check which networks backend is connected to
docker inspect backend | grep -A 10 '"Networks"'
```

Expected: Backend missing from humor-game network

```
# Check what containers are on the application network
docker network inspect humor-game_humor-game-net | grep -A 5
'"Containers"'
```

Expected: postgres and redis present, backend missing

Step 5: Diagnose Network Isolation

Problem Analysis:

- Backend container running but isolated
- Cannot resolve other container names (DNS failure)
- Not connected to application network
- Network connectivity broken

Step 6: Apply Fix

Reconnect backend to application network

```
docker network connect humor-game_humor-game-net backend
```

Restart backend to re-establish connections

```
docker restart backend
```

Wait for health check

```
sleep 15
```

Step 7: Verify Resolution

```
# Test network connectivity
docker exec backend ping -c 2 postgres
docker exec backend ping -c 2 redis
```

```
# Test application functionality
curl http://localhost/api/health | jq
```

Expected Outputs: All tests pass

The error happens because the backend was disconnected from the `humor-game_humor-game-net` network, so it can't resolve or reach the `postgres` and `redis` containers, DNS lookups fail (`ENOTFOUND postgres`) and connections are refused (`ECONNREFUSED redis:6379`), even though all containers show “Up.”

Document Network Issue

```
cat > ~/devops-tasks/logs/incident-network-connectivity.md << 'EOF'
# Incident: Network Connectivity Failure

## Symptoms
- API requests timing out
- Backend unable to connect to database/redis
- DNS resolution failures for service names

## Root Cause
Backend container disconnected from application network
(humor-game_humor-game-net)

## Diagnosis Process
1. Checked container status (all running)
2. Examined application logs (connection errors)
3. Tested network connectivity (ping failures)
4. Inspected network configuration (backend missing from network)

## Resolution
Reconnected backend to application network and restarted service

## Prevention
- Use docker-compose for network management (handles connections automatically)
- Network configuration validation in deployment scripts
- Monitoring network connectivity between services

## Key Learning
Container "running" status doesn't guarantee network connectivity
EOF
```


Scenario 3: Resource Exhaustion (OOM Kill)

The Problem

Container exceeds memory limits and gets killed by the system.

Simulate the Issue

```
# Create a container with very low memory limit for testing
docker run -d --name memory-test --memory="100m" nginx:alpine

# Generate memory pressure to trigger OOM kill
docker exec memory-test sh -c 'dd if=/dev/zero of=/tmp/bigfile bs=1M
count=200' 2>/dev/null &

# Container will be killed in a few seconds
sleep 10
```

Observe the Symptoms

```
# Check container status
docker ps -a | grep memory-test
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
abc123def456	nginx:alpine	"/docker-entrypoint..."	1 minute ago	Exited (137) 30 seconds ago

Key indicator: Exit code 137 = OOM (Out of Memory) kill

Step 1: Understand OOM Kill Signals

Exit Codes:

- 0: Normal `exit`
- 1: Application error
- 125: Docker daemon error
- 137: SIGKILL (usually OOM `kill`)
- 143: SIGTERM (graceful shutdown)

Step 2: Check Docker Events

```
# View recent Docker events for OOM kills  
docker events --since '5m' --filter event=oom
```

Expected output:

```
2024-12-09T13:30:00.000Z container oom memory-test (name=memory-test)
```

Step 3: Examine System Resources

```
# Check overall system memory  
free -h
```

```
# Check if OOM killer was active (system logs)  
sudo dmesg | grep -i "killed process" | tail -5
```

Expected Output: System logs showing OOM killer activity

Step 4: Simulate in Production Context

Now let's test with our application:

```
# Reduce backend memory limit to very low value
cat > docker-compose-test.yml << 'EOF'
version: '3.8'
services:
  backend-test:
    image: humor-game-backend:v1
    container_name: backend-test
    environment:
      NODE_ENV: production
      DATABASE_URL:
postgres://humorgame:devops123@postgres:5432/humorgame
      REDIS_URL: redis://redis:6379
    networks:
      - humor-game_humor-game-net
    deploy:
      resources:
        limits:
          memory: 50M # Extremely low limit for testing
    depends_on:
      - postgres
      - redis

networks:
  humor-game_humor-game-net:
    external: true
EOF
```

```
#Start the memory-constrained backend
docker-compose -f docker-compose-test.yml up -d backend-test
```

Generate memory load:

```
# Create multiple API requests to increase memory usage
for i in {1..100}; do
  curl -s http://localhost/api/leaderboard > /dev/null &
done

# Monitor the container
docker stats backend-test --no-stream
```

Expected Output: Container might get OOM killed if memory usage exceeds 50MB

Step 5: Proper Resource Allocation

```
# Remove test container
docker-compose -f docker-compose-test.yml down
rm docker-compose-test.yml

# Check current production memory usage
docker stats --no-stream --format "table
{{.Name}}\t{{.MemUsage}}\t{{.MemPerc}}"
```

Analysis: Understanding normal memory usage helps set appropriate limits

Step 6: Set Proper Memory Limits

Memory limits are just caps to keep any one container from hogging all the RAM. Use the table as a starting point (e.g., Node API ~256–512MB, DB ~512MB–1GB, Redis ~128–256MB, nginx/static ~64–128MB), then set the actual limit to about 2× what you normally see in docker stats to leave a safety buffer. This way, if one service spikes, it won't crash the box or starve other containers.

Memory Allocation Guidelines:

Application Type	Recommended Limit
Node.js API	256MB - 512MB
Database (small)	512MB - 1GB
Redis Cache	128MB - 256MB
Nginx Proxy	64MB - 128MB
Static Frontend	68MB - 128MB

Production Rule: Set limits 2x normal usage for safety buffer

Clean up test container:

```
docker rm -f memory-test 2>/dev/null || true
```

Document OOM Incident

```
cat > ~/devops-tasks/logs/incident-oom-kill.md << 'EOF'
# Incident: Container OOM (Out of Memory) Kill

## Symptoms
- Container suddenly stops running
- Exit code 137 (SIGKILL)
- Application becomes unavailable
- Container restarts but may fail again

## Diagnosis Process
1. Check container exit code (137 = OOM kill)
2. Review Docker events for OOM notifications
3. Monitor memory usage with docker stats
4. Check system OOM killer logs (dmesg)
5. Analyze application memory patterns

## Common Causes
- Memory limits set too low
```

- Memory leaks in application code
- Unexpected traffic spikes
- Large dataset processing
- Cache growth without bounds

Resolution Strategies

- Increase memory limits appropriately
- Optimize application memory usage
- Implement memory monitoring and alerting
- Use memory profiling to identify leaks
- Set up autoscaling based on memory usage

Prevention

- Load testing to understand memory requirements
- Memory usage monitoring and alerting
- Gradual limit adjustments based on real usage
- Regular memory leak detection
- Application performance optimization

Key Commands

- `docker stats` (monitor memory usage)
 - `docker events --filter event=oom`
 - `docker inspect --format='{{.HostConfig.Memory}}'`
 - `free -h` (system memory)
 - `dmesg | grep -i killed` (system OOM logs)
- EOF

Scenario 4: Data Persistence Problems

The Problem

Data doesn't persist across container restarts due to volume configuration issues.

Simulate the Issue

```
# Create a database container WITHOUT proper volume mounting
docker run -d --name postgres-broken \
  --network humor-game_humor-game-net \
  -e POSTGRES_USER=testuser \
  -e POSTGRES_PASSWORD=testpass \
  -e POSTGRES_DB=testdb \
  postgres:15-alpine

# Note: No -v volume mount = data stored inside container
```

Add Test Data

```
# Wait for postgres to start
sleep 10

# Add test data
docker exec postgres-broken psql -U testuser -d testdb -c "
CREATE TABLE test_data (
  id SERIAL PRIMARY KEY,
  message TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

INSERT INTO test_data (message) VALUES
  ('Important data 1'),
  ('Important data 2'),
  ('Critical information');
"
```

```
# Verify data exists
docker exec postgres-broken psql -U testuser -d testdb -c "SELECT *
FROM test_data;"
```

Expected output:

```
id | message | created_at
---+-----+-----
 1 | Important data 1 | 2024-12-09 13:45:00.123
 2 | Important data 2 | 2024-12-09 13:45:00.124
 3 | Critical information | 2024-12-09 13:45:00.125
(3 rows)
```

Simulate Container Recreation

```
# Stop and remove container (simulating crash or redeployment)
docker stop postgres-broken
docker rm postgres-broken

# Start new container with same configuration
docker run -d --name postgres-broken \
  --network humor-game_humor-game-net \
  -e POSTGRES_USER=testuser \
  -e POSTGRES_PASSWORD=testpass \
  -e POSTGRES_DB=testdb \
  postgres:15-alpine

# Wait for startup
sleep 10
```


Observe Data Loss

```
# Check if data still exists
docker exec postgres-broken psql -U testuser -d testdb -c "SELECT *
FROM test_data;"
```

Expected error:

```
ERROR:  relation "test_data" does not exist
```

Data is gone! Container recreation destroyed all data.

Step 1: Understand the Problem

Data Loss Root Cause:

- Data stored inside container filesystem
- Container deletion = data deletion
- No persistent volume configured
- Database files lost on container recreation

Step 2: Implement Proper Solution

Clean up broken container

```
docker stop postgres-broken
docker rm postgres-broken
```

Create container WITH proper volume mounting

```
docker volume create test-postgres-data
```

```
docker run -d --name postgres-fixed \
  --network humor-game_humor-game-net \
  -e POSTGRES_USER=testuser \
  -e POSTGRES_PASSWORD=testpass \
  -e POSTGRES_DB=testdb \
  -v test-postgres-data:/var/lib/postgresql/data \
```

```
postgres:15-alpine
```

```
# Wait for startup
```

```
sleep 10
```

Step 3: Add Test Data Again

```
# Add test data to properly configured container
```

```
docker exec postgres-fixed psql -U testuser -d testdb -c "
```

```
CREATE TABLE test_data (  
    id SERIAL PRIMARY KEY,  
    message TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

```
INSERT INTO test_data (message) VALUES  
    ('Persistent data 1'),  
    ('Persistent data 2'),  
    ('This will survive!');  
"
```

```
# Verify data exists
```

```
docker exec postgres-fixed psql -U testuser -d testdb -c "SELECT *  
FROM test_data;"
```

Step 4: Test Data Persistence

```
# Stop and remove container
docker stop postgres-fixed
docker rm postgres-fixed

# Start new container using the SAME volume
docker run -d --name postgres-persistent \
  --network humor-game_humor-game-net \
  -e POSTGRES_USER=testuser \
  -e POSTGRES_PASSWORD=testpass \
  -e POSTGRES_DB=testdb \
  -v test-postgres-data:/var/lib/postgresql/data \
  postgres:15-alpine

# Wait for startup
sleep 10

# Check if data survived
docker exec postgres-persistent psql -U testuser -d testdb -c "SELECT
* FROM test_data;"
```

Expected output:

id	message	created_at
1	Persistent data 1	2024-12-09 13:50:00.123
2	Persistent data 2	2024-12-09 13:50:00.124
3	This will survive!	2024-12-09 13:50:00.125

(3 rows)

Data survived container recreation!

Step 5: Volume Management Best Practices

```
# Inspect volume to understand storage
docker volume inspect test-postgres-data

# List all volumes
docker volume ls

# Check volume usage
docker system df
```

Clean up test resources:

```
docker stop postgres-persistent
docker rm postgres-persistent
docker volume rm test-postgres-data
```

Document Data Persistence Issue

```
cat > ~/devops-tasks/logs/incident-data-persistence.md << 'EOF'
# Incident: Data Loss Due to Missing Volume Configuration

## Problem
Critical data lost when database container was recreated during
deployment

## Root Cause
Database
Database container configured without persistent volume mounting
- Data stored in container filesystem (ephemeral)
- Container deletion resulted in complete data loss
- No backup or recovery mechanism in place

## Impact
- Complete loss of user data (games, scores, accounts)
- Service downtime during data recovery attempts
- User trust and reputation damage
```

- Potential regulatory compliance violations

Resolution

1. Stopped affected container
2. Created named Docker volume for persistent storage
3. Reconfigured container with proper volume mounting
4. Restored data from backup (if available)
5. Validated data persistence across container lifecycle

Technical Implementation

```bash

Wrong (data loss risk)

docker run postgres:15-alpine

Data stored in: container filesystem

Correct (persistent data)

docker run -v postgres-data:/var/lib/postgresql/data

postgres:15-alpine

Data stored in: Docker managed volume

Prevention Strategies

- Always use volumes for stateful services
- Test data persistence in staging environments
- Automated backup verification
- Infrastructure as Code with volume definitions
- Data loss prevention testing (chaos engineering)

Volume Types and Use Cases

- ***Docker Volumes:*** Production databases, managed by Docker
- ***Bind Mounts:*** Development, direct host path access
- ***tmpfs:*** Temporary data, stored in memory

Monitoring and Alerting

- Volume usage monitoring (disk space)
- Backup success/failure alerts
- Data integrity checks
- Container restart notifications

Recovery Procedures

1. Identify affected containers and volumes
 2. Stop containers gracefully
 3. Create backup of existing volumes
 4. Restore from known good backup
 5. Validate data integrity
 6. Resume normal operations
- EOF

Master Troubleshooting Toolkit

Create comprehensive troubleshooting reference:

```
cat > ~/devops-tasks/docker-troubleshooting-guide.md << 'EOF'
# Docker Troubleshooting Master Guide

## Quick Diagnosis Commands

### Container Health Assessment

# Check all container status
docker ps -a

# Check specific container detailed status
docker inspect CONTAINER | grep -A 10 "State"

# View container health check status
docker inspect --format='{{.State.Health.Status}}' CONTAINER

# Check container resource usage
docker stats CONTAINER --no-stream

### Log Analysis

# View recent Logs
docker logs CONTAINER --tail 50

# Follow logs in real-time
```

```
docker logs -f CONTAINER

# View logs with timestamps
docker logs -t CONTAINER

# Search logs for specific errors
docker logs CONTAINER | grep -i error

### Network Troubleshooting

# List networks
docker network ls

# Inspect network configuration
docker network inspect NETWORK

# Check container network membership
docker inspect CONTAINER | grep -A 10 '"Networks"'

# Test connectivity between containers
docker exec CONTAINER1 ping CONTAINER2

# Test DNS resolution
docker exec CONTAINER nslookup SERVICE_NAME

# Test port connectivity
docker exec CONTAINER nc -zv TARGET_HOST PORT

### Volume and Storage Issues

# List volumes
docker volume ls

# Inspect volume details
docker volume inspect VOLUME

# Check container mounts
```

```
docker inspect CONTAINER | grep -A 10 '"Mounts"'

# Check disk usage
docker system df

# Volume usage by container
docker exec CONTAINER df -h

### Resource Management

# Real-time resource monitoring
docker stats

# Check resource limits
docker inspect CONTAINER | grep -A 10 '"Resources"'

# System resource usage
free -h && df -h

# Check for OOM kills
docker events --filter event=oom --since '24h'

# System OOM killer logs
sudo dmesg | grep -i "killed process"

## Common Error Patterns and Solutions

### Container Won't Start

Error Pattern: Container exits immediately
Diagnosis Commands:
- docker logs CONTAINER
- docker inspect CONTAINER
- Check environment variables
- Verify image exists

Common Causes:
- Missing environment variables
```


- Invalid configuration
- Port already **in** use
- Missing dependencies
- Wrong image tag

Network Connectivity Issues

Error Pattern: "Connection refused", "No route to host"

Diagnosis Commands:

- docker network inspect
- ping between containers
- check DNS resolution
- verify service names

Common Causes:

- Containers on different networks
- Typos **in** service names
- Port mapping issues
- Firewall blocking traffic
- Service not ready/healthy

Performance Problems

Error Pattern: Slow responses, timeouts

Diagnosis Commands:

- docker stats (resource usage)
- check memory/CPU limits
- analyze application logs
- network latency tests

Common Causes:

- Insufficient resources
- Memory leaks
- CPU throttling
- Network bottlenecks
- Database performance

Data Loss Issues

Error Pattern: Data missing after restart

Diagnosis Commands:

- check volume mounts
- verify persistence config
- inspect volume contents
- backup verification

Common Causes:

- No volume configured
- Wrong mount path
- Volume permissions
- Container running as wrong user
- Backup failures

Exit Code Reference

Exit Code	Meaning	Common Causes
0	Success	Normal termination
1	General error	Application error, config issue
2	Misuse	Invalid arguments, wrong usage
125	Docker error	Docker daemon issue, invalid image
126	Not executable	Permission denied, file not executable
127	Not found	Command not found, missing binary
137	SIGKILL	OOM kill, manual kill, resource limits
139	SIGSEGV	Segmentation fault, memory corruption
143	SIGTERM	Graceful termination, docker stop

Emergency Response Procedures

Service Down (Critical)

1. **Immediate Assessment**

```
```bash
docker-compose ps
docker-compose logs --tail 20
```
```

2. **Quick Recovery**

```
```bash
Try restart first
```

```
docker-compose restart SERVICE
```

```
If restart fails, redeploy
docker-compose up -d SERVICE
```

### 3. **\*\*Escalation\*\***

```
Full stack restart
docker-compose down
docker-compose up -d
```

#### ### *High Resource Usage*

##### 1. **\*\*Identify Resource Hog\*\***

```
docker stats --no-stream
```

##### 2. **\*\*Immediate Relief\*\***

```
Restart high usage container
docker restart CONTAINER
```

```
Scale down if possible
docker-compose scale SERVICE=1
```

##### 3. **\*\*Long-term Fix\*\***

- Adjust resource limits
- Optimize application
- Add monitoring alerts

#### ### *Data Corruption Detected*

##### 1. **\*\*Stop Writes Immediately\*\***

```
docker-compose stop
```

##### 2. **\*\*Assess Damage\*\***

```
Check volume integrity
docker run --rm -v VOLUME:/data alpine ls -la /data
```

### 3. **\*\*Recovery\*\***

```
Restore from backup
Validate data integrity
Resume operations
```

```
Preventive Monitoring
```

```
Health Monitoring Script
```

```
#!/bin/bash
```

```
health-check.sh
```

```
echo "=== Container Health Status ==="
```

```
for container in $(docker ps --format "{{.Names}}"); do
 health=$(docker inspect --format '{{.State.Health.Status}}'
$container 2>/dev/null || echo "no-healthcheck")
 status=$(docker inspect --format '{{.State.Status}}' $container)
 echo "$container: $status ($health)"
done
```

```
echo -e "\n=== Resource Usage ==="
```

```
docker stats --no-stream --format "table
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}"
```

```
echo -e "\n=== Volume Usage ==="
```

```
docker system df
```

```
echo -e "\n=== Recent Errors ==="
```

```
docker events --filter type=container --filter event=die --since '1h'
--format "{{.Time}} {{.Actor.Attributes.name}} exited"
````
```

```
### Alerting Thresholds
```

- Memory usage > 80%
- CPU usage > 90% for 5+ minutes
- Container restart > 3 times in 1 hour
- Health check failures > 5 consecutive
- Disk usage > 85%
- Network errors > 1% of requests

```
## Advanced Debugging Tools

### Container Shell Access
```bash
Get shell in running container
docker exec -it CONTAINER sh

Run debugging container with same network
docker run -it --rm --network NETWORK nicolaka/netshoot

Debug container filesystem
docker run -it --rm -v VOLUME:/data alpine sh
```

### Network Analysis
```bash
Network packet capture
docker exec CONTAINER tcpdump -i eth0

Test external connectivity
docker exec CONTAINER curl -I external-api.com

DNS debugging
docker exec CONTAINER dig SERVICE_NAME
```

### Performance Profiling
```bash
Container resource limits
docker inspect CONTAINER | jq '.[0].HostConfig.Memory'

Process monitoring inside container
docker exec CONTAINER top

File system usage
docker exec CONTAINER du -sh /*
```
```

Remember: Systematic diagnosis beats random fixes every time!
EOF

Final Documentation and Summary

```
cat > ~/devops-tasks/logs/task-d7-troubleshooting-mastery.md << 'EOF'
# Task D7: Container Troubleshooting Mastery

## What I Mastered

### Essential Troubleshooting Skills
1. Systematic Diagnosis Approach
  - Symptoms identification → Information gathering → Hypothesis formation → Testing → Resolution
  - Container status analysis (docker ps, docker inspect)
  - Log analysis for error patterns
  - Resource monitoring and alerting

2. Common Issue Resolution
  - Startup Failures: Configuration errors, environment variables, dependency issues
  - Network Problems: Service discovery, DNS resolution, network isolation
  - Resource Exhaustion: OOM kills, CPU throttling, disk space issues
  - Data Persistence: Volume configuration, backup/restore, data integrity

3. Production Debugging Tools
  - Container inspection and analysis commands
  - Network connectivity testing
  - Performance monitoring and profiling
  - Log aggregation and error tracking

## Real-World Scenarios Practiced

### Scenario 1: Configuration Error (Backend Startup Failure)
```

- ****Problem:**** Typo in DATABASE_URL environment variable
- ****Symptoms:**** Container restart loop, DNS resolution errors
- ****Resolution:**** Fixed hostname in configuration, restarted service
- ****Prevention:**** Configuration validation, automated testing

Scenario 2: Network Isolation (Service Communication Failure)

- ****Problem:**** Backend disconnected from application network
- ****Symptoms:**** API timeouts, service unreachable errors
- ****Resolution:**** Reconnected to proper network, validated connectivity
- ****Prevention:**** Use docker-compose for network management

Scenario 3: Resource Exhaustion (OOM Kill)

- ****Problem:**** Container memory usage exceeded configured limits
- ****Symptoms:**** Exit code 137, sudden container termination
- ****Resolution:**** Analyzed memory requirements, adjusted limits appropriately
- ****Prevention:**** Capacity planning, memory usage monitoring

Scenario 4: Data Loss (Volume Misconfiguration)

- ****Problem:**** Database container created without persistent volume
- ****Symptoms:**** Data loss after container recreation
- ****Resolution:**** Configured proper volume mounting, restored from backup
- ****Prevention:**** Infrastructure as Code, backup verification

Key Diagnostic Commands Mastered

Container Health

| | |
|--------------------------|--------------------------|
| docker ps -a | # Container status |
| docker inspect CONTAINER | # Detailed configuration |
| docker logs CONTAINER | # Application logs |
| docker stats CONTAINER | # Resource usage |
| docker events | # System events |

Network Debugging

| | |
|---------------------------------|-------------------------|
| docker network inspect NETWORK | # Network configuration |
| docker exec CONTAINER ping HOST | # Connectivity testing |

```
docker exec CONTAINER nslookup SERVICE # DNS resolution
```

Resource Analysis

```
docker system df          # Disk usage
free -h                  # System memory
dmesg | grep -i killed    # OOM killer logs
docker events --filter event=oom # Container OOM events
```

Production Readiness Achievements

Incident Response Capability

- Systematic troubleshooting methodology
- Emergency response procedures
- Rollback and recovery strategies
- Communication and escalation protocols

Monitoring and Alerting

- Health check monitoring
- Resource usage tracking
- Error pattern recognition
- Performance baseline establishment

Documentation and Knowledge Transfer

- Incident response playbook
- Troubleshooting reference guide
- Common issues and solutions
- Prevention strategies and best practices

Troubleshooting Mindset Developed

Diagnostic Principles

1. ****Assume Nothing:**** Verify every assumption with data
2. ****Start Simple:**** Check obvious issues before complex solutions
3. ****Follow the Data:**** Logs and metrics tell the story
4. ****Think Systematically:**** Use structured approach, not random fixes
5. ****Document Everything:**** Knowledge sharing prevents future incidents

Production Debugging Philosophy

- ****Speed vs Accuracy:**** Balance quick fixes with proper diagnosis
- ****Impact Minimization:**** Stabilize first, root cause later
- ****Communication:**** Keep stakeholders informed throughout
- ****Learning:**** Every incident is a learning opportunity
- ****Prevention:**** Fix the process, not just the symptom

Real-World Impact

This troubleshooting mastery enables:

Faster Incident Resolution

- MTTR reduction from hours to minutes
- Systematic approach prevents random fixes
- Comprehensive toolkit for any container issue
- Confidence in production environments

Proactive Issue Prevention

- Understanding failure patterns
- Monitoring the right metrics
- Capacity planning and resource management
- Robust architecture decisions

Team Collaboration

- Structured incident response
- Knowledge sharing and documentation
- Training others on troubleshooting
- Building resilient systems

Business Value

- Reduced downtime and user impact
- Faster feature delivery confidence
- Lower operational costs
- Improved system reliability

Next-Level Skills Foundation

This troubleshooting foundation prepares for:

- Kubernetes cluster debugging
- Distributed system observability
- Advanced monitoring (Prometheus/Grafana)
- Site Reliability Engineering (SRE)
- Production operations at scale

Remember: Great troubleshooters are made, not born. Every incident makes you stronger!

EOF

```
# Commit all troubleshooting documentation
git add .
git commit -m "feat: complete container troubleshooting mastery with
incident response"
git push origin main
```

Acceptance Criteria

Essential Troubleshooting Skills:

- ☐ Diagnosed and fixed container startup failure (configuration error)
- ☐ Resolved network connectivity issues (service isolation)
- ☐ Handled resource exhaustion problem (OOM kill detection)
- ☐ Fixed data persistence issue (volume configuration)
- ☐ Created systematic troubleshooting methodology
- ☐ Documented incident response procedures
- ☐ Built comprehensive troubleshooting toolkit
- ☐ Understand common container failure patterns
- ☐ Can debug production issues independently
- ☐ Ready to handle real world container incidents

TASK D8: CI/CD Integration & Image Management

Ticket ID: DEVOPS-2411

Priority: High

Difficulty: Intermediate

Business Context

The H-Game application is containerized and working locally. We need automated CI/CD pipelines for production deployments.

Impact:

- Automated build and deployment process
- Consistent image versioning and tagging
- Integration with container registries
- Production-ready CI/CD workflow

Your Mission

Set up GitHub Actions CI/CD pipeline that:

- Automatically builds Docker images
- Tests code before building
- Pushes images to DockerHub
- Deploys to production on version tags
- Scans for security vulnerabilities

Step 1: Create GitHub Actions Workflow

Create the workflow file:

```
mkdir -p .github/workflows
```

```
#
=====
#
# Docker CI/CD Pipeline - Simple and Working
#
=====
```

```

=
# This workflow automates: testing → building → pushing → deploying
# It runs automatically when you push code or create version tags
#
=====
=

name: Docker CI/CD

#
=====
=
# WHEN DOES THIS WORKFLOW RUN? (Triggers)
#
=====
=
# Triggers define when GitHub Actions should start running this workflow
on:
  push:
    # Runs when code is pushed to these branches
    branches: [ main, develop ]
    # Also runs when you create version tags (like v1.0.0)
    tags: [ 'v*' ]
  pull_request:
    # Runs when someone creates a pull request to main branch
    # This lets you test changes before merging
    branches: [ main ]

#
=====
=
# GLOBAL ENVIRONMENT VARIABLES
#
=====
=
# These variables are available to ALL jobs in the workflow
# We define them here so we don't have to repeat them in each job
env:
  # Your DockerHub username (stored as GitHub secret for security)
  # Secrets are encrypted and never shown in logs
  DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
  # Image names - these must match what's in your docker-compose.yml

```

```

BACKEND_IMAGE: humor-game-backend
FRONTEND_IMAGE: humor-game-frontend

#
=====
=
# JOB 1: TEST - Run Tests Before Building
#
=====
=
# WHY: We test code BEFORE building Docker images to catch errors early
#   This saves time and resources - if tests fail, we don't build
# HOW: Runs on GitHub's free Ubuntu virtual machine
jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest # GitHub provides free Ubuntu runners

    steps:
      # Step 1: Get your code from GitHub repository
      # WHY: GitHub Actions needs your code to run tests
      # HOW: Downloads your repository to the virtual machine
      - name: Checkout code
        uses: actions/checkout@v4

      # Step 2: Set up Node.js (needed to run npm commands)
      # WHY: Your backend uses Node.js, so we need it installed
      # HOW: GitHub Actions installs Node.js version 18 and caches npm
      packages
      #   Caching makes future runs faster (doesn't re-download packages)
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '18' # Use Node.js version 18
          cache: 'npm' # Cache npm packages for faster installs
          cache-dependency-path: backend/package-lock.json # Where to find
      dependencies

      # Step 3: Install backend dependencies
      # WHY: Tests need dependencies (like testing libraries) to run
      # HOW: npm ci installs exact versions from package-lock.json (faster,
      more reliable than npm install)

```

```

- name: Install backend dependencies
  run: |
    cd backend
    npm ci

# Step 4: Run backend tests
# WHY: Verify that your backend code works correctly
# HOW: Runs the test script defined in backend/package.json
#       If tests fail, the workflow stops here (saves time)
- name: Run backend tests
  run: |
    cd backend
    npm test

# Step 5: Install frontend dependencies
# WHY: Frontend build needs dependencies
# HOW: Same as backend - installs from package-lock.json
- name: Install frontend dependencies
  run: |
    cd frontend
    npm ci

# Step 6: Build frontend (compile source code into production files)
# WHY: Frontend needs to be built before putting it in Docker image
# HOW: Runs npm run build which creates dist/ directory with optimized
files
- name: Build frontend
  run: |
    cd frontend
    npm run build

#
=====
=
# JOB 2: BUILD AND PUSH - Create Docker Images and Push to Registry
#
=====
=
# WHY: We need to build Docker images and push them to DockerHub so they
can be deployed
# HOW: Only runs if tests passed AND it's a push event (not a pull request)
build-and-push:

```

```

name: Build and Push Images
runs-on: ubuntu-latest
needs: test # Wait for test job to finish successfully
# Only run on push events (not pull requests)
# WHY: We don't want to push images for every PR, only for actual code
pushes
if: github.event_name == 'push'
  steps:
    # Step 1: Get your code (needed to access Dockerfiles)
    - name: Checkout code
      uses: actions/checkout@v4

    # Step 2: Set up Docker Buildx (advanced Docker builder)
    # WHY: Buildx allows building for multiple platforms and has better
    caching
    # HOW: GitHub Actions sets up Docker Buildx automatically
    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    # Step 3: Login to DockerHub (so we can push images)
    # WHY: DockerHub requires authentication to push images
    # HOW: Uses your DockerHub username and access token from GitHub secrets
    #       Access token is safer than password (can be revoked, has limited
    permissions)
    - name: Login to DockerHub
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN } # Access token, not
        password!

    # Step 4: Determine what tag to use for the images
    # WHY: Different tags for different scenarios:
    #       - Version tags (v1.0.0) → use the version number
    #       - Main branch → use "latest" (most recent stable)
    #       - Other branches → use branch name (develop, feature-branch, etc.)
    # HOW: Checks the git reference and sets appropriate tag
    - name: Determine image tag
      id: tag # Give this step an ID so we can use its output later
      run: |
        if [[ "${ github.ref }" == refs/tags/* ]]; then
          # If it's a version tag (v1.0.0), use the tag name

```

```

    TAG="{{ github.ref_name }}"
elif [[ "{{ github.ref }}" == refs/heads/main ]]; then
    # If it's main branch, use "latest"
    TAG="latest"
else
    # Otherwise, use branch name (develop, feature-branch, etc.)
    TAG="{{ github.ref_name }}"
fi
# Save the tag so other steps can use it
echo "tag=${TAG}" >> $GITHUB_OUTPUT
echo "Using tag: ${TAG}"

# Step 5: Build and push backend Docker image
# WHY: Backend needs to be containerized and available on DockerHub for
deployment
# HOW:
#     - context: Where to find Dockerfile and source code
#     - file: Which Dockerfile to use
#     - push: true means push to DockerHub (false = only build locally)
#     - tags: What to name the image
(yourusername/humor-game-backend:tag)
#     - cache: Uses GitHub Actions cache to speed up builds (reuses
layers)
- name: Build and push backend
  uses: docker/build-push-action@v5
  with:
    context: ./backend # Directory containing Dockerfile and source code
    file: ./backend/Dockerfile # Which Dockerfile to use
    push: true # Push to DockerHub (false = only build, don't push)
    tags: "{{ env.DOCKERHUB_USERNAME }}" / "{{ env.BACKEND_IMAGE }}" : "{{
steps.tag.outputs.tag }}"
    # Cache layers for faster builds
    # WHY: If code didn't change, reuse previous build layers (saves time)
    cache-from: type=gha # Get cache from GitHub Actions
    cache-to: type=gha,mode=max # Save cache to GitHub Actions

# Step 6: Build and push frontend Docker image
# WHY: Frontend also needs to be containerized and available on DockerHub
# HOW: Same as backend - builds from frontend/Dockerfile and pushes to
DockerHub
- name: Build and push frontend
  uses: docker/build-push-action@v5

```



```

    with:
      context: ./frontend
      file: ./frontend/Dockerfile
      push: true
      tags: ${{ env.DOCKERHUB_USERNAME }}/${{ env.FRONTEND_IMAGE }}:${{
steps.tag.outputs.tag }}
      cache-from: type=gha
      cache-to: type=gha,mode=max

#
=====
=
# JOB 3: DEPLOY TO PRODUCTION - Deploy to Live Environment
#
=====
=
# WHY: After building and pushing images, we need to deploy them to
production
# HOW: Only runs when you create a version tag (like v1.0.0)
#       This ensures only tested, versioned releases go to production
deploy-production:
  name: Deploy to Production
  runs-on: ubuntu-latest
  needs: build-and-push # Wait for images to be built and pushed first
  # Only run on version tags (v1.0.0, v1.2.3, etc.)
  # WHY: We don't want to deploy every push - only specific versions
  #       This gives you control over what goes to production
  if: startsWith(github.ref, 'refs/tags/v')
  environment: production # GitHub environment (can have approval
requirements)
  steps:
    # Step 1: Get your code (needed to access docker-compose files)
    - name: Checkout code
      uses: actions/checkout@v4

    # Step 2: Set up SSH connection to production server
    # WHY: We need to connect to your production server to deploy
    # HOW: Uses SSH private key from GitHub secrets to authenticate
    - name: Set up SSH
      uses: webfactory/ssh-agent@v0.9.0
      with:
        ssh-private-key: ${{ secrets.PRODUCTION_SSH_KEY }}

```

```

# Step 3: Deploy to production server
# WHY: This is where the actual deployment happens
# HOW:
#     1. Connects to production server via SSH
#     2. Pulls new Docker images from DockerHub
#     3. Updates docker-compose.yml with new image tags
#     4. Deploys using docker-compose (merges docker-compose.yml +
docker-compose.prod.yml)
#     5. Runs health checks to verify deployment worked
#     6. Rolls back if health checks fail
- name: Deploy
  env:
    # These environment variables come from GitHub secrets
    PROD_SERVER: "${{ secrets.PRODUCTION_SERVER }}" # Server hostname/IP
    PROD_USER: "${{ secrets.PRODUCTION_USER }}" # SSH username (e.g.,
ubuntu, ec2-user)
    PROD_PATH: "${{ secrets.PRODUCTION_DEPLOY_PATH }}" # Path on server
(e.g., /home/ubuntu/app)
  run: |
    # Get the version from the tag (v1.0.0 -> 1.0.0)
    VERSION="${{ github.ref_name }}"
    # Build full image names with DockerHub username
    BACKEND_IMAGE="${{ env.DOCKERHUB_USERNAME }}/${{ env.BACKEND_IMAGE
}}:${VERSION}"
    FRONTEND_IMAGE="${{ env.DOCKERHUB_USERNAME }}/${{ env.FRONTEND_IMAGE
}}:${VERSION}"

    # Connect to production server and run deployment commands
    ssh ${PROD_USER}@${PROD_SERVER} << EOF
      set -e # Exit immediately if any command fails

      cd ${PROD_PATH} # Go to your application directory

      # Login to DockerHub (needed to pull private images)
      echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login -u "${{
secrets.DOCKERHUB_USERNAME }}" --password-stdin

      # Pull new images from DockerHub
      # WHY: Get the latest versioned images we just built
      docker pull ${BACKEND_IMAGE}
      docker pull ${FRONTEND_IMAGE}

```

```

    # Update docker-compose.yml with new image tags
    # WHY: docker-compose.yml currently has old image names
(humor-game-backend:v1)
    #       We need to update it to use the new versioned images from
DockerHub
    # HOW: sed replaces the old image names with new ones
    #       -i.bak creates a backup file (for rollback if needed)
    sed -i.bak "s|image:.*humor-game-backend.*|image:
${BACKEND_IMAGE}|g" docker-compose.yml
    sed -i.bak "s|image:.*humor-game-frontend.*|image:
${FRONTEND_IMAGE}|g" docker-compose.yml

    # Deploy using both docker-compose files
    # WHY: docker-compose.yml has service definitions
    #       docker-compose.prod.yml adds production settings (resource
limits, logging)
    #       Using both files merges them together
    # HOW: -f flag specifies which files to use
    #       pull: Downloads the images we just updated
    #       up -d: Starts containers in detached mode (runs in
background)
    docker-compose -f docker-compose.yml -f docker-compose.prod.yml pull
backend frontend
    docker-compose -f docker-compose.yml -f docker-compose.prod.yml up
-d backend frontend

    # Wait for services to start
    # WHY: Containers need time to start and become healthy
    sleep 30

    # Health checks - verify deployment worked
    # WHY: We need to make sure the services are actually running and
responding
    # HOW:
    #       - Backend: Check /health endpoint
    #       - Frontend: Check if homepage loads
    #       If both pass, deployment is successful
    #       If either fails, rollback to previous version
    if docker-compose -f docker-compose.yml -f docker-compose.prod.yml
exec -T backend wget --spider -q --timeout=5 http://127.0.0.1:3001/health
&& \

```

```

        docker-compose -f docker-compose.yml -f docker-compose.prod.yml
exec -T frontend curl -f http://localhost/ > /dev/null 2>&1; then
    echo "✅ Deployment successful"
else
    echo "❌ Health check failed, rolling back..."
    # Restore backup (previous version)
    mv docker-compose.yml.bak docker-compose.yml
    # Restart with previous version
    docker-compose -f docker-compose.yml -f docker-compose.prod.yml up
-d
    exit 1 # Exit with error so GitHub Actions knows deployment
failed
    fi
EOF

#
=====
=
# WORKFLOW SUMMARY - How It All Works Together
#
=====
=
# 1. Developer pushes code or creates version tag → Triggers workflow
# 2. Test job runs → Verifies code works (if tests fail, workflow stops)
# 3. Build job runs → Creates Docker images, pushes to DockerHub
# 4. Deploy job runs → Deploys to production server (only on version tags)
#
# If any job fails, workflow stops (prevents bad code from deploying)
#
=====
=

```

The workflow file is already created at [all code files in repo.github/workflows/docker-ci-cd.yml](#). It includes:

- **Test job:** Runs tests before building
- **Build job:** Builds and pushes images to DockerHub (includes security scanning)
- **Deploy job:** Deploys **only** when a version tag is pushed (e.g., **v1.0.0**)

Local vs Production Deployment (Important)

This CI/CD pipeline is designed in two parts:

- **CI (Active):** tests, builds, and pushes Docker images to DockerHub
- **CD (Prepared):** production deployment via SSH (currently commented out)

Why SSH Deployment Is Commented Out

If you are running this project locally on your MacBook, not on a remote server (EC2/VPS). Because there is no production server yet, the SSH deployment step is intentionally commented, not removed.

This allows you to:

- Learn real-world CI/CD structure
- Build and push images automatically
- Run containers locally using Docker Compose

When to Enable It

Uncomment the SSH deployment job only when you have:

- A remote Linux server
- SSH access and GitHub secrets configured
- Docker and Docker Compose installed on the server

At that point, the pipeline becomes full CI → CD → Production.

Key idea: CI is active now. CD is ready for later.

Step 2: Set Up DockerHub

Create DockerHub Account

1. Go to hub.docker.com
2. Sign up for a free account
3. Verify your email

Create Repositories

Create two public repositories:

- [your-username/humor-game-backend](#)
- [your-username/humor-game-frontend](#)

Note: Use [humor-game-backend](#) and [humor-game-frontend](#) (not [h-game-backend](#)) to match your docker-compose.yml.

Generate Access Token

1. **DockerHub** → **Account Settings** → **Security**
2. Create new Access Token
3. Name: "GitHub Actions CI/CD"
4. Permissions: Read, Write, Delete
5. **Copy the token** (you won't see it again!)

Why tokens instead of passwords?

- More secure (can be revoked independently)
- Better audit trail
- Required for organizations with SSO

Step 3: Configure GitHub Secrets

In your GitHub repository:

1. Go to **Settings** → **Secrets and variables** → **Actions**
2. Click **New repository secret**
3. Add these secrets:

| Secret Name | Value |
|---------------------------------|-----------------------------|
| <code>DOCKERHUB_USERNAME</code> | Your DockerHub username |
| <code>DOCKERHUB_TOKEN</code> | Your DockerHub access token |

Important: Never commit secrets to your repository. Always use GitHub Secrets.

Step 4: Understanding Image Tagging

Your CI/CD pipeline creates different tags for different scenarios:

Branch-Based Tags

- `veeno/humor-game-backend:latest` - Main branch (most recent stable)
- `veeno/humor-game-backend:develop` - Develop branch
- `veeno/humor-game-backend:main-abc1234` - Branch + commit SHA

Version Tags

When you create a git tag like `v1.0.0`:

- `veeno/humor-game-backend:v1.0.0` - Exact version
- `veeno/humor-game-backend:1.0` - Minor version
- `veeno/humor-game-backend:1` - Major version

Production Strategy

- **Development:** Use `develop` tag
- **Staging:** Use `main` or `latest` tag
- **Production:** Use specific version tags only (`v1.0.0`)

Step 5: Test the CI/CD Pipeline

Commit and Push

```
git add .github/workflows/docker-ci-cd.yml
git commit -m "feat: add Docker CI/CD pipeline"
git push origin main
```

Monitor Execution

1. Go to your GitHub repository
2. Click the **Actions** tab
3. Watch the workflow run

Expected Flow:

1. **Test Job** (5-10 min) - Runs tests to verify code works
2. **Build Job** (10-15 min) - Builds images, scans for vulnerabilities, pushes to DockerHub
3. **Deploy Job** (only on version tags) - Deploys to production **only** when you push a version tag

The screenshot displays the GitHub Actions interface for the repository 'Osomudeya / humour-game-playbook'. The 'Actions' tab is selected, showing a workflow named 'Docker CI/CD' with a job 'cicd #3' that has completed successfully. The left sidebar contains a 'Summary' section with links to 'All jobs', 'Run Tests', 'Build and Push Images' (the active job), 'Run details', 'Usage', and 'Workflow file'. The main area shows the 'Build and Push Images' job, which succeeded in 53 seconds. A list of steps is shown, including 'Set up job', 'Checkout code', 'Set up Docker Buildx', 'Login to DockerHub', and 'Determine image tag'. The 'Build and push backend' step is expanded, showing a terminal log with commands like 'Run docker/build-push-action@v5', 'GitHub Actions runtime token ACs', 'Docker info', 'Proxy configuration', 'Buildx version', 'Builder info', and a 'docker buildx build' command with various flags.

Step 6: Deploy to Production

Create Version Tag

```
# Create a version tag
git tag -a v1.0.0 -m "Release version 1.0.0"

# Push the tag (this triggers production deployment)
git push origin v1.0.0
```

What Happens

1. Workflow builds images with tag `v1.0.0`
2. Images are scanned for security vulnerabilities
3. Images are pushed to DockerHub
4. Production deployment job runs **only** for version tags
5. Server pulls new images
6. Containers are updated
7. Health checks verify deployment succeeded

Step 7: Security Scanning

The workflow includes Trivy security scanning that:

- Scans images for vulnerabilities
- Reports CRITICAL and HIGH severity issues
- Uploads results to GitHub Security tab

View Results:

- Go to your repository → **Security** tab
- Click **Code scanning alerts**

Step 8: Image Verification (SHA Digests)

Understanding Digests

- Tags are mutable - `latest` can change

- **Digests are immutable** - Never change for the same image

Get Image Digest

```
# Get digest of an image
docker inspect veeno/humor-game-backend:latest --format='{{index
.RepoDigests 0}}'
```

Use Digests in Production

```
# Pull by digest (more secure)
docker pull veeno/humor-game-backend@sha256:abc123...
```

Why use digests?

- Ensures you get the exact same image
- Prevents accidental updates
- Better for production deployments

Step 9: Private vs Public Repositories

Public Repositories (Current Setup)

Pros:

- Free unlimited pulls
- Easy sharing
- Good for learning

Cons:

- Code visible to everyone
- Rate limiting (100 pulls/6hrs for anonymous)
- No access control

Private Repositories

Pros:

- Code protected
- Access control
- No rate limiting
- Better for production

Cons:

- Cost (after free tier: 1 private repo free)
- Authentication required

When to Use Each

- **Public:** Learning, open source, demos
- **Private:** Production, proprietary code, enterprise

Step 10: Production Deployment Process**Manual Deployment (If Needed)**

```
# Pull specific version
docker pull veeno/humor-game-backend:v1.0.0
docker pull veeno/humor-game-frontend:v1.0.0

# Update docker-compose.yml with new image tags
sed -i "s|image:.*humor-game-backend.*|image:
veeno/humor-game-backend:v1.0.0|g" docker-compose.yml
sed -i "s|image:.*humor-game-frontend.*|image:
veeno/humor-game-frontend:v1.0.0|g" docker-compose.yml

# Deploy using both compose files
docker-compose -f docker-compose.yml -f docker-compose.prod.yml pull
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d

# Verify health
curl http://localhost/health
curl http://localhost:3001/health
```

Automated Deployment (Recommended)

The CI/CD pipeline handles this automatically when you push a version tag.

Step 11: Using `docker-compose.prod.yml`

Your `docker-compose.prod.yml` adds production settings:

- Resource limits (CPU, memory)
- Log rotation
- Production logging configuration

Deploy with both files:

```
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

This merges both files:

- `docker-compose.yml` - Service definitions
- `docker-compose.prod.yml` - Production overrides

Step 12: Validate and Monitor CI/CD

Monitor Pipeline Execution

In GitHub:

1. Go to your repository → **Actions** tab
2. Click on a workflow run to see:
 - Which jobs ran
 - Which jobs passed/failed
 - Execution time for each job
 - Logs for each step

Using GitHub CLI:

```
# List recent workflow runs
gh run list

# View details of a specific run
gh run view <run-id>
```

```
# Watch a running workflow  
gh run watch
```

Validate Deployment

Check if images were pushed:

```
# View your images on DockerHub  
# Go to:  
https://hub.docker.com/r/your-username/humor-game-backend/tags  
  
# Or check locally  
docker images | grep humor-game
```

Verify production deployment:

```
# Check if containers are running  
docker ps | grep humor-game  
  
# Test health endpoints  
curl http://localhost/health           # Frontend  
curl http://localhost:3001/health      # Backend API
```

Common Monitoring Checks

Pipeline Status:

- Green checkmark = Success
- Red X = Failed (check logs)
- Yellow circle = In progress

What to Monitor:

- Build time (should be consistent)
- Test pass rate (should be 100%)
- Security scan results (review vulnerabilities)
- Deployment success rate

Quick Health Check Script:

```
# Simple validation script
echo "=== CI/CD Health Check ==="
echo "Recent workflow runs:"
gh run list --limit 3
echo ""
echo "Container status:"
docker ps --format "table {{.Names}}\t{{.Status}}" | grep -E
"NAME|humor-game"
echo ""
echo "Service health:"
curl -s http://localhost/health && echo "  Frontend healthy" ||
echo "  Frontend down"
curl -s http://localhost:3001/health && echo "  Backend healthy" ||
echo "  Backend down"
```

Acceptance Criteria Checklist

- ☐ GitHub Actions workflow created and working
- ☐ DockerHub repositories created ([humor-game-backend](#), [humor-game-frontend](#))
- ☐ GitHub secrets configured ([DOCKERHUB_USERNAME](#), [DOCKERHUB_TOKEN](#))
- ☐ Images successfully pushed to DockerHub
- ☐ Multiple tags working (latest, branch, version)
- ☐ Security scanning integrated (Trivy)
- ☐ Production deployment working (on version tags)
- ☐ Understand private vs public repositories
- ☐ Can verify images using SHA digests

Quick Reference

Common Commands

```
# View workflow runs
gh run list

# View workflow logs
gh run view <run-id>

# Check DockerHub images
docker images | grep humor-game

# Get image digest
docker inspect veeno/humor-game-backend:latest --format='{{index
.RepoDigests 0}}'

# Deploy specific version
git tag v1.0.0 && git push origin v1.0.0
```

Troubleshooting

Workflow fails at DockerHub login:

- Check `DOCKERHUB_TOKEN` secret is correct
- Verify token has Read, Write, Delete permissions

Images not found:

- Verify repository names match (humor-game-backend, not h-game-backend)
- Check DockerHub username in secrets

Deployment fails:

- Check SSH key is correct
- Verify server details in secrets
- Check docker-compose files exist on server

What Can Go Wrong (And How to Fix It)

This section covers common failures you'll encounter. Knowing these shows real-world experience.

1. Pipeline Fails: "DockerHub authentication failed"

Symptoms:

- Build job fails with "unauthorized" or "authentication required"
- Images not pushed to DockerHub

Common Causes:

- Token expired or revoked
- Wrong token in GitHub secrets
- Token doesn't have Write permissions

Fix:

```
# 1. Generate new token in DockerHub
# 2. Update GitHub secret: DOCKERHUB_TOKEN
# 3. Re-run the workflow
```


Prevention: Set token expiration reminder, use separate tokens per environment

2. Pipeline Fails: "Image name mismatch"

Symptoms:

- Build succeeds but deployment can't find image
- Error: "manifest unknown" or "repository not found"

Common Causes:

- Repository name typo (h-game vs humor-game)
- Wrong DockerHub username in secrets
- Image tag doesn't exist

Fix:

```
# Verify repository exists
curl
https://hub.docker.com/v2/repositories/your-username/humor-game-backe
nd/

# Check what tags exist
docker images | grep humor-game

# Verify secrets match
echo $DOCKERHUB_USERNAME # Should match your DockerHub username
```

Prevention: Use environment variables consistently, validate before deployment

3. Tests Pass Locally But Fail in CI/CD

Symptoms:

- Tests work on your machine
- GitHub Actions test job fails
- Different Node.js versions or missing dependencies

Common Causes:

- Node.js version mismatch
- Missing environment variables
- Different package-lock.json state

Fix:

```
# Ensure package-lock.json is committed
git add package-lock.json

# Match Node.js version in workflow
# .github/workflows/docker-ci-cd.yml should use same version as local
node-version: '18' # Match your local version
```

Prevention: Use `.nvmrc` file, always commit `package-lock.json`

4. Security Scan Finds Vulnerabilities

Symptoms:

- Trivy scan reports CRITICAL or HIGH vulnerabilities
- Workflow continues but shows warnings

Common Causes:

- Outdated base images
- Vulnerable dependencies
- Missing security patches

Fix:

```
# Update base image in Dockerfile
FROM node:18-alpine # Use latest 18-alpine

# Update dependencies
npm audit fix

# Rebuild and scan again
docker build -t test-image .
trivy image test-image
```

Prevention: Regular dependency updates, use minimal base images, automated scanning

5. Deployment Succeeds But Service Unhealthy

Symptoms:

- Deployment completes
- Health checks fail
- Containers restarting

Common Causes:

- Database connection issues
- Missing environment variables
- Port conflicts
- Health check endpoint wrong

Fix:

```
# Check container logs
docker logs backend
docker logs frontend

# Verify environment variables
docker-compose exec backend env | grep DATABASE

# Test health endpoint manually
docker-compose exec backend wget -O- http://127.0.0.1:3001/health

# Check if ports are available
netstat -tulpn | grep 3001
```

Prevention: Test health checks locally, validate environment variables, check logs before deployment

6. Images Build But Push Fails

Symptoms:

- Build job succeeds
- Push step fails with "denied" or "forbidden"

Common Causes:

- Repository doesn't exist on DockerHub
- Token doesn't have Write permission
- Rate limiting (free tier)

Fix:

```
# Create repository on DockerHub first
# Go to hub.docker.com → Create repository

# Verify token permissions
# DockerHub → Account Settings → Security → Check token
permissions
```

```
# Check rate limits
# Free tier: 200 pulls per 6 hours
# Upgrade to Pro for unlimited
```

Prevention: Create repositories before first push, monitor rate limits

7. Version Tag Doesn't Trigger Deployment

Symptoms:

- Created tag `v1.0.0` and pushed
- Build job runs but deploy job doesn't

Common Causes:

- Tag format wrong (must start with `v`)
- Deploy job condition not matching
- Workflow file not in correct branch

Fix:

```
# Verify tag format
git tag -l # Should show v1.0.0, not 1.0.0

# Check workflow condition
# Should be: if: startsWith(github.ref, 'refs/tags/v')

# Ensure workflow file is in main branch
git checkout main
git log --oneline .github/workflows/docker-ci-cd.yml
```

Prevention: Use semantic versioning consistently, test tag format locally

8. Deployment Rollback Doesn't Work

Symptoms:

- Health check fails
- Rollback script runs but service still broken

Common Causes:

- Backup file missing or corrupted
- Previous image no longer available
- Rollback script has errors

Fix:

```
# Verify backup exists
ls -la docker-compose.yml.bak

# Check previous image still exists
docker images | grep humor-game-backend

# Manual rollback
git checkout HEAD~1 -- docker-compose.yml
docker-compose up -d
```

Prevention: Keep multiple image versions, test rollback process, version control docker-compose.yml

Debugging Strategy

When something goes wrong:

1. **Check the logs** - GitHub Actions logs show exactly what failed
2. **Reproduce locally** - Try the same command on your machine
3. **Verify secrets** - Ensure all secrets are set correctly
4. **Check permissions** - DockerHub tokens, SSH keys, etc.
5. **Test incrementally** - Break down the problem into smaller parts

Pro Tip: Always test your pipeline with a small change first, not a major release.

Summary

You've now implemented:

- Automated CI/CD pipeline
- DockerHub integration
- Image versioning strategy
- Security scanning
- Production deployment automation

Next Steps:

- Monitor deployments in GitHub Actions
- Review security scan results
- Set up alerts for failed deployments
- Consider private repositories for production

Key Learnings

1. **CI/CD automates** the build, test, and deploy process
2. **Image tags** help manage different versions
3. **SHA digests** provide immutable image references
4. **Security scanning** catches vulnerabilities early
5. **Version tags** control what goes to production

Remember: The workflow runs automatically when you push code or create version tags. You don't need to manually build and push images anymore!

Image Scanning Explained

What is Image Scanning?

Think of Docker images like shipping containers. Before you ship a container, you want to check if it has any dangerous items inside. Image scanning does the same thing, it checks your Docker images for known security vulnerabilities (like bugs or weaknesses that hackers could exploit).

Why Scan Before Pushing?

The Problem: If you push a vulnerable image to DockerHub, anyone who pulls it gets the vulnerabilities too. It's like shipping a container with dangerous items, bad for everyone.

The Solution: Scan the image before pushing it. If vulnerabilities are found, the workflow stops and the image never gets pushed. Only safe images make it to DockerHub.

How It Works in Your Workflow

Your CI/CD pipeline now follows this simple flow:

1. Build image → Create the Docker image locally
2. Scan image → Check for CRITICAL and HIGH severity vulnerabilities
3. Push image → Only if scan passes (no dangerous vulnerabilities found)

What Happens:

- **If scan passes:** Image is pushed to DockerHub (safe to use)
- **If scan fails:** Workflow stops, image is NOT pushed (keeps vulnerabilities out)

See It in Action (From Your Workflow)

Here's the actual code that scans your images:

```
# Step 6: Scan backend image for vulnerabilities
- name: Scan backend image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.DOCKERHUB_USERNAME }/${ env.BACKEND_IMAGE
    }}:${ steps.tag.outputs.tag }}
    severity: 'CRITICAL,HIGH'
    exit-code: '1' # Fail workflow if vulnerabilities found
```

What this does:

- Scans the image you just built
- Only reports CRITICAL and HIGH severity issues (ignores minor ones)
- Stops the workflow (`exit-code: '1'`) if dangerous vulnerabilities are found

Image Tags: Managing Different Versions

What are tags? Tags are like labels on your Docker images. They help you identify which version you're using.

Your Workflow's Tagging Strategy

Your CI/CD automatically tags images based on what you push:

```
# From your workflow - determines the tag
if [[ "${ github.ref }" == refs/tags/* ]]; then
  TAG="${ github.ref_name }"          # v1.0.0 → tag is "v1.0.0"
elif [[ "${ github.ref }" == refs/heads/main ]]; then
  TAG="latest"                        # main branch → tag is
  "latest"
else
  TAG="${ github.ref_name }"          # develop branch → tag is
  "develop"
fi
```

Real Examples from our Images

When you push code, your images get tagged like this:

| What You Push | Image Tag | Full Image Name |
|-------------------------------------|----------------------|--|
| Push to <code>main</code> branch | <code>latest</code> | <code>yourusername/humor-game-backend:latest</code> |
| Create tag <code>v1.0.0</code> | <code>v1.0.0</code> | <code>yourusername/humor-game-backend:v1.0.0</code> |
| Push to <code>develop</code> branch | <code>develop</code> | <code>yourusername/humor-game-backend:develop</code> |

Why this matters:

- `latest` = most recent stable version
- `v1.0.0` = specific version you can always go back to
- `develop` = work-in-progress version

Pulling Images with Tags

```
# Pull the latest version
docker pull yourusername/humor-game-backend:latest

# Pull a specific version
docker pull yourusername/humor-game-backend:v1.0.0

# Pull development version
docker pull yourusername/humor-game-backend:develop
```

SHA Digests: Immutable Image References

What is a SHA digest? It's a unique fingerprint for your image. Even if the tag changes, the SHA stays the same.

Understanding SHA vs Tags

Tags can change:

```
# Today, "latest" might point to version 1.0
yourusername/humor-game-backend:latest # Could be v1.0

# Tomorrow, "latest" might point to version 2.0
yourusername/humor-game-backend:latest # Now it's v2.0 (different
image!)
```

SHA digests never change:

```
# This SHA always points to the exact same image, forever
yourusername/humor-game-backend@sha256:abc123def456...
# Even if you push a new "latest", this SHA still points to the old
image
```

Why Use SHA Digests?

Problem with tags:

- **latest** tag can change → You might get a different image than expected
- Tags can be overwritten → Hard to track what you're actually running

Solution with SHA:

- SHA is permanent → Always points to the exact same image
- Immutable reference → Perfect for production deployments

Example: Tag vs SHA

```
# Using tag (can change)
docker pull yourusername/humor-game-backend:latest
```

```
# ⚠ Tomorrow, "latest" might be a different image!

# Using SHA (never changes)
docker pull yourusername/humor-game-backend@sha256:abc123def456789...
# ✅ This always pulls the exact same image, forever
```

Finding Your Image's SHA

After pushing, DockerHub shows the SHA:

```
# View image details on DockerHub or use:
docker inspect yourusername/humor-game-backend:latest | grep -i
sha256
```

Output:

```
"RepoDigests": [
  "yourusername/humor-game-backend@sha256:abc123def456789..."
]
```

Where to See Results

After a workflow runs, check your GitHub repository:

1. Go to **Security** tab
2. Click **Code scanning alerts**
3. See all vulnerabilities Trivy found (if any)

Visual Example of Scan Results

When Trivy finds vulnerabilities, you'll see:

```
🔍 Scanning: yourusername/humor-game-backend:latest

❌ CRITICAL: CVE-2024-1234 in package: openssl
Description: Remote code execution vulnerability
Fixed in version: 1.1.1t
```

```

! HIGH: CVE-2024-5678 in package: node
  Description: Denial of service vulnerability
  Fixed in version: 18.17.0

✓ Scan complete: 2 vulnerabilities found
✗ Workflow failed: Image not pushed to DockerHub

```

Real-World Example

Imagine you're building a house (your Docker image):

- **Without scanning:** You build it, ship it, and later discover it has weak foundations (vulnerabilities). Too late - it's already shipped!
- **With scanning:** You build it, inspect it for problems, fix any issues found, THEN ship it. Only safe houses get shipped.

That's exactly what Trivy does, it inspects your Docker images before they're "shipped" to DockerHub.

Quick Reference: Your Workflow Steps

Here's what happens in order when you push code:

```

# 1. Build backend image (locally, not pushed yet)
- name: Build backend image
  push: false # ← Not pushed yet!

# 2. Scan backend image
- name: Scan backend image with Trivy
  severity: 'CRITICAL,HIGH' # ← Only checks serious issues

# 3. Upload scan results to GitHub
- name: Upload backend Trivy results to GitHub Security
  # ← Results appear in Security tab

# 4. Push backend image (only if scan passed)
- name: Push backend image to DockerHub
  push: true # ← Only reaches here if no CRITICAL/HIGH
vulnerabilities

```

```
# Same process for frontend...
```

Key Takeaways

- **Image Scanning** = Security check before pushing
Image Tags = Version labels (**latest**, **v1.0.0**, **develop**)
SHA Digests = Permanent, immutable image references
Scan Before Push = Vulnerable images never reach DockerHub
Results in GitHub = Check Security tab → Code scanning alerts

Common Commands

```
# Pull latest image
docker pull yourusername/humor-game-backend:latest

# Pull specific version
docker pull yourusername/humor-game-backend:v1.0.0

# Pull by SHA (most reliable)
docker pull yourusername/humor-game-backend@sha256:abc123...

# Check image SHA
docker inspect yourusername/humor-game-backend:latest | grep
RepoDigests

# View scan results
# Go to: GitHub repo → Security → Code scanning alerts
```